



Εισαγωγή στην Παράλληλη Επεξεργασία

Αθανάσιος Ι. Μάργαρης

**ΚΕΦΑΛΑΙΟ 4 Προγραμματισμός σε Συστήματα
Κοινόχρηστης Μνήμης**

Copyright © Εκδόσεις TZIOΛA, 2024. Με επιφύλαξη παντός νόμιμου δικαιώματος
ISBN 978-618-221-094-9



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η διαχείριση των διεργασιών στα λειτουργικά συστήματα

Η **διεργασία** αποτελεί μία από τις **πιο κεντρικές έννοιες** των **λειτουργικών συστημάτων** και μπορεί να οριστεί με **πολλούς διαφορετικούς τρόπους**, μερικοί από τους οποίους είναι οι εξής:

- ένα **πρόγραμμα** που **εκτελείται**
- ένα **στιγμιότυπο προγράμματος** που εκτελείται **ανεξάρτητα** από τα υπόλοιπα
- μία **ακολουθία εντολών** που βρίσκεται **υπό καθεστώς εκτέλεσης**, χρησιμοποιώντας τους **δικούς της πόρους**
- μία **ενότητα** ή **οντότητα**, που έχει ανατεθεί στον **επεξεργαστή** προκειμένου να **εκτελεστεί** από αυτόν

Κατά την **εκκίνηση** μιας **διεργασίας**, το λειτουργικό σύστημα εκχωρεί σε αυτή, τους **κατάλληλους υπολογιστικούς πόρους** οι οποίοι σε γενικές γραμμές περιλαμβάνουν

- Μία **περιοχή μνήμης** στην **κεντρική μνήμη του συστήματος** στην οποία **φορτώνεται** το εκτελέσιμο αρχείο
- Χώρος στον **σκληρό δίσκο** για την **ανάγνωση** και **εγγραφή** των εμπλεκόμενων αρχείων
- **Χρόνος εκτέλεσης** στην **κεντρική μονάδα επεξεργασίας**
- **Δικτυακοί πόροι** εάν η λειτουργία της διεργασίας απαιτεί **πρόσβαση σε δίκτυο**

Οι παραπάνω πόροι **δεσμεύονται** κατά την **υποβολή** της εργασίας στο σύστημα εφόσον **εγκριθεί η εκτέλεσή της** και **αποδεσμεύονται** κατά τον **τερματισμό** της.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η διαχείριση των διεργασιών στα λειτουργικά συστήματα

Όλα τα **μοντέρνα λειτουργικά συστήματα** υποστηρίζουν τον **πολυπρογραμματισμό** που συνίσταται στην **ταυτόχρονη εκτέλεση** πολλών διεργασιών από έναν **επεξεργαστή** και στηρίζεται στην υλοποίηση **αλγορίθμων χρονοδρομολόγησης**, που προσδιορίζουν την **σειρά** και τις **συνθήκες ανάθεσης** του επεξεργαστή στις διεργασίες.

Αν και σε κάθε χρονική στιγμή, είναι δυνατή η εκτέλεση **μιας και μοναδικής διεργασίας**, ωστόσο, στην πράξη, η **εναλλαγή** των **διεργασιών** στον **επεξεργαστή** προκειμένου να εκτελεστούν η μία μετά την άλλη, είναι **τόσο γρήγορη**, ώστε οι χρήστες να διατηρούν την **ψευδαίσθηση**, πως σε κάθε χρονική στιγμή εκτελούνται **ταυτόχρονα** πολλές διεργασίες.

Αυτή η **ταυτόχρονη εκτέλεση** είναι δυνατή μόνο σε ένα **σύστημα πολυεπεξεργασίας**, όπως είναι ένας **πολυπύρηνος επεξεργαστής**, στον οποίο ο κάθε **πυρήνας** διαθέτει τους δικούς του **καταχωρητές** και την δική του **ALU**.

Θα πρέπει να ληφθεί μέριμνα, ώστε οι **ανεξάρτητες μεταξύ τους** διεργασίες, που μπορούν να ανήκουν τόσο **στον ίδιο** χρήστη, όσο και σε **διαφορετικούς χρήστες**, να **μην** αλληλοεπιδρούν μεταξύ τους, εκτός εάν κάτι τέτοιο είναι **επιθυμητό**.

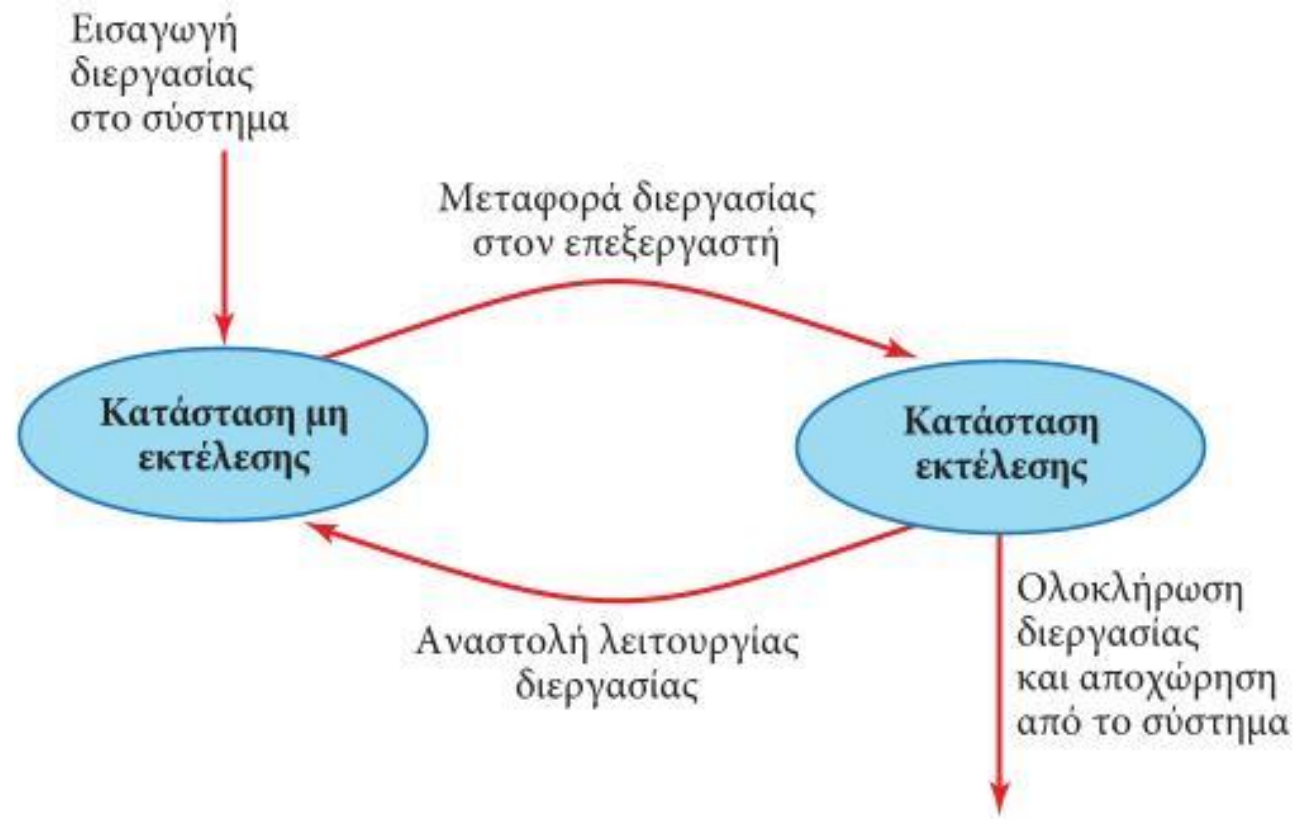
Για παράδειγμα, εάν σε κάποια διεργασία εμφανιστεί κάποιο **σφάλμα**, αυτό **δεν θα πρέπει** σε καμία περίπτωση, να επηρεάσει την λειτουργία των **υπόλοιπων** διεργασιών.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Σε ένα **λειτουργικό σύστημα** η κάθε διεργασία χαρακτηρίζεται από έναν **κύκλο ζωής** κατά τη διάρκεια του οποίου και καθώς οδεύει **προς την ολοκλήρωσή της**, υφίσταται **διαδοχικές μεταβάσεις** ανάμεσα σε ένα **συγκεκριμένο και πεπερασμένο πλήθος καταστάσεων**, με αυτή την συμπεριφορά να περιγράφεται από το επονομαζόμενο **διάγραμμα καταστάσεων**



Στο πιο σύνθετο μοντέλο των **πέντε καταστάσεων**, η κάθε διεργασία κατά τη διάρκεια του **κύκλου ζωής** της, βρίσκεται σε μία από τις καταστάσεις

1. **Νέα**
2. **Έτοιμη**
3. **Σε εκτέλεση**
4. **Μπλοκαρισμένη**
5. **Ολοκληρωμένη**

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Μία διεργασία χαρακτηρίζεται ως **Νέα** όταν **έχει δημιουργηθεί** ως **οντότητα**, αλλά, ωστόσο, **δεν έχει ακόμη τοποθετηθεί** στην **λίστα των διεργασιών προς εκτέλεση**. Υπάρχουν **πολλοί λόγοι** που οδηγούν στην δημιουργία μιας **νέας διεργασίας**, όπως είναι

- η **αρχικοποίηση** του συστήματος
- η **δημιουργία** μιας **θυγατρικής** διεργασίας από μία **υφιστάμενη** διεργασία που εκτελείται
- η **εξυπηρέτηση του αιτήματος κάποιου χρήστη**, ο οποίος έχει ζητήσει την δημιουργία μιας νέας διεργασίας

Ας σημειωθεί, πως **δεν είναι βέβαιο** πως το **αίτημα δημιουργίας** μιας **νέας** διεργασίας θα γίνει **αποδεκτό** από το σύστημα

Όταν μία **διεργασία** βρίσκεται στην κατάσταση **Νέας Διεργασίας**, το λειτουργικό σύστημα έχει κάνει **αποδεκτό** το **αίτημα δημιουργίας** της και έχει προβεί σε όλες τις **αναγκαίες διαδικασίες αρχικοποίησης**, όπως είναι η εκχώρηση στην διεργασία ενός **μοναδικού PID** και η δέσμευση των **κατάλληλων δομών δεδομένων** που περιέχουν πληροφορίες σχετικές με την νέα διεργασία.

Ωστόσο, η διεργασία **δεν βρίσκεται ακόμη στην κεντρική μνήμη**, ενώ επιπλέον, **δεν έχει δεσμευτεί ο παραμικρός χώρος** για τα δεδομένα που θα χρησιμοποιηθούν από αυτή.

Σε αυτό το στάδιο, αυτές οι δομές, διατηρούνται σε **δευτερεύοντες αποθηκευτικούς χώρους**, όπως είναι οι **σκληροί δίσκοι**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Το **πλήθος των διεργασιών** που σε κάθε χρονική στιγμή μπορεί να **εξυπηρετηθεί** από το λειτουργικό σύστημα, είναι πεπερασμένο, διότι υπόκειται σε περιορισμούς όπως είναι, για παράδειγμα, το μέγεθος της εικονικής μνήμης.

Όταν όλες οι **διαδικασίες αρχικοποίησης** έχουν **ολοκληρωθεί** η διεργασία μεταπίπτει στην **κατάσταση ετοιμότητας** και προστίθεται στην **ουρά των έτοιμων διεργασιών**.

Η επιλογή της **επόμενης διεργασίας** που θα αποσταλεί στον **επεξεργαστή** για **εκτέλεση**, καθώς και ο ορισμός της **χρονικής διάρκειας χρήσης** του επεξεργαστή για την κάθε διεργασία, αποτελούν το αντικείμενο της **χρονοδρομολόγησης διεργασιών**. Υπάρχουν πολλοί **αλγόριθμοι χρονοδρομολόγησης**, όπως είναι οι

1. FCFS (First Come, First Served)
2. SJF (Shortest Job First)
3. SRTF (Shortest Remaining Time First)
4. RR (Round Robin)

Μιλώντας γενικά, η **χρονοδρομολόγηση διεργασιών** θα πρέπει να λάβει τις **βέλτιστες αποφάσεις** στον **μέγιστο δυνατό βαθμό**, έτσι ώστε να **μεγιστοποιηθεί** η **απόδοση** του συστήματος και να **ικανοποιηθεί** το **αίσθημα της δικαιοσύνης**, όσον αφορά στην χρήση του επεξεργαστή από τις **διεργασίες** που εκτελούνται σε αυτόν.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Η **μετάβαση** μιας διεργασίας από την **κατάσταση ετοιμότητας** στην **κατάσταση εκτέλεσης**, πραγματοποιείται όταν η διεργασία **απομακρύνεται** από την ουρά των **έτοιμων διεργασιών** και διαβιβάζεται στον **επεξεργαστή** για να **εκτελεστεί**.

Σε κάθε χρονική στιγμή, **μόνο μία και μοναδική διεργασία** μπορεί να βρίσκεται σε **κατάσταση εκτέλεσης**, τουλάχιστον για τα **απλά** συστήματα που χρησιμοποιούν **έναν απλό** επεξεργαστή.

Μετά την **παρέλευση** του **κβάντου χρόνου** που έχει καθορισθεί για την **χρήση** του επεξεργαστή από την **κάθε διεργασία**, η διεργασία **απομακρύνεται** από τον επεξεργαστή και **επανατοποθετείται** στην ουρά των **έτοιμων διεργασιών**.

Άλλος παράγοντας που **λαμβάνεται υπόψη** όσον αφορά στον **προσδιορισμό** της **χρονικής στιγμής απομάκρυνσης** μιας διεργασίας από τον επεξεργαστή, είναι οι **τιμές προτεραιότητας** που έχουν εκχωρηθεί στις διεργασίες του συστήματος.

Η **ολοκλήρωση** μιας διεργασίας απαιτεί την **επαναληπτική μετάβαση** της από την **κατάσταση ετοιμότητας** στην **κατάσταση εκτέλεσης** και στην συνέχεια, **πίσω στην κατάσταση ετοιμότητας**, μέχρι τελικά να **ολοκληρωθεί** η εκτέλεσή της και η διεργασία να **αποχωρήσει** από το σύστημα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Εάν μία διεργασία ζητήσει την **πραγματοποίηση συμβάντος** η εξυπηρέτηση του οποίου απαιτεί **μεγάλο** χρονικό διάστημα, ο επεξεργαστής **δεν** παραμένει αδρανής, αναμένοντας **την ολοκλήρωσή του**, αφού μία τέτοια αναμονή θα οδηγούσε σε **πολύ μεγάλες χρονικές καθυστερήσεις** και σε **μείωση** της απόδοσης του συστήματος.

Εάν μία διεργασία **ζητήσει** μία τέτοια εξυπηρέτηση, το **λειτουργικό σύστημα** την **απομακρύνει** από την ΚΜΕ και την μεταφέρει στην **μπλοκαρισμένη κατάσταση**, όπου και παραμένει μέχρι να **ολοκληρωθεί** το αίτημα εξυπηρέτησης που ζήτησε.

Μετά την **ολοκλήρωση** του αιτήματος, η διεργασία τοποθετείται στην ουρά των **έτοιμων διεργασιών** έτσι ώστε να **επανακτήσει** τον επεξεργαστή, όταν έρθει και πάλι η σειρά της, έτσι όπως αυτό υπαγορεύεται από τον **αλγόριθμο χρονοδρομολόγησης**.

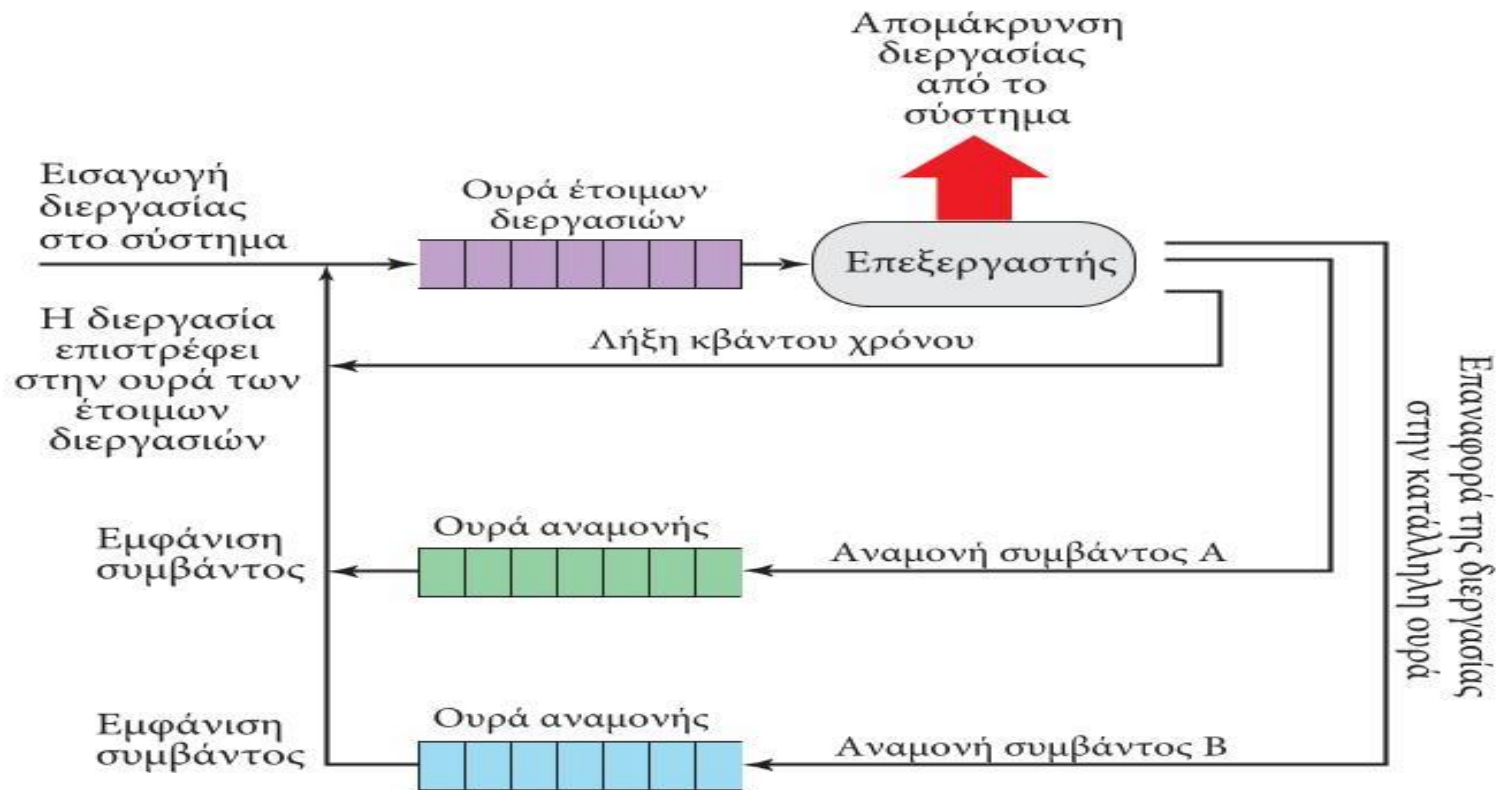
Το σύστημα διατηρεί μία **ουρά μπλοκαρισμένων διεργασιών** και μάλιστα επειδή υπάρχουν **πολλά διαφορετικά αιτήματα** που **ενδέχεται** να οδηγήσουν την διεργασία στην **μπλοκαρισμένη κατάσταση**, μπορούν να υπάρχουν **περισσότερες από μία** ουρές αναμονής **μπλοκαρισμένων** διεργασιών, **μία** ουρά αναμονής **για κάθε διαφορετικό τύπο** αιτήματος εξυπηρέτησης

Μετά την **ολοκλήρωση** του αιτήματός εξυπηρέτησης, η διεργασία τελικά επιστρέφει στην **ουρά των έτοιμων διεργασιών** και το σύστημα λειτουργεί με τον τρόπο που περιγράψαμε προηγουμένως.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας



Σε ένα λειτουργικό σύστημα ενδέχεται να υπάρχουν **πολλές ουρές αναμονής μπλοκαρισμένων διεργασιών**, **μία** ουρά αναμονής **για κάθε διαφορετικό τύπο συμβάντος** που προκαλεί την διακοπή της λειτουργίας μίας διεργασίας.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας

Η ολοκλήρωση μιας διεργασίας, πραγματοποιείται σε δύο στάδια.

Στο πρώτο στάδιο, η διεργασία μεταβαίνει στην κατάσταση εξόδου και δεν είναι πλέον διαθέσιμη προς εκτέλεση, αν και οι σχετιζόμενες δομές δεδομένων διατηρούνται στη μνήμη, για λόγους καταγραφής στατιστικών πληροφοριών.

Στο δεύτερο στάδιο, τόσο η διεργασία όσο και οι εμπλεκόμενες δομές δεδομένων, διαγράφονται οριστικά από το σύστημα.

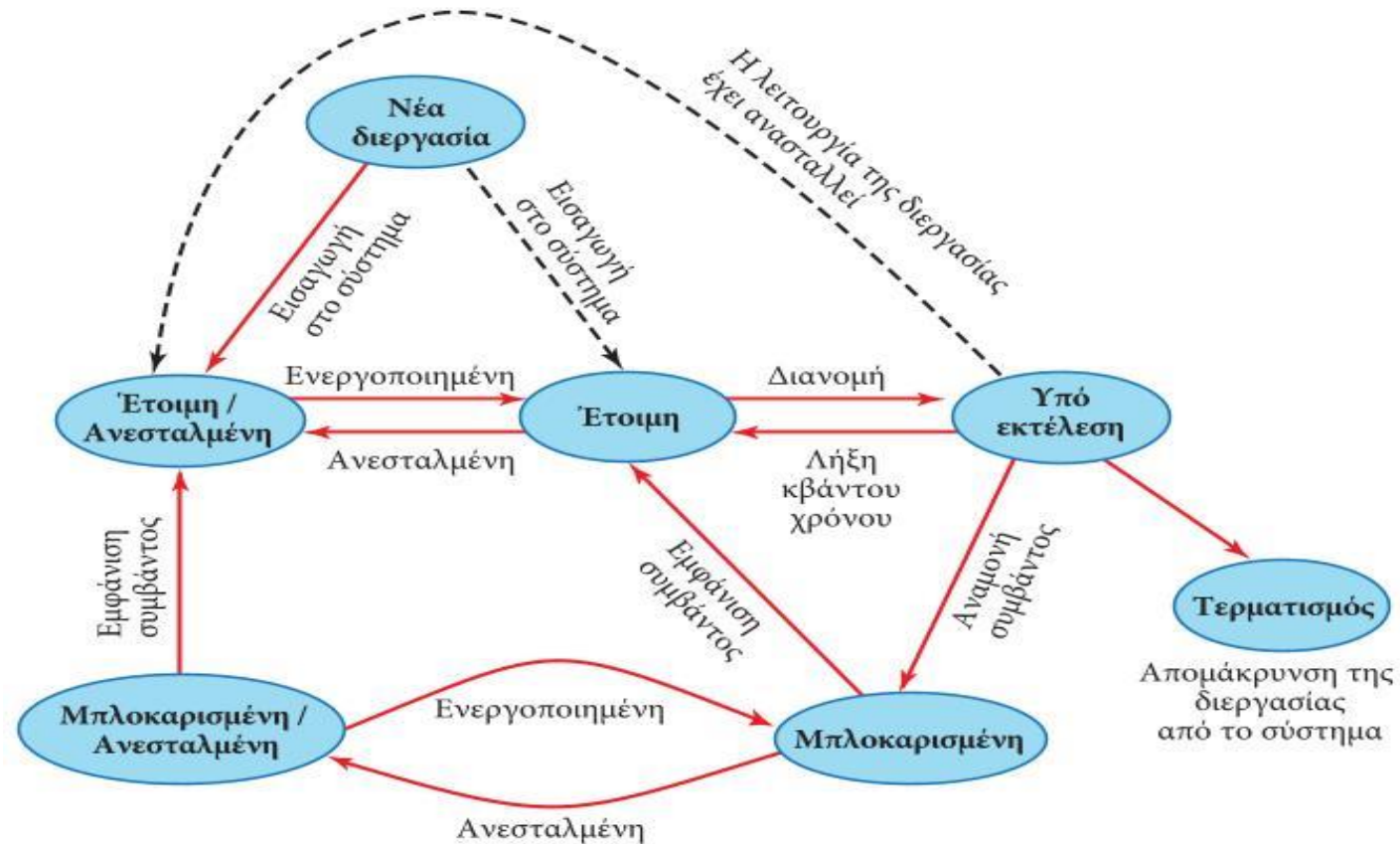
Μία διεργασία ολοκληρώνεται είτε φυσιολογικά είτε απότομα λόγω της εμφάνισης κάποιου σφάλματος όπως είναι

1. η υπέρβαση του χρονικού ορίου χρήσης του επεξεργαστή που της έχει εκχωρηθεί
2. η προσπέλαση περιοχών μνήμης στις οποίες δεν διαθέτει δικαίωμα πρόσβασης
3. η αδυναμία του λειτουργικού συστήματος να καλύψει τις απαιτήσεις της σε υπολογιστικούς πόρους
4. η απόπειρα χρήσης προνομίων τα οποία δεν της έχουν εκχωρηθεί
5. πάσης φύσεως αριθμητικά σφάλματα
6. η υπέρβαση του μέγιστου χρονικού διαστήματος αναμονής της εκδήλωσης ενός συμβάντος
7. πάσης φύσεως σφάλματα εισόδου / εξόδου
8. πάσης φύσεως σφάλματα προστασίας
9. πάσης φύσεως σφάλματα που σχετίζονται με την γονική διεργασία που την δημιούργησε



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο κύκλος ζωής και οι καταστάσεις μιας διεργασίας



Επαυξημένο διάγραμμα **επτά καταστάσεων** που περιγράφει τον **κύκλο ζωής μιας διεργασίας** και τις **μεταβάσεις** που αυτή **υφίσταται** ανάμεσα στις διάφορες καταστάσεις.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι βασικές λειτουργίες διαχείρισης διεργασιών

- 1. Δημιουργία διεργασίας.** Περιλαμβάνει στάδια όπως είναι η **εκχώρηση PID**, η **ανάθεση μνήμης** στην **εικόνα** της διεργασίας, η **αρχικοποίηση** της δομής ελέγχου, η τοποθέτηση της διεργασίας στην κατάλληλη **ουρά αναμονής**, καθώς και η **αρχικοποίηση** των **οποιαδήποτε άλλων δομών δεδομένων** εμπλέκονται στην **διαχείριση** της διεργασίας.
- 2. Τερματισμός διεργασίας.** Μια διεργασία **τερματίζεται**, όταν έχει ολοκληρωθεί ή όταν έχει ανακύψει κάποιο σφάλμα.
- 3. Αναστολή διεργασίας.** Μία διεργασία μπορεί να **ανασταλεί** εάν το σύστημα αποφασίσει πως αυτό είναι κάτι που **θα λειτουργήσει προς όφελος** των άλλων διεργασιών και της συνολικής του **απόδοσης**.
- 4. Επαναφορά διεργασίας.** Ορίζεται ως η **διαδικασία**, κατά την οποία κάποια διεργασία της οποίας η λειτουργία **είχε προσωρινά ανασταλεί** για κάποιο λόγο, **επανέρχεται στο προσκήνιο** και **συνεχίζει** την εκτέλεσή της.
- 5. Αλλαγή προτεραιότητας διεργασίας.** Οι διεργασίες χαρακτηρίζονται συνήθως από κάποια **τιμή προτεραιότητας**, η οποία καθορίζει τον **χρόνο** και την **συχνότητα χρήσης** του επεξεργαστή και μπορεί να **μεταβληθεί** κατά βούληση.
- 6. Μπλοκάρισμα διεργασίας.** Μια διεργασία μπορεί να **μπλοκαριστεί**, όταν περιμένει να πραγματοποιηθεί κάποιο συμβάν.
- 7. Ενεργοποίηση διεργασίας.** Ορίζεται ως η **μετάβαση** μιας **μπλοκαρισμένης διεργασίας**, στην κατάσταση **ετοιμότητας**.
- 8. Διανομή διεργασίας.** Ορίζεται ως η **μετάβαση** μιας διεργασίας από την κατάσταση **ετοιμότητας** στην κατάσταση **εκτέλεσης**.
- 9. Επικοινωνία με άλλες διεργασίες.** Κάθε διεργασία μπορεί να **επικοινωνήσει** με **άλλες** διεργασίες του συστήματος.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η δομή ελέγχου διεργασίας

Δημιουργείται από το **λειτουργικό σύστημα** για **κάθε νέα** διεργασία που δημιουργείται στο σύστημα.

Το **σύνολο** αυτών των δομών για **όλες** τις διεργασίες του συστήματος βρίσκεται αποθηκευμένο στον **πίνακα διεργασιών**.

Ο βασικός της ρόλος είναι η **υποστήριξη** του χαρακτηριστικού του **πολυπρογραμματισμού** που ορίζεται ως η **εκτέλεση πολλών διεργασιών** από τον **ίδιο** απλό επεξεργαστή.

Όταν μία διεργασία **διακόπτεται**, οι τιμές των **χαρακτηριστικών** της την στιγμή της **διακοπής** αποθηκεύονται σε αυτή τη **δομή**.

Όταν η διεργασία **επανέλθει για εκτέλεση**, το λειτουργικό σύστημα **επαναφέρει** αυτές τις τιμές, έτσι ώστε η **εκτέλεση** της διεργασίας να **συνεχίσει** από το σημείο που είχε **σταματήσει**.

Αυτή η **εναλλαγή διεργασιών** στην **ΚΜΕ**, ονομάζεται **μεταγωγή περιβάλλοντος** (context switching).

Δείκτης στη γονική διεργασία	Κατάσταση διεργασίας
Δείκτης στις θυγατρικές διεργασίες	
Αναγνωριστικό διεργασίας	
Προτεραιότητα διεργασίας	
Μετρητής προγράμματος	
Καταχωρητές	
Δείκτες στη μνήμη της διεργασίας	
Όρια χρήσης της μνήμης του συστήματος	
Λίστα ανοικτών αρχείων	
...	

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα πεδία της δομής ελέγχου διεργασίας

1. Η τιμή του **αναγνωριστικού** της διεργασίας (**PID**)
2. Η **τρέχουσα κατάσταση** της διεργασίας
3. Ένας δείκτης προς την **γονική** διεργασία
4. Ένας δείκτης προς τις **θυγατρικές** διεργασίες
5. Πληροφορίες για **σχετιζόμενες** διεργασίες
6. Η **προτεραιότητα** της διεργασίας
7. Πληροφορίες **χρονοδρομολόγησης**
8. Ένας **δείκτης** προς την **περιοχή μνήμης** που περιέχει τα **δεδομένα** της διεργασίας.
9. Ένας **δείκτης** προς την **περιοχή μνήμης** που περιέχει τον **κώδικα** του προγράμματος
10. Ένας **δείκτης** προς την **περιοχή μνήμης** περιέχει **πληροφορίες** του τμήματος της στοίβας



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα πεδία της δομής ελέγχου διεργασίας

11. Ένας δείκτης προς την περιοχή της κοινόχρηστης μνήμης
12. Ένας δείκτης προς την περιοχή μνήμης στην οποία έχουν δηλωθεί οι πόροι του συστήματος
13. Ένας δείκτης προς την περιοχή μνήμης στην οποία αποθηκεύονται πληροφορίες εισόδου / εξόδου
14. Ένας δείκτης προς την περιοχή μνήμης στην οποία ο επεξεργαστής αποθηκεύει τα περιεχόμενα των καταχωρητών
15. Κωδικοί συνθήκης, που περιέχουν τα αποτελέσματα των πιο πρόσφατων αριθμητικών και λογικών πράξεων
16. Πληροφορίες κατάστασης, που σχετίζονται με τις τιμές των σημαιών ενεργοποίησης και απενεργοποίησης διακοπών.
17. Πληροφορίες σχετικά με τα εμπλεκόμενα σήματα και μηνύματα
18. Πληροφορίες σχετικά με τα προνόμια και τα πάσης φύσεως δικαιώματα πρόσβασης
19. Χρονικές πληροφορίες, όπως είναι ο χρόνος εκκίνησης της διεργασίας

Στο λειτουργικό σύστημα Linux, η δομή ελέγχου διεργασίας υλοποιείται από την δομή `task_struct`, η οποία αποτελεί μία από τις μεγαλύτερες, αν όχι τη μεγαλύτερη δομή του λειτουργικού συστήματος και ορίζεται στο αρχείο επικεφαλίδας `sched.h`



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα βασικά πεδία της δομής task_struct (1)

struct task_struct {		
volatile long state;	/* The state of the process	*/
void *stack;	/* A pointer to the thread_info structure	*/
refcount_t usage;	/* The usage count of the process descriptor	*/
unsigned int flags;	/* The detailed state of the process in the kernel	*/
unsigned int ptrace;	/* Used by ptrace system call to observe and control process execution	*/
int on_rq;	/* True if the process is on a runqueue and false otherwise	*/
int prio;	/* The dynamic priority of the process	*/
int static_prio;	/* The static or nominal priority of the process	*/
int normal_prio;	/* The normalized priority of the process	*/
unsigned int rt_priority;	/* The real time priority of the process, if it has one	*/
const struct sched_class *sched_class;	/* The scheduling class to which the process belongs	*/
struct sched_entity se;	/* The scheduling entity to which a normal process belongs	*/
struct sched_rt_entity rt;	/* The scheduling entity to which a real time process belongs	*/
struct sched_dl_entity dl;	/* The associated sched_entity for the dl_sched_class scheduling class	*/
unsigned int policy;	/* The scheduling policy applied to the process	*/
int nr_cpus_allowed;	/* The number of CPUs to which the process is allowed to run	*/
const cpumask_t *cpus_ptr;	/* A pointer to the CPU mask of the process	*/
cpumask_t cpus_mask;	/* The set of CPUs to which the process is allowed to run	*/
struct sched_info sched_info;	/* Runtime information for the scheduler statistics process	*/
struct list_head tasks;	/* Double linked list of tasks in the system	*/
struct mm_struct *mm;	/* User space memory descriptor owned by the process	*/
struct mm_struct *active_mm;	/* Points to the memory descriptor used by the process runtime	*/



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα βασικά πεδία της δομής task_struct (2)

struct vmacache vmacache;	/* Per-thread vma caching	*/
int exit_state;	/* The exit state for process	*/
int exit_code;	/* The termination code of the process	*/
int exit_signal;	/* A signal used in thread groups to notify for process termination	*/
int pdeath_signal;	/* The signal sent when the parent dies	*/
unsigned long jobctl;	/* It is used to control the process behavior	*/
unsigned int personality;	/* Used for emulating ABI behavior of previous Linux versions	*/
unsigned sched_reset_on_fork:1;	/* Revert to default priority / policy when forking	*/
unsigned sched_contributes_to_load:1;	/* True if the process contributes to CPU load	*/
unsigned sched_migrated:1;	/* Associated with the perf_event perf_events observability tool	*/
unsigned :0;	/* Force alignment to the next boundary	*/
unsigned sched_remote_wakeup:1;	/* Associated with task remote wakeup	*/
unsigned in_execve:1;	/* A bit that informs LSMs that the call execve() is executed	*/
unsigned in_iowait:1;	/* Informs the process accounting that the task is in IO wait state	*/
unsigned long atomic_flags;	/* Flags requiring atomic access	*/
struct restart_block restart_block;	/* The restart block of the process	*/
pid_t pid;	/* The process identifier (PID)	*/
pid_t tgid;	/* The PID of the lead process of the group in which it is located	*/
struct task_struct __rcu *real_parent;	/* Points to the descriptor of the parent of the current process	*/
struct task_struct __rcu *parent;	/* Identifies the recipient of SIGCHLD sent by wait4 system call	*/
struct list_head children;	/* The linked list of children of the current process (list head)	*/
struct list_head sibling;	/* The linked list of children of the current process (list entries)	*/



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα βασικά πεδία της δομής task_struct (3)

struct task_struct *group_leader;	/* The descriptor of the lead process of the group the process belongs */
struct list_head ptraced;	/* Identifies the linked list of tasks traced by the current process */
struct list_head ptrace_entry;	/* Identifies the linked list of tasks traced by the parent process */
struct pid *thread_pid;	/* A pointer to the pid struct associated with the current process */
struct hlist_node pid_links [PIDTYPE_MAX];	/* The list of task structures that share an identifier */
struct list_head thread_group;	/* The list of all processes in the thread group */
struct list_head thread_node;	/* The list of all processes in the thread group */
struct completion *vfork_done;	/* Associated with the execution of the do_fork() system call */
int __user *set_child_tid;	/* Indicates that the child process user address space must be changed */
int __user *clear_child_tid;	/* Indicates that the child process user address space must be changed */
u64 utime;	/* The running time of the process in the user mode */
u64 stime;	/* The running time of the process in the kernel mode */
u64 gtime;	/* The guest time of the process */
struct prev_cputime prev_cputime;	/* Stored previous user / system time to guarantee monotonicity */
unsigned long nvcs;	/* The number of voluntary context switches */
unsigned long nivcs;	/* The number of involuntary context switches */
u64 start_time;	/* The monotonic process creation time in nanoseconds */
u64 start_boottime;	/* The monotonic process creation time, also contains sleep time */
unsigned long min_flt;	/* The number of minor page faults caused by the process */
unsigned long maj_flt;	/* The number of major page faults caused by the process */
struct posix_cputimers posix_cputimers;	/* A container for posix CPU timer related data */



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα βασικά πεδία της δομής task_struct (4)

const struct cred __rcu *ptracer_cred;	/* The credentials of the tracer of the current process	*/
const struct cred __rcu *real_cred;	/* The objective and real subjective task credentials	*/
const struct cred __rcu *cred;	/* The effective (overridable) subjective task credentials	*/
char comm [TASK_COMM_LEN];	/* The name of the process executable, excluding path	*/
struct nameidata *nameidata;	/* The per-process current filename lookup	*/
struct fs_struct *fs;	/* The file system information structure for the current task	*/
struct files_struct *files;	/* A structure that keeps information for the task's open files	*/
struct nsproxy *nsproxy;	/* The namespace information of the current task	*/
struct signal_struct *signal;	/* A pointer to the signal descriptor of the process	*/
struct sighand_struct __rcu *sighand;	/* A pointer to the signal handler descriptor of the process	*/
sigset_t blocked;	/* The mask of the blocked signals of the process	*/
sigset_t real_blocked;	/* The temporary mask of the blocked signals	*/
sigset_t saved_sigmask;	/* This mask is restored if set_restore_sigmask () was used	*/
struct sigpending pending;	/* A data structure for storing suspended signals	*/
unsigned long sas_ss_sp;	/* The address of the signal handler alternate stack	*/
size_t sas_ss_size;	/* The size of the signal handler alternate stack	*/
unsigned int sas_ss_flags;	/* A set of flags for the signal handler alternate stack	*/
struct callback_head *task_works;	/* A field associated with the task_work mechanism	*/
struct seccomp seccomp;	/* A flag associated with seccomp (secure computing) filtering	*/
u64 parent_exec_id;	/* Used in thread group tracking together with self_exec_id	*/
u64 self_exec_id;	/* Used in thread group tracking together with parent_exec_id	*/
spinlock_t alloc_lock;	/* A spinlock that offers protection to a set of task_struct fields	*/



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Τα βασικά πεδία της δομής task_struct (5)

<code>raw_spinlock_t pi_lock;</code>	<code>/* The lock used for protection of the PI data structures</code>	<code>*/</code>
<code>struct wake_q_node wake_q;</code>	<code>/* A field associated with the wake_q mechanism</code>	<code>*/</code>
<code>void *journal_info;</code>	<code>/* ISF file system Journalling filesystem info</code>	<code>*/</code>
<code>struct bio_list *bio_list;</code>	<code>/* Stacked block device info</code>	<code>*/</code>
<code>struct reclaim_state *reclaim_state;</code>	<code>/* Memory recycling VM state</code>	<code>*/</code>
<code>struct backing_dev_info *backing_dev_info;</code>	<code>/* Store block device IO data flow information</code>	<code>*/</code>
<code>struct io_context *io_context;</code>	<code>/* Information used by the IO scheduler</code>	<code>*/</code>
<code>unsigned long ptrace_message;</code>	<code>/* Ptrace state associated with ptrace mechanism</code>	<code>*/</code>
<code>kernel_siginfo_t *last_siginfo;</code>	<code>/* Used in the ptrace mechanism to test a stopped process</code>	<code>*/</code>
<code>struct task_io_accounting ioac;</code>	<code>/* Used for recording a single task's IO statistics</code>	<code>*/</code>
<code>struct tlbflush_unmap_batch tlb_ubic;</code>	<code>/* Used to track memory pages that require TLB flushes</code>	<code>*/</code>
<code>union { refcount_t rcu_users;</code>	<code>/* A count of task RCU users</code>	<code>*/</code>
<code> struct rcu_head rcu; };</code>	<code>/* An RCU related field</code>	<code>*/</code>
<code>struct pipe_inode_info *splice_pipe;</code>	<code>/* Cache last used pipe for splice()</code>	<code>*/</code>
<code>struct page_frag task_frag;</code>	<code>/* Described a page fragment used in network socket managements</code>	<code>*/</code>
<code>int nr_dirtied;</code>	<code>/* The number of dirty pages of the current process</code>	<code>*/</code>
<code>int nr_dirtied_pause;</code>	<code>/* A limiting value for the number of dirty pages</code>	<code>*/</code>
<code>unsigned long dirty_paused_when;</code>	<code>/* Start of a write-and-pause period</code>	<code>*/</code>
<code>u64 timer_slack_ns;</code>	<code>/* Used to round up poll() and select() timeout values (nsecs)</code>	<code>*/</code>
<code>u64 default_timer_slack_ns;</code>	<code>/* The default timer_slack_ns value</code>	<code>*/</code>
<code>int pagefault_disabled;</code>	<code>/* Decouples the pagefault - disabled logic from the preempt count</code>	<code>*/</code>
<code>struct thread_struct thread; };</code>	<code>/* CPU specific state of this task</code>	<code>*/</code>



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Δημιουργία και αναπαραγωγή διεργασιών

Ο **τρόπος δημιουργίας διεργασίας**, εξαρτάται από το **λειτουργικό**, με αυτή την διαδικασία στο **Linux** να στηρίζεται στην χρήση της **κλήσεως συστήματος fork**, η οποία δημιουργεί ένα **πλήρες αντίγραφο** της διεργασίας που την καλεί.

Μετά την **δημιουργία** της **νέας** διεργασίας, οι δύο διεργασίες εκτελούνται **παράλληλα** και **ανεξάρτητα** η μία από την άλλη, ενώ σχετίζονται μεταξύ τους μέσω **σχέσης γονέα - παιδιού**, υπό την έννοια πως η διεργασία που **δημιουργήθηκε** από την κλήση της **fork** νοείται ως **θυγατρική** διεργασία, ενώ η διεργασία που κάλεσε την **fork** νοείται ως **γονική** διεργασία.

Η **διάκριση** ανάμεσα σε αυτές τις **δύο διεργασίες** στηρίζεται στην χρήση της **επιστρεφόμενης τιμής** της **fork**. **Ειδικότερα:**

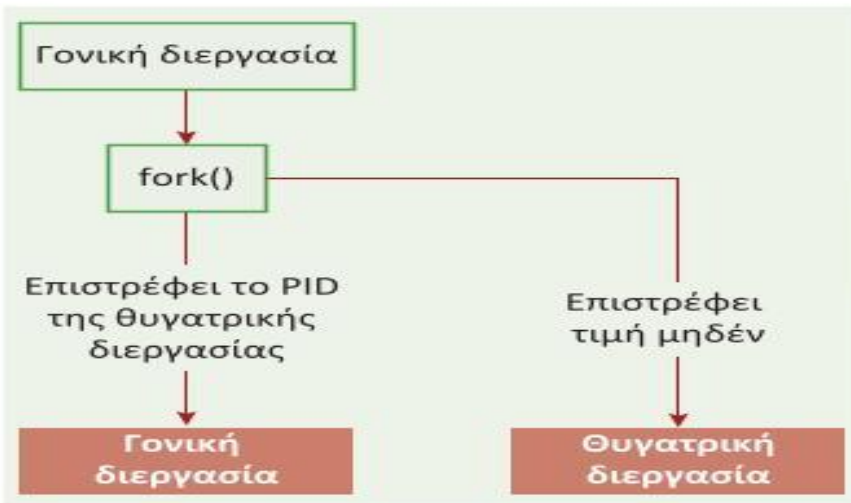
- η τιμή που επιστρέφεται στην **γονική διεργασία** είναι **μη μηδενική** και προσδιορίζει το **PID** της **θυγατρικής** διεργασίας
- η τιμή που επιστρέφεται στην **θυγατρική διεργασία**, είναι **ίση με το μηδέν**.

Το **βασικό χαρακτηριστικό** της **δημιουργίας διεργασιών** δια της χρήσεως της **κλήσεως συστήματος fork**, είναι πως χαρακτηρίζεται από **πολύ μεγάλο υπολογιστικό κόστος**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Δημιουργία και αναπαραγωγή διεργασιών



```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid, pid1;
```

A Κώδικας πριν την κλήση της fork.
Υπάρχει μία και μοναδική διεργασία
η οποία είναι η γονική διεργασία

```
/* fork a child process */
pid = fork();

if (pid == 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
    return 1;}
```

```
else if (pid == 0) { /* child process */
```

Γ Κώδικας που εκτελείται από
τη θυγατρική διεργασία

```
}
else if { /* parent process */
```

B Κώδικας που εκτελείται από
τη γονική διεργασία

```
}
return 0; }
```

Εάν λοιπόν θέλουμε να
γράψουμε κώδικα που
να εκτελείται από τη
γονική διεργασία,
αυτός μπορεί
να γραφεί είτε στην
Ενότητα A είτε στην
Ενότητα B, οι οποίες
είναι ισοδύναμες.

Αντίθετα ο κώδικας για
τη θυγατρική διεργασία
γράφεται μόνο
στην Ενότητα Γ

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ο πηγαίος κώδικας της κλήσης συστήματος fork

```
SYSCALL_DEFINE0 (fork) --> sys_fork ()
kernel_clone
copy_process
if ((clone_flags & (CLONE_NEWNS|CLONE_FS)) == (CLONE_NEWNS|CLONE_FS)) return ERR_PTR(-EINVAL); /* Check clone flags */
if ((clone_flags & (CLONE_NEWUSER|CLONE_FS)) == (CLONE_NEWUSER|CLONE_FS)) return ERR_PTR(-EINVAL); /* Check clone flags */
if ((clone_flags & CLONE_THREAD) && !(clone_flags & CLONE_SIGHAND)) return ERR_PTR(-EINVAL); /* Check clone flags */
if ((clone_flags & CLONE_SIGHAND) && !(clone_flags & CLONE_VM)) return ERR_PTR(-EINVAL); /* Check clone flags */

p = dup_task_struct (current, node); /* Create an identical copy of the task structure of the parent process */
alloc_task_struct_node (node) /* Allocates a new task_struct for the child process */
alloc_thread_stack_node (tsk, node) /* Allocates a new stack and assigns it to the new process */
task_stack_vm_area (tsk) /* Allocated the associated virtual memory area */
arch_dup_task_struct (tsk, orig) /* Architecture dependent task_struct duplication */
task_rlimit (p, RLIMIT_NPROC) /* Checks if the maximum number of processes has been exceeded */
copy_creds (p, clone_flags) /* Copies the subjective credentials of the parent process */

INIT_LIST_HEAD (&p->children); /* Initializes the process list of children */
INIT_LIST_HEAD (&p->sibling); /* Initializes the process list of siblings */

p->utime = p->stime = p->gtime = 0; /* Initializes the utime, stime and gtime task_struct members */
#ifdef CONFIG_ARCH_HAS_SCALED_CPUTIME
p->utimescaled = p->stimescaled = 0; /* Initializes the utimescaled and stimescaled task_struct members */
#endif
#ifdef CONFIG_VIRT_CPU_ACCOUNTING_GEN
sched_fork (clone_flags, p); /* Performs the required tasks associated with the process scheduling */
shm_init_task (p); /* Initializes the environment associated with the shared memory */
security_task_alloc(p, clone_flags); /* Initializes the environment associated with the security modules */
copy_semundo (clone_flags, p); /* Copies the System V semaphores of the parent process if COPY_SYSVSEM is set */
copy_files (clone_flags, p); /* Copies the file descriptors of the parent process if CLONE_FILES is set */
copy_fs (clone_flags, p); /* Copies the filesystem context of the parent process if CLONE_FS is set */
copy_sigand (clone_flags, p); /* Uses the signal handlers of the parent if CLONE_SIGHAND or CLONE_THREAD is set */
copy_signal (clone_flags, p); /* Uses the non-handler-specific part of signal handling if CLONE_THREAD is set */
copy_mm (clone_flags, p); /* Forces the parent & child proc to share the address space if COPY_MM is set */
copy_namespaces (clone_flags, p); /* Sets up namespaces for the child process */
copy_io (clone_flags, p); /* Shares io context with parent if CLONE_IO is set */
copy_thread (clone_flags, args->stack, args->stack_size, p, args->tls); /* Copies thread specific data of proc */
alloc_pid (p->nsproxy->pid_ns_for_children, args->set_tid, args->set_tid_size); /* Allocates the pid struct of child */

p->group_leader = current->group_leader; /* Initializes the group leader field of the task_struct of the child */
p->tgid = current->tgid; /* Initializes the tgid field of the task_struct of the child */
p->nr_dirtied = 0; /* Initializes the nr_dirtied field of the task_struct of the child */
p->nr_dirtied_pause = 128 >> (PAGE_SHIFT - 10); /* Initializes the nr_dirtied_pause field of the task_struct of child */
p->dirty_paused_when = 0; /* Initializes the nr_dirtied_paused_when field of the child */
p->pdeath_signal = 0; /* Initializes the pdeath_signal field of the task_struct of the child */
get_task_pid /* Retrieves the PID of the new task */
put_user /* Copies the PID of the new task to the user space */
wake_up_new_task /* Wakes up the new task and puts it to the kernel run queue */
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι σημαίες που υπαγορεύουν τον τρόπο δημιουργίας της θυγατρικής διεργασίας

- CLONE_VM.** Η γονική και η θυγατρική διεργασία εκτελούνται **στον ίδιο χώρο της μνήμης**.
- CLONE_FS.** Η γονική και η θυγατρική διεργασία **διαμοιράζονται τις ίδιες πληροφορίες** όσον αφορά στο **σύστημα αρχείων**.
- CLONE_FILES.** Η γονική και η θυγατρική διεργασία διαμοιράζονται **τον ίδιο πίνακα περιγραφών αρχείων**.
- CLONE_SIGHAND.** Η γονική και η θυγατρική διεργασία διαμοιράζονται **τον ίδιο πίνακα χειριστών σημάτων**.
- CLONE_VFORK.** Η γονική διεργασία **αναστέλλεται** μέχρι την **αποδέσμευση της εικονικής μνήμης** από την θυγατρική.
- CLONE_PARENT.** Ως γονική διεργασία ορίζεται **ο γονέας της διεργασίας που κάλεσε την fork**.
- CLONE_THREAD.** Η θυγατρική διεργασία τοποθετείται **στην ίδια ομάδα νημάτων** με τη γονική διεργασία.
- CLONE_NEWNS.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων MNT**.
- CLONE_SYSVSEM.** Η γονική και η θυγατρική διεργασία μοιράζονται **μία απλή λίστα τιμών σημαφόρων System V**.
- CLONE_CHILD_SETTID.** Το αναγνωριστικού TID της θυγατρικής διεργασίας, **αποθηκεύεται** στη θέση child_tid.
- CLONE_NEWCGROUP.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων CGROUP**.
- CLONE_NEWUTS.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων UTS**.
- CLONE_NEWIPC.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων IPC**.
- CLONE_NEWUSER.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων USER**.
- CLONE_NEWPID.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων PID**.
- CLONE_NEWNET.** Η θυγατρική διεργασία ξεκινά την λειτουργία της σε έναν **νέο χώρο ονομάτων NET**.
- CLONE_IO.** Η θυγατρική διεργασία **μοιράζεται ένα περιβάλλον I/O** με την γονική διεργασία.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

Στις **παραδοσιακές αρχιτεκτονικές απλού επεξεργαστή**, η κάθε **διεργασία** περιγράφεται από

1. ένα **σύνολο υπολογιστικών πόρων** οι οποίοι μεταξύ άλλων περιλαμβάνουν
 - τις τιμές των **αναγκάιων αναγνωριστικών** (PID, PPID, PGID, UID, GID, SID)
 - τις **μεταβλητές περιβάλλοντος** και την τιμή της **umask**
 - τον **τρέχοντα** και τον **ριζικό** κατάλογο εργασίας
 - τους **περιγραφείς αρχείων**
 - τα **σήματα** που επιτρέπουν την υλοποίηση του μηχανισμού της **διαδιεργασιακής επικοινωνίας**
 - τα περιεχόμενα του **σωρού**, της **στοίβας** και γενικότερα του σχετιζόμενου **χώρου διευθύνσεων μνήμης**
 - τις **κοινόχρηστες βιβλιοθήκες** και τις δομές **επικοινωνίας** και **συγχρονισμού** διεργασιών
2. τα **περιεχόμενα των καταχωρητών του επεξεργαστή**, τα οποία συνιστούν το επονομαζόμενο **περιβάλλον εκτέλεσης**.

Ωστόσο, αυτός ο ορισμός **δεν βρίσκει** εφαρμογή στις **σύγχρονες παράλληλες αρχιτεκτονικές**, στις οποίες το σύνολο των καταχωρητών **δεν είναι μοναδικό** (αφού ο κάθε πυρήνας διαθέτει **τους δικούς του** καταχωρητές).

Για τον λόγο αυτό, η **διεργασία** ορίστηκε έτσι ώστε να σχετίζεται **μόνο με τις παραμέτρους** που απαιτούνται για την διαχείρισή της, ενώ **αποσυνδέθηκε εντελώς** από το περιβάλλον της εκτέλεσής της, αφού αυτό πλέον **δεν ορίζονται** με μοναδικό τρόπο.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

Στον **νέο ορισμό** της διεργασίας εμφανίζεται πλέον **μόνο η λίστα των υπολογιστικών πόρων** που αναφέραμε προηγουμένως, ενώ το **περιβάλλον εκτέλεσης**, που **περιλαμβάνει** τους καταχωρητές και γενικότερα, **οτιδήποτε απαιτείται** για την **εκτέλεση** του **κώδικα** της διεργασίας, αποτέλεσε την βάση για τον ορισμό μιας **δεύτερης οντότητας εκτέλεσης**, η οποία είναι γνωστή ως **νήμα**.

Ειδικότερα το νήμα, ως **οντότητα εκτέλεσης**, περιλαμβάνει **μόνο τις τελείως αναγκαίες πληροφορίες** που απαιτούνται για την ανεξάρτητη **χρονοδρομολόγηση** και **εκτέλεσή** του και πιο συγκεκριμένα:

1. τον **δείκτη της στοίβας** που περιέχει τις **τοπικές μεταβλητές** του νήματος
2. τα περιεχόμενα των **καταχωρητών**
3. τις ιδιότητες που σχετίζονται με την **χρονοδρομολόγηση**
4. το σύνολο των **εκκρεμών** και **μπλοκαρισμένων** σημάτων
5. την **καθολική μεταβλητή σφάλματος** errno
6. τα πάσης φύσεως **τοπικά δεδομένα** του νήματος.

Το κάθε **νήμα** περιγράφεται από το δικό του **αναγνωριστικό TID (Thread ID)**, το οποίο ταυτοποιεί το νήμα με **μοναδικό** τρόπο, ανάμεσα στο **σύνολο των νημάτων** που ανήκουν στην **ίδια** διεργασία.

Αυτές οι πληροφορίες **αποθηκεύονται χωριστά** για το κάθε νήμα ενώ οι **υπόλοιπες ιδιότητες** της διεργασίας που αναφέραμε προηγουμένως, είναι πλέον **κοινόχρηστες** για όλα τα νήματα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

Τι είναι λοιπόν ένα νήμα;

- Από την **οπτική γωνία του λειτουργικού συστήματος**, αποτελεί **ανεξάρτητο σύνολο εντολών**, οι οποίες μπορούν να **δρομολογηθούν** προκειμένου να εκτελεστούν από έναν πυρήνα
- Από την **οπτική γωνία του προγραμματιστή** ορίζεται ως μια **διαδικασία** η οποία δημιουργείται από το **κυρίως** πρόγραμμα, **συνυπάρχει** με αυτό και εκτελείται **ταυτόχρονα** με αυτό, αλλά ωστόσο, **ανεξάρτητα** από αυτό.

Μία διεργασία είναι αδύνατο να υπάρξει **αυτόνομα**, δηλαδή **χωρίς να συσχετιστεί** με κάποιο **νήμα** αλλά όταν το λειτουργικό σύστημα **εκτελεί** μία **διεργασία**, δημιουργεί για αυτή ένα νήμα, το οποίο νοείται ως το **κύριο** ή το **αρχικό** ή **ηγετικό** νήμα.

Από την στιγμή που ξεκινά η **εκτέλεση** αυτού του **αρχικού νήματος**, αυτό μπορεί, να δημιουργήσει **πρόσθετα νήματα**, στα οποία **διαμοιράζει** τον υπολογιστικό φόρτο της εφαρμογής, με όλα αυτά τα νήματα να εκτελούνται **παράλληλα**.

Επιπλέον, από την στιγμή που τα νήματα **διαμοιράζονται** τους **υπολογιστικούς πόρους** της διεργασίας στην οποία ανήκουν, **ούτε αυτά είναι δυνατόν να υπάρξουν αυτόνομα**, δηλαδή **έξω** από την διεργασία που τα δημιούργησε, αλλά αντίθετα, αποτελούν **αναπόσπαστο** τμήμα της.

Ο **τερματισμός** της λειτουργίας μιας διεργασίας, **ανεξάρτητα** από τον λόγο για τον οποίο αυτός πραγματοποιείται, έχει ως αποτέλεσμα και τον **τερματισμό** της λειτουργίας **όλων των νημάτων** που σχετίζονται με αυτή την διεργασία.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

Εάν η λειτουργία κάποιου από τα **νήματα μιας διεργασίας** οδηγήσει στην εμφάνιση κάποιου **σφάλματος** και οδηγηθεί σε **απότομο τερματισμό**, αυτή η **κατάρρευση** θα προκαλέσει κατά πάσα πιθανότητα την **κατάρρευση** της εφαρμογής, αφού τα νήματα **διαμοιράζονται** πολλούς **κοινόχρηστους** πόρους.

Ένα ακόμη **μειονέκτημα** της χρήσης των νημάτων, είναι ο **σχετικά περιορισμένος χώρος εικονικών διευθύνσεων** που χρησιμοποιείται από τα αυτά, σε αντίθεση με τις **διεργασίες**, οι οποίες, θεωρητικά μπορούν να διεκδικήσουν **το σύνολο της διαθέσιμης** εικονικής μνήμης.

Η χρήση **σημάτων** στις **πολυνηματικές εφαρμογές**, γενικά **δεν συνίσταται** και θα πρέπει να γίνει με **πολύ μεγάλη** προσοχή.

Βέβαια, από την άλλη πλευρά, η **διαδικασία μεταγωγής περιβάλλοντος** στην περίπτωση των **νημάτων** πραγματοποιείται **πολύ πιο γρήγορα**, αφού το **πλήθος των πληροφοριών** που θα πρέπει να **αποθηκευτούν** και να επανέλθουν αργότερα, είναι **πολύ μικρότερο** από εκείνο που χρησιμοποιείται στην περίπτωση των **διεργασιών**.

Σε αυτόν λοιπόν τον **νέο τρόπο θεώρησης των πραγμάτων**, μία **διεργασία** ενός σύγχρονου λειτουργικού συστήματος, αποτελεί πλέον μία **πολυνηματική οντότητα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

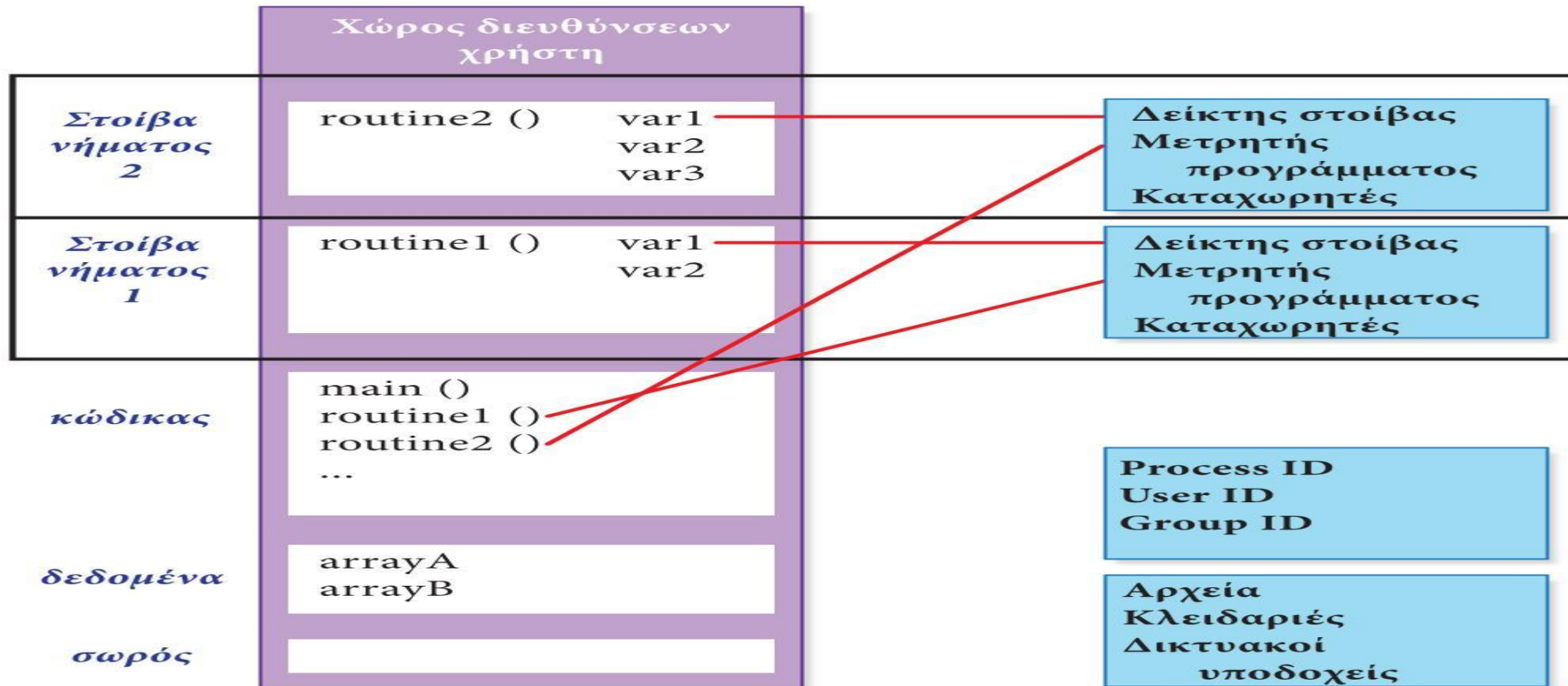
Η διαφορά στη δομή μιας απλής και μιας πολυνηματικής διεργασίας



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Από τις διεργασίες στα νήματα – πολυνηματικός προγραμματισμός

Το **σύνολο των πληροφοριών** που διατηρούνται για **το κάθε νήμα** μιας **πολυνηματικής** διεργασίας.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Ο τρόπος δημιουργίας των νημάτων και το είδος της συσχέτισης ανάμεσα στα νήματα και στις διεργασίες, εξαρτάται από το λειτουργικό, αν και γενικά, όλες οι διαθέσιμες υλοποιήσεις προσφέρουν την ίδια λειτουργικότητα, η οποία με τη σειρά της υπαγορεύεται από το πρότυπο POSIX που διασφαλίζει την φορητότητα των εφαρμογών.

Σύμφωνα με το POSIX, μία βιβλιοθήκη υποστήριξης νημάτων, θα πρέπει να προσφέρει τις επόμενες κατηγορίες συναρτήσεων:

1. συναρτήσεις διαχείρισης νημάτων
2. συναρτήσεις συγχρονισμού
3. συναρτήσεις χρονοδρομολόγησης νημάτων
4. συναρτήσεις χειρισμού σημάτων
5. συναρτήσεις γενικής χρήσεως

Τα POSIX Threads προβλέπουν επίσης, την δυνατότητα συσχέτισης, με το κάθε νήμα, ενός συνόλου ιδιοτήτων, με την μορφή πεδίων / κλειδιών τα οποία λαμβάνουν τις κατάλληλες σε κάθε περίπτωση τιμές.

Αυτές οι ιδιότητες προσδιορίζουν εξειδικευμένα χαρακτηριστικά νημάτων, όπως είναι το μέγεθος της στοίβας, ο τρόπος συμπεριφοράς του νήματος όταν ολοκληρώνεται η διαδικασία που υλοποιείται από αυτό, καθώς και ο τρόπος με τον οποίο συμπεριφέρεται ένα νήμα όταν δοθεί εντολή τερματισμού του από ένα άλλο νήμα



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Η υποστήριξη των νημάτων στα Microsoft Windows Στην περίπτωση κατά την οποία το λειτουργικό σύστημα που χρησιμοποιείται είναι κάποια έκδοση των **Microsoft Windows**, η υποστήριξη της χρήσης νημάτων προσφέρεται **από το ίδιο** το λειτουργικό σύστημα, υπό την έννοια πως αποτελεί **αναπόσπαστο** τμήμα του.

Οι συναρτήσεις που υποστηρίζουν την ανάπτυξη **πολυνηματικών εφαρμογών** ανήκουν σε ένα από **ανώτερα** στρώματα της **πολυεπίπεδης οργάνωσης** που χαρακτηρίζει την δομή των Windows και το οποίο είναι γνωστό ως **CRT (C Runtime Library)**. Για την χρήση αυτών των συναρτήσεων θα πρέπει να **συμπεριλάβουμε** το αρχείο **windows.h** και ίσως και το **process.h**

Το κάθε νήμα προσδιορίζεται από έναν **ειδικό χειριστή** τύπου **HANDLE** (συνώνυμο του τύπου **unsigned long int**) και δημιουργείται από την συνάρτηση **CreateThread** το πρωτότυπο της οποίας έχει τη μορφή

```
HANDLE CreateThread (LPSECURITY_ATTRIBUTES lpThreadAttributes, SIZE_T dwStackSize,  
                    LPTHREAD_START_ROUTINE lpStartAddress,  
                    __drv_aliasesMem LPVOID lpParameter, DWORD dwCreationFlags,  
                    LPDWORD lpThreadId);
```

Αυτή η συνάρτηση καλείται μέσα από τις συναρτήσεις υψηλού επιπέδου **_beginthread** και **_beginthreadex** οι οποίες επιτρέπουν τον καθορισμό **περισσότερων** πληροφοριών. Οι αντίστοιχες **συναρτήσεις τερματισμού** της λειτουργίας ενός νήματος είναι οι συναρτήσεις **_endthread** και **_endthreadex**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Ο **μηχανισμός υποστήριξης νημάτων** στα Microsoft Windows προσφέρει **πολλές οικογένειες συναρτήσεων** που επιτρέπουν την διαχείριση των νημάτων και **όλων των σχετιζόμενων** αντικειμένων. Ας αναφέρουμε ενδεικτικά στο σημείο αυτό τις συναρτήσεις

```
DWORD WaitForSingleObject (HANDLE hHandle, DWORD dwMilliseconds);  
DWORD WaitForMultipleObjects (DWORD nCount, const HANDLE *lpHandles,  
                               BOOL bWaitAll, DWORD dwMilliseconds);
```

οι οποίες υλοποιούν την διαδικασία της **σύνδεσης νημάτων** και δίνουν την δυνατότητα σε ένα νήμα να **αναστείλει** την λειτουργία του αναμένοντας **τον τερματισμό** κάποιου άλλου νήματος

```
#include <windows.h>  
#include <stdio.h>  
  
DWORD WINAPI ThreadFunc (void* data) {  
    printf ("Hello from the new thread\n");  
    return 0; }  
  
int main() {  
    printf ("Hello from the main thread\n");  
    HANDLE thread = CreateThread (NULL, 0, ThreadFunc, NULL, 0, NULL);  
    if (thread)  
        WaitForSingleObject(thread, INFINITE); }
```

```
Hello from the main thread  
Hello from the new thread
```

```
-----  
Process exited after 0.8488 seconds with return value 0  
Press any key to continue . . . |
```

Απλό παράδειγμα **δημιουργίας νήματος** στο περιβάλλον των **Microsoft Windows**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Η υποστήριξη των νημάτων στο λειτουργικό σύστημα Linux Στο Linux το **νήμα** και η **διεργασία** περιγράφονται από την δομή δεδομένων **task_struct** (δείτε τις **Διαφάνειες 16** έως **20**) και ως **έννοιες** είναι τόσο **στενές** και **αλληλένδετες** μεταξύ τους, ώστε πολλές φορές **συγχέονται** και αντιμετωπίζονται **με τον ίδιο ακριβώς** τρόπο.

Ωστόσο, για το λειτουργικό σύστημα, αποτελούν **δύο διαφορετικές οντότητες**, γεγονός που **πιστοποιείται** από την ύπαρξη **δύο μακροεντολών** για την ανάκτηση τους και ειδικότερα της μακροεντολής **do_each_pid_thread** η οποία επιτρέπει την **διάσχιση** της λίστας των **νημάτων** και της μακροεντολής **do_each_pid_task** η οποία επιτρέπει την διάσχιση της **λίστας** των **διεργασιών**.

Αυτή η **στενή συσχέτιση** ανάμεσα στις έννοιες της **διεργασίας** και του **νήματος**, οφείλεται στο γεγονός, πως ο πυρήνας του λειτουργικού υποστηρίζει τις επονομαζόμενες **διεργασίες μικρού βάρους (Lightweight Processes, LWP)** οι οποίες αποτελούν γενίκευση **αμφότερων** των εννοιών και μπορούν να χρησιμοποιηθούν για την υλοποίηση **αμφότερων** αυτών των οντοτήτων.

Αμφότερα οι **διεργασίες** και τα **νήματα** μπορούν να κατασκευαστούν με την **κλήση συστήματος clone**, που χαρακτηρίζεται από **πολύ μικρότερο υπολογιστικό κόστος** σε σχέση με την **fork** (οδηγώντας σε διαδικασίες έως και **10 φορές** πιο γρήγορες) και το τι από τα δύο θα κατασκευαστεί εξαρτάται από το εάν οι **σημαίες** που χρησιμοποιούνται στην συνάρτηση **clone** υπαγορεύουν ή **όχι** τον **διαμοιρασμό των πόρων** του συστήματος

Στην **πρώτη περίπτωση** οι διεργασίες που δημιουργούνται, **διαμοιράζονται** τους **ίδιους πόρους** και χαρακτηρίζονται ως **νήματα**, ενώ στην **δεύτερη περίπτωση** η κάθε **διεργασία** (που περιγράφεται **πλέον** ως τέτοια) διαθέτει **το δικό της** αντίγραφο.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Δημιουργία **νήματος** στο **Linux** δια της χρήσεως της κλήσης συστήματος **clone**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define _SCHED_H 1
#define __USE_GNU 1
#include <bits/sched.h>

#define STACK_SIZE 4096

int func(void *arg) {
    printf("Inside func.\n");
    sleep(1);
    printf("Terminating func...\n");
    return 0; }

int main() {
    printf("This process pid: %u\n", getpid());
    char status_file[] = "/proc/self/status";
    void *child_stack = malloc(STACK_SIZE);
    int thread_pid;
    printf("Creating new thread...\n");
    thread_pid = clone(&func, child_stack+STACK_SIZE,
                      CLONE_SIGHAND|CLONE_FS|CLONE_VM|
                      CLONE_FILES|CLONE_THREAD, NULL);
    printf("Done! Thread pid: %d\n", thread_pid);

    FILE *fp = fopen(status_file, "rb");
    printf("Looking into %s...\n", status_file);
    while(1) {
        char ch = fgetc(fp);
        if(!feof(fp)) break;
        printf("%c", ch); }
    fclose(fp);
    getchar();
    return 0; }
```

```
amarg@amarg-vbox:~$ ./linuxThread
This process pid: 4459
Creating new thread...
Done! Thread pid: 4460
Looking into /proc/self/status...
Name:      linuxThread
Umask:     0002
State:     R (running)
Tgid:      4459
Ngid:       0
Pid:        4459
PPid:       3300
TracerPid:  0
Uid:        1001      1001      1001      1001
Gid:        1001      1001      1001      1001
FDSize:     256
Groups:     4  24  27  30  46  122  134  135  1001
Nstgid:     4459
Nsuid:       4459
NSpid:       4459
NSpgid:      4459
NSSid:       3300
Kthreid:    0
VmPeak:     2776 kB
VmSize:     2776 kB
VmLck:       0 kB
VmPin:       0 kB
```

```
amarg@amarg-vbox:~$ ps -eLF
UID      PID      PPID      LWP      C  NLWP      SZ      RSS  PSR  STIME  TTY
amarg    4459     3300     4459     0    2      694    1408   5  17:46 pts/1
amarg    4459     3300     4460     0    2      694    1408   9  17:46 pts/1
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Οι πολυνηματικές δυνατότητες της γλώσσας C++11 Η γλώσσα C++11 προσφέρει πλήρη υποστήριξη όσον αφορά στην ανάπτυξη πολυνηματικών εφαρμογών, μέσω ενός συνόλου από κλάσεις που σχετίζονται με τον χώρο ονομάτων std και ενός συνόλου υπηρεσιών και δυνατοτήτων που δηλώνονται σε ένα σύνολο αρχείων επικεφαλίδας, όπως είναι τα

1. `<thread>` (διαχείριση νημάτων)
2. `<mutex>` (διαχείριση μεταβλητών αμοιβαίου αποκλεισμού)
3. `<condition variable>` (διαχείριση μεταβλητών συνθήκης)
4. `<future>` (διαχείριση καταστάσεων που ανακύπτουν με ασύγχρονο τρόπο)
5. `<atomic>` (υποστήριξη ατομικών τύπων και διαδικασιών)
6. `<chrono>` (διαχείριση χρονικών πληροφοριών).

Αυτές οι πολυνηματικές δυνατότητες που υποστηρίζονται από το πρότυπο C++11 έχουν χρησιμοποιηθεί για την ανάπτυξη βιβλιοθηκών που προσφέρουν πολυνηματική υποστήριξη, όπως είναι οι βιβλιοθήκες Boost και TBB (Threading Building Blocks).

Η δημιουργία ενός νήματος πραγματοποιείται από τη συνάρτηση

```
std::thread t (&thread_function);
```

που δέχεται ως όρισμα έναν δείκτη προς την συνάρτηση που θα εκτελέσει το νήμα, όταν αυτό ξεκινήσει την λειτουργία του



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Απλό παράδειγμα **δημιουργίας νήματος** στην γλώσσα προγραμματισμού **C++11**.

```
#include <iostream>
#include <thread>

void thread_function() {
    std::cout << "Hello from new thread\n"; }

int main() {
    std::thread t (&thread_function);
    std::cout << "Hello from main thread\n";
    t.join();
    return 0; }
```

```
amarg@amarg-vbox:~$ g++ threadC++.cpp -o threadC++ -std=c++11 -pthread
amarg@amarg-vbox:~$ ./threadC++
Hello from main thread
Hello from new thread
amarg@amarg-vbox:~$
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι πιο γνωστές υλοποιήσεις του πολυνηματικού μοντέλου

Η βιβλιοθήκη **TBB (Threading Building Blocks)** της **Intel** Επιτρέπει την ανάπτυξη **πολυνηματικών εφαρμογών** κατάλληλων για εκτέλεση σε **πολυπύρηνους επεξεργαστές**, είναι **ελεύθερα διαθέσιμη** στο **παγκόσμιο διαδίκτυο** και χρησιμοποιείται σε εφαρμογές που χαρακτηρίζονται από **πολύ μεγάλο υπολογιστικό κόστος** και ανήκουν σε **όλο το φάσμα** των επιστημών.

Οι **συναρτήσεις** αυτής της βιβλιοθήκης λειτουργούν σε **τρία διαφορετικά** επίπεδα:

- το **ανώτερο επίπεδο** υποστηρίζει την ανάπτυξη αλγορίθμων στην μορφή **παραμετρικών κλάσεων και συναρτήσεων**
- το **μεσαίο επίπεδο** προσφέρει **περιορισμένη πρόσβαση** στον **χρονοδρομολογητή εργασιών**
- το **κατώτερο επίπεδο** προσφέρει **πιο εξειδικευμένες λειτουργίες** που περιλαμβάνουν την **πλήρη πρόσβαση** στο σύστημα **χρονοδρομολόγησης** καθώς και την δυνατότητα ορισμού **ιεραρχικών συσχετίσεων** ανάμεσα στις **εργασίες**.

Η βιβλιοθήκη **TBB** περιλαμβάνει ένα σύνολο από **παράλληλους αλγορίθμους υψηλού επιπέδου** όπως είναι οι αλγόριθμοι

- **parallel_for** (που επιτρέπει την **αυτόματη παραλληλοποίηση** ενός βρόχου)
- **parallel_reduce** (που επιτρέπει την εφαρμογή **πράξεων υποβιβασμού** δεδομένων)
- **parallel_scan** (που επιτρέπει τον **παράλληλο υπολογισμό προθεμάτων**)
- **parallel_for_each** (που αποτελεί μία **απλοποιημένη έκδοση** του **parallel_for**)
- **parallel_do** (που είναι **παρόμοιο** με το **parallel_for**, με μικρές διαφορές)
- **parallel_invoke** (που επιτρέπει την **παράλληλη εκτέλεση** μέχρι **10 συναρτήσεων**)
- **parallel_sort** (που αποτελεί την **παράλληλη εκδοχή** αλγορίθμου **ταξινόμησης**)



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Γνωστά ζητήματα σχεδίασης πολυνηματικών εφαρμογών

Ερώτημα: εάν κάθε φορά που επιθυμούμε να **διεκπεραιώσουμε** κάποιο έργο δημιουργούμε ένα **νήμα**, το οποίο μετά από την ολοκλήρωση του έργου **τερματίζεται** και **απομακρύνεται**, αυτή η **συνεχής δημιουργία και καταστροφή νημάτων**, δεν συνιστά παράγοντα επιβάρυνσης που οδηγεί σε **μείωση** της απόδοσης; Η απάντηση είναι **καταφατική!**

Μία **πιο αποδοτική** τεχνική είναι να **δημιουργούμε** κατά την **έναρξη** της εφαρμογής και για **μία και μοναδική** φορά, μία **ομάδα νημάτων** διαθέσιμων προς χρήση, τα οποία στο τέλος της εφαρμογής **καταστρέφονται** για **μία και μοναδική** φορά.

Κατά τη διάρκεια της εκτέλεσης **σειριακού κώδικα**, αυτά τα νήματα παραμένουν **αδρανή** και σε **κατάσταση υπνώσεως**.

Κάθε φορά που **απαιτείται** η εκτέλεση **παράλληλου κώδικα**, **ενεργοποιούνται** ορισμένα από αυτά τα νήματα προκειμένου να **εκτελέσουν** το **απαιτούμενο έργο** και στη συνέχεια **μεταπίπτουν εκ νέου** στην **αδράνεια** και στην **κατάσταση υπνώσεως**.

Υπάρχουν **αρκετές υλοποιήσεις** για την **λειτουργικότητα** αυτού του τύπου, όπως είναι οι **αντικειμενοστραφείς βιβλιοθήκες SPool και NPool**, εκ των οποίων η πρώτη λειτουργεί σε **επίπεδο νήματος**, ενώ η δεύτερη σε **επίπεδο εργασίας**.

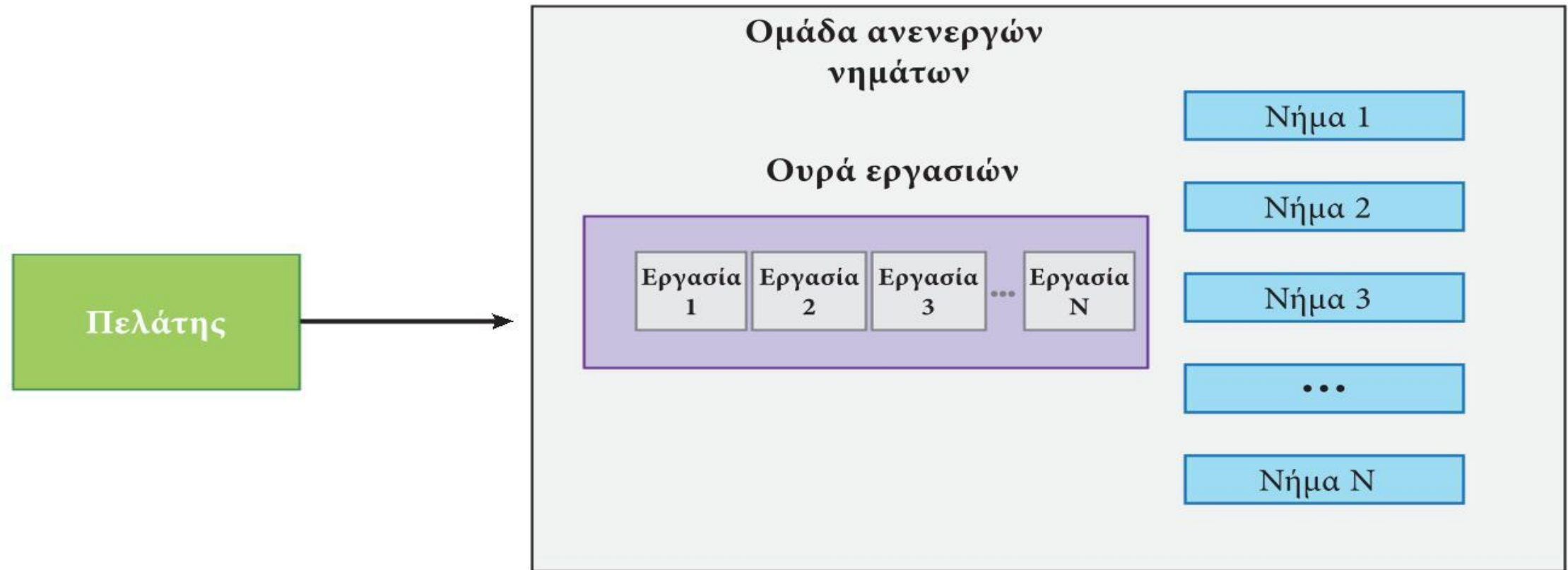
Για την περιγραφή του **τρόπου λειτουργίας** αυτών των βιβλιοθηκών, ανατρέξτε στις σελίδες **454-456**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Γνωστά ζητήματα σχεδίασης πολυνηματικών εφαρμογών

Η δημιουργία ομάδων ανενεργών νημάτων που ενεργοποιούνται και απενεργοποιούνται κατά βούληση για την παράλληλη εκτέλεση υπολογιστικού έργου κάθε φορά που αυτό απαιτείται από τις περιστάσεις, γενικά οδηγεί σε αύξηση απόδοσης.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η χρονοδρομολόγηση των πολυνηματικών εφαρμογών

Τι είδους **οντότητα** θα αποστείλουμε στον επεξεργαστή του συστήματος, προκειμένου να **εκτελεστεί**; Αυτό το ερώτημα, επιδέχεται **αρκετές απαντήσεις** και πιο συγκεκριμένα:

Εάν η εφαρμογή **δεν χρησιμοποιεί** νήματα, αυτή η **οντότητα χρονοδρομολόγησης** μπορεί να είναι είτε μία **απλή διεργασία**, είτε μία **ολόκληρη ομάδα διεργασιών** ενώ στην **αντίθετη** περίπτωση, αυτή η οντότητα μπορεί να είναι ένα **απλό νήμα**. Υφίστανται **τρεις δυνατότητες**, που οδηγούν σε **ισάριθμα** μοντέλα **πολυνηματικών εφαρμογών**:

A. Ως **οντότητα χρονοδρομολόγησης** επιλέγεται η **διεργασία**. Εάν αυτή περιέχει **νήματα**, η **δημιουργία**, η **χρονοδρομολόγηση** και ο **συγχρονισμός** των νημάτων αποτελούν ζητήματα της **διεργασίας** και όχι του **πυρήνα**, ο οποίος αντιλαμβάνεται **μόνο** την διεργασία, ενώ **δεν διαθέτει την παραμικρή γνώση** για την **εσωτερική της δομή**, η οποία περιλαμβάνει τα **νήματα**.

Πλεονεκτήματα: (α) **μεγάλη απόδοση**, αφού **δεν** απαιτείται μετάπτωση σε **κατάσταση πυρήνα** και (β) **φορητότητα**, αφού **δεν είναι αναγκαία** η υποστήριξη των νημάτων από τον πυρήνα του λειτουργικού.

Μειονεκτήματα: (α) **δεν υποστηρίζεται** δρομολόγηση σε **επίπεδο νήματος** και (β) η **αναστολή** της λειτουργίας ενός νήματος προκαλεί την **αναστολή** της λειτουργίας και όλων των **υπολοίπων** νημάτων



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η χρονοδρομολόγηση των πολυνηματικών εφαρμογών

B. Ως **οντότητα χρονοδρομολόγησης** ορίζεται το **νήμα**. Ο **πυρήνας** του λειτουργικού συστήματος **είναι ενήμερος** για την ύπαρξη των **νημάτων** και μπορεί να **χρονοδρομολογήσει** το **καθένα** από αυτά, **ξεχωριστά** και **ανεξάρτητα** από τα υπόλοιπα.

Ο **συγχρονισμός** της **λειτουργίας** των νημάτων πραγματοποιείται με τη βοήθεια **κλήσεων συστήματος** στον χώρο του πυρήνα.

Η **αναστολή** της **λειτουργίας** ενός νήματος **δεν οδηγεί** στην **αναστολή** της **λειτουργίας** και των **υπολοίπων** νημάτων, ενώ ταυτόχρονα είναι δυνατή η **ανάθεση** των **διαφορετικών νημάτων** της **ίδιας διεργασίας**, σε **διαφορετικούς** επεξεργαστές.

Η λειτουργία σε **κατάσταση πυρήνα**, οδηγεί σε **μείωση** της **απόδοσης** αφού οι διαδικασίες της **δημιουργίας νημάτων**, της **μεταγωγής περιβάλλοντος** και του **συγχρονισμού**, πραγματοποιούνται με **μικρότερη** ταχύτητα. Το λειτουργικό σύστημα **Linux** χρησιμοποιεί για την **δρομολόγηση** των διεργασιών, **αυτό ακριβώς το μοντέλο**.

Γ. Συνδυάζοντας τα δύο παραπάνω μοντέλα προκύπτει ένα **νέο μοντέλο** στο οποίο μία **διεργασία** μπορεί να περιέχει **πολλαπλές οντότητες χρονοδρομολόγησης**, ενώ **πολλά νήματα** μπορούν να **απεικονιστούν** στην **ίδια οντότητα** χρονοδρομολόγησης.

Αυτό το μοντέλο δίνει την δυνατότητα στον **πυρήνα** να αναθέσει τα **νήματα** μιας εφαρμογής σε **πολλαπλούς επεξεργαστές** και παράλληλα να διαχειριστεί τα **προβλήματα κλιμάκωσης** που προκύπτουν όταν μία εφαρμογή χρησιμοποιεί ένα **μεγάλο πλήθος νημάτων**. Αυτό το μοντέλο χρησιμοποιεί τόσο **τον χώρο του χρήστη** όσο και **τον χώρο του πυρήνα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Νήματα στον χώρο του χρήστη και στον χώρο του πυρήνα

Ένα νήμα μπορεί να υλοποιηθεί είτε **στον χώρο του χρήστη** είτε **στον χώρο του πυρήνα**, με κριτήριο το εάν ως **οντότητα χρονοδρομολόγησης** χρησιμοποιείται **αυτό το ίδιο** το νήμα, ή η **διεργασία** στην οποία αυτό ανήκει.

Τα **νήματα στον χώρο του χρήστη** δημιουργούνται από τις **εφαρμογές** των **χρηστών** και η ύπαρξή τους **δεν είναι γνωστή** στον πυρήνα, ο οποίος αντιλαμβάνεται **μόνο τις διεργασίες** στις οποίες αυτά ανήκουν.

Η υλοποίησή τους είναι **εύκολη** και **αποδοτική**, η **μεταγωγή περιβάλλοντος** είναι πιο απλή, αλλά επειδή ο πυρήνας **αγνοεί** την ύπαρξή τους, η **αναστολή της λειτουργίας** του νήματος οδηγεί σε **αναστολή της διεργασίας** που το περιέχει.

Τα **νήματα στον χώρο του πυρήνα**, υλοποιούνται από το **λειτουργικό σύστημα** και είναι **γνωστά** στον **πυρήνα**.

Η υλοποίησή τους είναι **πιο περίπλοκη**, η **μεταγωγή περιβάλλοντος** είναι **πιο συχνή** και απαιτείται **υποστήριξη** από το **υλικό**.

Η διαχείριση των **πάσης φύσεως καταστάσεων** που ενδέχεται να ανακύψουν, καθώς και η **αντιμετώπιση των προβλημάτων** που **ενδέχεται** να εμφανιστούν, είναι καθήκον του **προγραμματιστή**.

Επειδή αυτά τα νήματα **χρονοδρομολογούνται ξεχωριστά** από τα υπόλοιπα, η **αναστολή της λειτουργίας** ενός νήματος **δεν προκαλεί**, την **αναστολή της λειτουργίας ολόκληρης της διεργασίας** που το περιέχει.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Νήματα στον χώρο του χρήστη και στον χώρο του πυρήνα

Στην περίπτωση των νημάτων **στον χώρο του πυρήνα**, αν και αυτά σχετίζονται με κάποια **διεργασία**, ωστόσο, κάποιο άλλο νήμα του συστήματος, **μπορεί να κάνει αναφορά σε αυτά**, κάτι που **δεν γίνεται** για τα **νήματα στον χώρο του χρήστη**, όπου ένα νήμα που ανήκει σε μία **διεργασία A**, δεν είναι δυνατόν να κάνει αναφορά **σε ένα άλλο νήμα** που ανήκει σε μία **διεργασία B**.

Μιλώντας γενικά, ο προγραμματιστής **δεν έχει την δυνατότητα** να προχωρήσει στον **άμεσο έλεγχο** των νημάτων του πυρήνα, εκτός εάν καταφύγει στην ανάπτυξη **επεκτάσεων του πυρήνα** ή στην υλοποίηση των **κατάλληλων** οδηγών συσκευής.

Η βασική χρησιμότητα των νημάτων **στον χώρο του πυρήνα** τα οποία μοιάζουν με τους **daemons** του **Unix**, είναι η πραγματοποίηση **πάσης φύσεως μορφών διαχείρισης**, όπως είναι η **διαχείριση μνήμης** και **διεργασιών**. Για παράδειγμα, μπορούμε να αναθέσουμε σε ένα τέτοιο νήμα

- τον **περιοδικό συγχρονισμό** του περιεχομένου των **τροποποιημένων σελίδων μνήμης** με το **αντίστοιχο περιεχόμενο** της **αποθηκευτικής** διάταξης από την οποία αυτό έχει διαβαστεί
- την **αποθήκευση στην εικονική μνήμη** των **σελίδων μνήμης** που χρησιμοποιούνται με **πολύ μικρή** συχνότητα
- την πραγματοποίηση των **πάσης φύσεως συναλλαγών** που σχετίζονται με το **ημερολόγιο καταγραφής μεταβολών** (journal), που χρησιμοποιείται σε όλα τα **μοντέρνα** συστήματα αρχείων.

Αυτά τα νήματα είτε ξεκινούν για **μία και μοναδική φορά** και παραμένουν **αδρανή**, αναμένοντας την αποστολή **αιτημάτων εξυπηρέτησης** από τον πυρήνα, είτε εκτελούνται σε **τακτά χρονικά διαστήματα** ελέγχοντας την κατάσταση του συστήματος και πραγματοποιώντας **περιοδικά** την λειτουργία που τους έχει ανατεθεί, εάν οι συνθήκες απαιτούν κάτι τέτοιο.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η υλοποίηση του μηχανισμού των νημάτων στο λειτουργικό σύστημα Linux

Το **Linux** υποστηρίζει τον **μηχανισμό των νημάτων** από το **1996**, μέσω της υλοποίησης **LinuxThreads** η οποία δημιουργήθηκε από τον **Xavier Leroy**. Τα νήματα αυτής της υλοποίησης δημιουργούνται δια της χρήσεως της κλήσης συστήματος **clone** με τις σημαίες **CLONE_VM**, **CLONE_FILES**, **CLONE_FS** και **CLONE_SIGHAND**.

Εκτός από την δημιουργία των νημάτων της εφαρμογής, ελάμβανε χώρα η δημιουργία και ενός **πρόσθετου νήματος** με **συντονιστικό ρόλο**, το οποίο διαχειρίζονταν τις διαδικασίες της **δημιουργίας** και του **τερματισμού** των νημάτων, ενώ η λειτουργία του μηχανισμού, στηρίζονταν στην χρήση **σημάτων**.

Ωστόσο, αυτή η υλοποίηση χαρακτηρίζονταν από **αρκετές ασυμβατότητες** σε σχέση με τις **οριζόμενες προδιαγραφές**.

Το 2002, η υλοποίηση **LinuxThreads** αντικαταστάθηκε από την πιο σύγχρονη υλοποίηση **NPTL (Native Posix Thread Library)**, που χρησιμοποιείται **μέχρι σήμερα** και διευθετεί τα **περισσότερα από τα προβλήματα** της βιβλιοθήκης **LinuxThreads**.

Προσφέρει χαρακτηριστικά όπως νέες μέθοδοι **συγχρονισμού** και **κλήσεις συστήματος**, προηγμένες δυνατότητες **αποσφαλμάτωσης**, βελτιωμένος **χειρισμός σημάτων** και βελτίωση της λειτουργίας του **χρονοδρομολογητή** του συστήματος.

Αυτή η υλοποίηση χρησιμοποιεί τα **δύο πρώτα σήματα πραγματικού χρόνου**, με κωδικούς **32** και **33**, ενώ **δεν** χρησιμοποιεί κάποιο **κεντρικό νήμα** για τη διαχείριση των υπολοίπων τμημάτων της εφαρμογής όπως η υλοποίηση **LinuxThreads**.

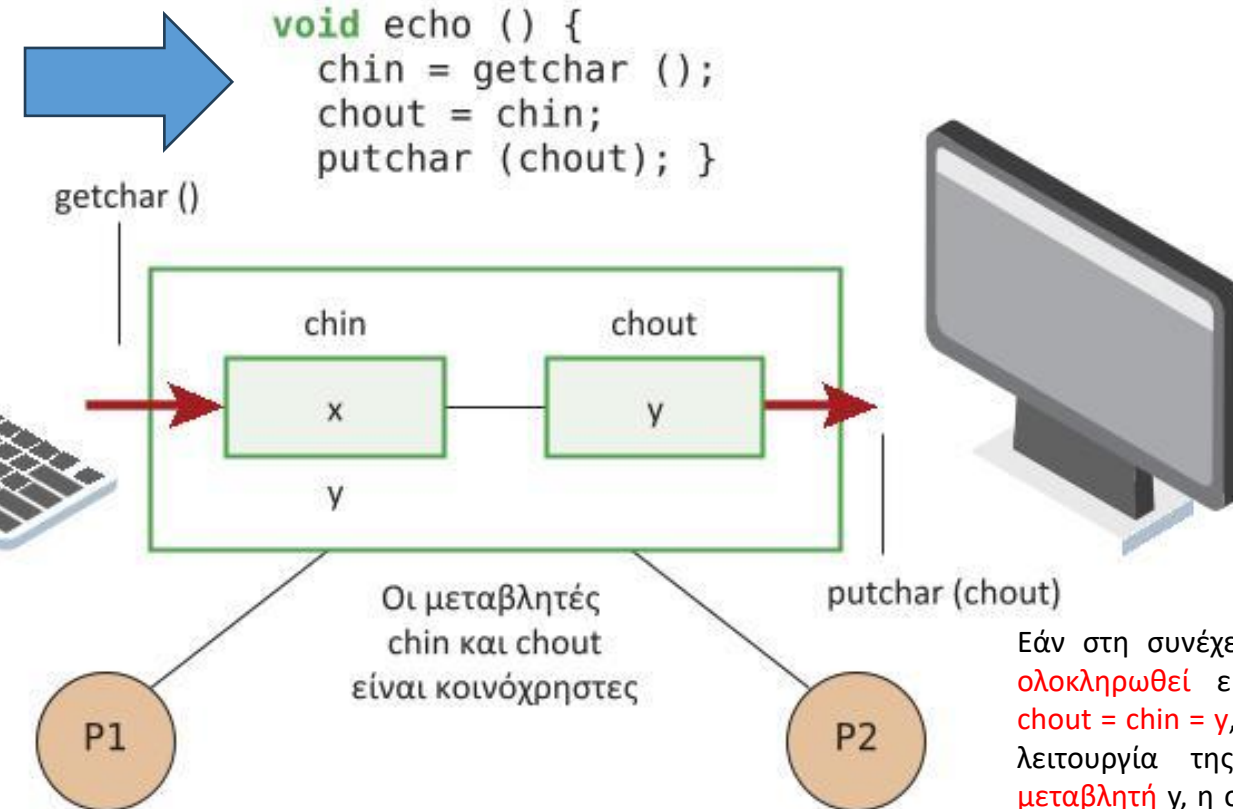


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το πρόβλημα του ταυτοχρονισμού

Σχετίζεται με την **ταυτόχρονη προσπέλαση κοινόχρηστων πόρων** από ένα σύνολο **διεργασιών** ή **νημάτων** και στην αντιμετώπιση των **συνθηκών ανταγωνισμού** που εμφανίζονται σε αυτές τις διαδικασίες και οδηγούν στην εμφάνιση **ασυνεπών δεδομένων**.

Η συνάρτηση **echo** διαβάζει από το **πληκτρολόγιο** έναν χαρακτήρα τον οποίο αποθηκεύει στη μεταβλητή **chin**, στη συνέχεια **αντιγράφει** το **περιεχόμενο** της μεταβλητής **chin** σε μία άλλη μεταβλητή **chout** και τέλος, **εκτυπώνει** το περιεχόμενο της μεταβλητής **chout** στην **οθόνη** του χρήστη



Η P_1 καλεί την **echo** και μετά από την εκτέλεση της `chin = getchar()` με τιμή `chin = x`, **αναστέλλει** την λειτουργία της.

Εάν στη συνέχεια **επανέλθει** η P_2 και **ολοκληρωθεί** εκτυπώνοντας την τιμή `chout = chin = y`, όταν η P_1 συνεχίσει την λειτουργία της, θα εκτυπώσει την **μεταβλητή** `y`, η οποία θα εκτυπωθεί **δύο φορές**, ενώ η τιμή της `x` **χάνεται**.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η αναγκαιότητα και η υλοποίηση του αμοιβαίου αποκλεισμού

Ο πιο προφανής τρόπος αντιμετώπισης τέτοιων καταστάσεων, είναι η προσπέλαση των κοινόχρηστων πόρων υπό συνθήκες ατομικής λειτουργίας, κάτι που σημαίνει πως στην περίπτωση της κοινόχρηστης μεταβλητής θα πρέπει να επιτρέψουμε μόνο σε μία διεργασία να καλέσει τη συνάρτηση *echo*. Η διαδικασία έχει ως εξής:

- Η P_1 εκτελεί την πρώτη εντολή της *echo* και αναστέλλει τη λειτουργία της
- Η P_2 αντιλαμβάνομενη πως την τρέχουσα χρονική στιγμή η P_1 εκτελεί εντολή της *echo*, αναστέλλει την λειτουργία της.
- Όταν η P_1 ενεργοποιηθεί, θα συνεχίσει τη λειτουργία της μέχρι να ολοκληρωθεί.
- Αμέσως μετά, ενεργοποιείται η P_2 και εκτελείται μέχρι το τέλος της χωρίς κανένα πρόβλημα.

Επομένως, η λύση στο πρόβλημα είναι να χρησιμοποιείται ο κοινόχρηστος πόρος από μία και μοναδική διεργασία σε κάθε χρονική στιγμή, εάν αυτός εμπλέκεται στην πραγματοποίηση διαδικασιών εγγραφής.

Αυτός ο τρόπος λειτουργίας είναι γνωστός ως αμοιβαίος αποκλεισμός αφού η κάθε διεργασία μέσω μηχανισμών που θα παρουσιαστούν στη συνέχεια, κάθε φορά που προσπελαύνει για τροποποίηση έναν κοινόχρηστο πόρο, αποκλείει την πρόσβαση σε αυτόν τον πόρο στις άλλες διεργασίες.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η έννοια του κρίσιμου τμήματος

Μία **διεργασία** λέγεται πως **βρίσκεται στο κρίσιμο τμήμα** της ή πως **εκτελεί το κρίσιμο τμήμα της**, όταν ο κώδικας που εκτελείται προχωρεί σε **τροποποίηση** του περιεχομένου ενός **κοινόχρηστου πόρου**, όπως είναι ένα **αρχείο** ή μία **περιοχή μνήμης**.

Προκειμένου μία εφαρμογή να **λειτουργήσει σωστά** και να **μην εμφανιστούν** προβλήματα **ασυνέπειας δεδομένων**, είναι **αναγκαία προϋπόθεση**, κάθε φορά που μία διεργασία εκτελεί το **κρίσιμο τμήμα της**, **καμία άλλη** διεργασία να μην έχει το δικαίωμα να εισέλθει στο **δικό της κρίσιμο τμήμα**, που σχετίζεται φυσικά με τον **ίδιο** κοινόχρηστο πόρο.

Αυτή η **απαίτηση** είναι γνωστή ως **συνθήκη αμοιβαίου αποκλεισμού** και στη βιβλιογραφία μπορούν να βρεθούν αρκετοί **αλγόριθμοι** που διασφαλίζουν την ύπαρξη μιας τέτοιας **συνθήκης**, όπως είναι οι **αλγόριθμοι του Dekker** και του **Peterson**.

Όταν μία διεργασία εισέλθει στο **κρίσιμο τμήμα** της, δεν σημαίνει πως αυτό θα εκτελεστεί **υπό καθεστώς ατομικής λειτουργίας**, δηλαδή από **την αρχή έως το τέλος του**, χωρίς να υπάρχει πιθανότητα **απότομης διακοπής** του.

Για παράδειγμα, εάν κατά τη **διάρκεια εκτέλεσης του κρίσιμου τμήματος** παρέλθει το **κβάντο χρόνου** που προσδιορίζει το **χρονικό διάστημα χρήσης του επεξεργαστή** από τη διεργασία, τότε η διεργασία θα **διακοπεί**, ακόμη και στην περίπτωση κατά την οποία εκείνη την χρονική στιγμή, η **διεργασία** εκτελεί το **κρίσιμο τμήμα** της.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ζητήματα ταυτοχρονισμού και αμοιβαίου αποκλεισμού

Υπάρχουν **πολλά ερωτήματα** που σχετίζονται με το ζήτημα του **αμοιβαίου αποκλεισμού**, όπως είναι τα ακόλουθα:

- Ποια είναι, η **σχέση** που υφίσταται ανάμεσα στις **διεργασίες** που διεκδικούν **τους ίδιους** κοινόχρηστους πόρους;
- Γνωρίζουν **η μία την ύπαρξη της άλλης ή όχι**, δηλαδή είναι **εντελώς ανεξάρτητες** μεταξύ τους;
- Είναι **ενημερωμένες** για τις τιμές των **αναγνωριστικών** των άλλων διεργασιών ή όχι;
- Υφίσταται **αλληλεξάρτηση** ανάμεσα στις **εμπλεκόμενες** διεργασίες ή όχι;

Σημειώστε, πως αν και η συνθήκη του **αμοιβαίου αποκλεισμού** επιλύει όλα τα προβλήματα χρήσης **κοινόχρηστων πόρων**, ωστόσο ενδέχεται να οδηγήσει σε **άλλα προβλήματα** ελέγχου, όπως είναι το **αδιέξοδο** και η **λιμοκτονία**.

Αδιέξοδο: ορίζεται ως η κατάσταση της **ατέρμονης αμοιβαίας αναμονής** που εκδηλώνεται όταν δύο διεργασίες P_1 και P_2 αναμένουν η μία την άλλη για **άπειρο χρονικό διάστημα** προκειμένου η **καθεμία** να λάβει τον πόρο που **κατέχει** η άλλη διεργασία προκειμένου να συνεχίσει την λειτουργία της, κάτι που **δεν πρόκειται να συμβεί ποτέ**.

Η **αντιμετώπιση** των αδιεξόδων μπορεί να πραγματοποιηθεί με **αρκετούς τρόπους** και στην βιβλιογραφία μπορούν να βρεθούν αλγόριθμοι τόσο για την **πρόληψη**, όσο και για την **ανίχνευση** και **αποφυγή** αδιεξόδων.

Λιμοκτονία: ορίζεται ως η κατάσταση της **ατέρμονης αναμονής χορήγησης** του **αιτούμενου πόρου**, επειδή ο **αλγόριθμος εκχώρησης πόρων**, αναθέτει **συνεχώς** τον αιτούμενο πόρο σε **όλες** τις υπόλοιπες διεργασίες, **εκτός από την ίδια**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ζητήματα ταυτοχρονισμού και αμοιβαίου αποκλεισμού

Από ένα **σύνολο διεργασιών** που χρησιμοποιούν **τον ίδιο** κοινόχρηστο χώρο, **μόνο μία διεργασία** επιτρέπεται σε κάθε χρονική στιγμή να βρίσκεται στο **κρίσιμο τμήμα** της, ενώ όλες οι υπόλοιπες που **αναστέλλουν την λειτουργία** τους ευρισκόμενες στο **μη κρίσιμο** τμήμα τους, θα πρέπει να **παραμένουν εκεί** χωρίς να παρεμβαίνουν στην λειτουργία των άλλων διεργασιών.

Εάν **καμία διεργασία** δεν βρίσκεται στο **κρίσιμο τμήμα** της, το λειτουργικό σύστημα θα επιτρέψει **χωρίς την παραμικρή καθυστέρηση**, την εισαγωγή στο κρίσιμο τμήμα της **σε όποια διεργασία το ζητήσει**, ενώ η παραμονή μιας διεργασίας στο κρίσιμο τμήμα της, θα λάβει χώρα **μόνο για ένα πεπερασμένο** χρονικό διάστημα.

Δεν θα πρέπει να γίνονται οι **παραμικρές υποθέσεις** σχετικά με το **πλήθος των επεξεργαστών** ή την **ταχύτητα των διεργασιών**, ενώ η όλη διαδικασία θα πρέπει να υλοποιείται έτσι ώστε **να μην εμφανίζονται** καταστάσεις **αδιεξόδου** και **λιμοκτονίας**.

Η υλοποίηση του **αμοιβαίου αποκλεισμού** μπορεί να γίνει τόσο **σε επίπεδο υλικού** όσο και **σε επίπεδο λογισμικού**.

Σε **επίπεδο υλικού**, χρησιμοποιούμε **ειδικές εντολές της γλώσσας μηχανής**, ή καταφεύγουμε σε μία προσέγγιση που στηρίζεται στην **ενεργοποίηση** και **απενεργοποίηση** των διακοπών, με **υψηλό** όμως **κόστος**.

Σε **επίπεδο λογισμικού**, ο **αμοιβαίος αποκλεισμός** υλοποιείται με χρήση **σημαφόρων** και **παρακολουθητών**, καθώς και άλλων αντικειμένων, όπως είναι οι **μεταβλητές συνθήκης** και **αμοιβαίου αποκλεισμού**, οι **σημαίες γεγονότων**, η **ανταλλαγή μηνυμάτων** ανάμεσα σε **διεργασίες**, καθώς και η **χρήση κλειδαριών**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ζητήματα ταυτοχρονισμού και αμοιβαίου αποκλεισμού

Γενικοί και δυαδικοί σημαφόροι Ένας **σημαφόρος**, αποτελεί ένα **σήμα** που ανταλλάσσεται **ανάμεσα σε διεργασίες** μέσω των συναρτήσεων **semSignal** και **semWait** προκειμένου αυτές να **συντονίσουν** την λειτουργία τους.

Ένας σημαφόρος περιγράφεται από μία **ακέραια μεταβλητή** που αρχικοποιείται σε μία **μηδενική** ή **θετική** τιμή που ισούται με το **πλήθος των διεργασιών** που μπορούν να εκδώσουν **αίτημα αναμονής** και αμέσως να **συνεχίζουν** να εκτελούνται. Αυτοί οι σημαφόροι ονομάζονται **γενικοί**, ενώ εάν οι χρησιμοποιούμενες τιμές είναι μόνο **η 0 και η 1**, χαρακτηρίζονται ως **δυαδικοί**.

Η **semWait** **μειώνει** την τιμή του σημαφόρου **κατά μία μονάδα**, ενώ η **semSignal** **αυξάνει** αυτή την τιμή κατά μία μονάδα.

Ο **συγχρονισμός** των διεργασιών πραγματοποιείται με τον ακόλουθο τρόπο.

- εάν μετά από την κλήση της **semWait** η νέα τιμή του σημαφόρου είναι **αρνητική**, η διεργασία **αναστέλλει** την λειτουργία της, ενώ στην **αντίθετη** περίπτωση, η διεργασία **εξακολουθεί να εκτελείται κανονικά**.
- εάν μετά από την κλήση της **semSignal**, η **νέα** τιμή του σημαφόρου είναι **θετική ή μηδέν**, **ενεργοποιείται** μία μπλοκαρισμένη διεργασία (εάν υπάρχει) και εκτελείται **ταυτόχρονα** με την διεργασία που **κάλεσε** την semSignal.

Εάν υπάρχουν **πολλές** μπλοκαρισμένες διεργασίες αυτές τοποθετούνται σε **ουρά αναμονής** και ανάλογα με το εάν η σειρά με την οποία **απομακρύνονται** όταν **ενεργοποιούνται** εκ νέου, **είναι καθορισμένη ή όχι**, ο σημαφόρος χαρακτηρίζεται ως **ισχυρός** ή ως **ασθενής αντίστοιχα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ζητήματα ταυτοχρονισμού και αμοιβαίου αποκλεισμού

Συγχρονισμός διεργασιών με τη χρήση σηματοφόρων



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Ζητήματα ταυτοχρονισμού και αμοιβαίου αποκλεισμού

Μεταβλητές συνθήκης και αμοιβαίου αποκλεισμού Οι μεταβλητές αμοιβαίου αποκλεισμού (mutual exclusion, mutex), λαμβάνουν μία από τις τιμές **0** και **1** που προκαλούν αντίστοιχα το **κλείδωμα** και το **ξεκλείδωμα** του **κοινόχρηστου πόρου**.

Η βασική διαφορά ανάμεσα στις **μεταβλητές αμοιβαίου αποκλεισμού** και στους **δυαδικούς σημαφόρους** είναι πως η **αύξηση** και η **μείωση** της τιμής του σημαφόρου μπορεί να πραγματοποιηθεί από **διαφορετικές διεργασίες**, ενώ, η **αύξηση** και η **μείωση** της τιμής μιας **μεταβλητής αμοιβαίου αποκλεισμού**, θα πρέπει να πραγματοποιηθεί **υποχρεωτικά** από την **ίδια** διεργασία.

Μεταβλητές συνθήκης Είναι **παρόμοιες** στο πνεύμα με τις **μεταβλητές αμοιβαίου αποκλεισμού**, με το **μπλοκάρισμα** μιας **διεργασίας** ή ενός **νήματος** και η **κατάργησή** του, να στηρίζονται στην **ικανοποίηση** ή **όχι** μιας συνθήκης.

Η **βασική διαφορά** ανάμεσά τους είναι πως η **μεταβλητή αμοιβαίου αποκλεισμού** συνιστά μία **πρωτογενή λειτουργία συγχρονισμού διεργασιών χαμηλού επιπέδου**, ενώ η **μεταβλητή συνθήκης**, εντάσσεται σε ένα **υψηλότερο** επίπεδο αφαίρεσης.

Παρακολουθητές Αποτελούν **σύνθετους** τύπους δεδομένων με **ισοδύναμη λειτουργικότητα** με αυτή των **σημαφόρων** αλλά με την δυνατότητα να χειρίζονται **πιο εύκολα περίπλοκες καταστάσεις**. Συνοδεύονται από **διαδικασίες** παρόμοιες με αυτές που συναντούμε στις **αντικειμενοστραφείς γλώσσες** και μπορούν να συσχετιστούν μόνο με **μία διεργασία** σε κάθε χρονική στιγμή.

Μεταβίβαση μηνυμάτων επιτρέπει την **επικοινωνία** και τον **συγχρονισμό** διεργασιών δια της **ανταλλαγής μηνυμάτων** συγκεκριμένης **μορφής**, **οργάνωσης** και **μεγέθους** και λειτουργούν με **παρεμποδιστικό** αλλά και με **μη παρεμποδιστικό** τρόπο.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι μηχανισμοί κλειδώματος και συγχρονισμού στο λειτουργικό σύστημα Linux

Στο **Linux**, οι μηχανισμοί που υλοποιούν τον **αμοιβαίο αποκλεισμό** περιγράφονται από **ειδικά σχεδιασμένες δομές δεδομένων** που φέρουν τα ονόματα **mutex**, **spinlock_t**, **seqcount_t**, **secklock_t**, **semaphore** και **sem**.

Τα **αντικείμενα mutex** αποτελούν **κλειδαριές ειδικού τύπου**, που ενεργοποιούνται **πριν από την χρήση** ενός **κοινόχρηστου πόρου** και **απενεργοποιούνται** με την ολοκλήρωσή της. Οι **πιο σημαντικοί** κανόνες χρήσης τους είναι οι εξής:

- σε **κάθε χρονική στιγμή**, μία **μεταβλητή mutex** μπορεί να **ενεργοποιηθεί** (και να **απενεργοποιηθεί**) από **μία και μοναδική διεργασία**, η οποία νοείται ως ο **κάτοχος** αυτού του αντικειμένου
- **δεν επιτρέπεται ο τερματισμός διεργασίας**, εάν **δεν έχουν απομακρυνθεί** όλες οι **κλειδαριές** που έχουν οριστεί από αυτή
- τα αντικείμενα τύπου **mutex** **δεν μπορούν να χρησιμοποιηθούν** μαζί με **διακοπές** που προέρχονται από **υλικό** ή **λογισμικό**.

Τα **αντικείμενα spinlock** συνιστούν **μηχανισμό συγχρονισμού χαμηλού επιπέδου** και ειδικότερα, μεταβλητές που **δεσμεύονται** και **αποδεσμεύονται** από μία **διεργασία** ή **νήμα**, προκειμένου να κλειδώσουν κάποιο **κοινόχρηστο πόρο** του συστήματος.

Πριν την εισαγωγή στο **κρίσιμο τμήμα**, ελέγχεται η **τιμή** της κλειδαριάς. Εάν είναι **0** γίνεται **1** και πραγματοποιείται **εισαγωγή** στο **κρίσιμο τμήμα**, ενώ διαφορετικά, λαμβάνει χώρα **αναστολή της λειτουργίας** και **αδιάλειπτος έλεγχος** της τιμής μέχρι αυτή να γίνει **0**. Οι διαδικασίες πραγματοποιούνται με **ατομικό τρόπο** για να **μην ανακύψουν** συνθήκες **ανταγωνισμού**.

Χρησιμοποιούνται σε **συστήματα πολυεπεξεργαστών**, ή εναλλακτικά σε **συστήματα απλού επεξεργαστή** με ενεργοποιημένη την δυνατότητα **προεκτόπισης πυρήνα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Οι μηχανισμοί κλειδώματος και συγχρονισμού στο λειτουργικό σύστημα Linux

Κλειδαριές αναγνώστη – συγγραφέα: προσφέρουν **μεγαλύτερο βαθμό ταυτοχρονισμού** σε σχέση με τον **basικό τύπο κλειδαριάς** και επιτρέπουν την **ταυτόχρονη ανάγνωση** κοινόχρηστων πόρων από **πολλά νήματα**, προσφέροντας δυνατότητες **αποκλειστικής** εγγραφής. Αποτελούνται από έναν **μετρητή ανάγνωσης** μήκους **24 bits** και από μία **σημαία**, με τιμές **0 ή 1**.

- εάν **αμφότερες** οι τιμές του **μετρητή** και της **σημαίας** είναι **μηδενικές**, η κλειδαριά έχει **δεσμευθεί** από νήμα για **εγγραφή**.
- εάν ο **μετρητής** είναι **0**, ενώ η **σημαία** είναι **1**, η κλειδαριά **αποδεσμεύεται** και είναι **διαθέσιμη προς χρήση**.
- εάν ο μετρητής έχει τιμή **$m > 0$** , ενώ η **σημαία** είναι **1**, η **κλειδαριά** έχει δεσμευθεί από **m νήματα** για ταυτόχρονη **ανάγνωση**.

Ευνοούνται οι **αναγνώστες** έναντι των **συγγραφέων**, οι οποίοι ενδέχεται να περιέλθουν σε **κατάσταση λιμοκτονίας**.

Μεταβλητές seqlock: αποτελούνται από μία **κλειδαριά** που διασφαλίζει τον **συγχρονισμό** ανάμεσα σε δύο **διεργασίες – συγγραφείς** και από έναν **μετρητή ακολουθίας** που ελέγχει την λειτουργία των **διεργασιών ανάγνωσης**.

Εάν η τιμή του μετρητή είναι **περιττή**, η κλειδαριά έχει **δεσμευτεί από συγγραφέα** και ενδεχομένως τα δεδομένα έχουν **αλλάξει**, κάτι που γίνεται εάν οι τιμές αυτής της μεταβλητής **πριν και μετά** από μία διαδικασία **ανάγνωσης** έχουν **διαφορετική** τιμή.

Αντικείμενα τύπου sem και semaphore: υλοποιούν τους **σημαφόρους**, στον χώρο του **χρήστη** και του **πυρήνα**, αντίστοιχα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Η βιβλιοθήκη pthreads αποτελεί μία βιβλιοθήκη συναρτήσεων (API, Application Programming Interface) που επιτρέπει την ανάπτυξη πολυνηματικών εφαρμογών σύμφωνα με το πρότυπο των POSIX Threads

Αυτή η βιβλιοθήκη μας δίνει την δυνατότητα να δημιουργήσουμε εύκολα πολυνηματικές εφαρμογές με πλεονεκτήματα, όπως η χρήση λιγότερων πόρων συστήματος και η ταχύτατη μεταγωγή περιβάλλοντος και την υποστήριξη δυνατοτήτων όπως

- η χρονική επικάλυψη και εναλλαγή λειτουργιών του επεξεργαστή με λειτουργίες εισόδου – εξόδου
- η δυνατότητα καθορισμού τιμών προτεραιότητας και χρονοπρογραμματισμού σε πραγματικό χρόνο
- η διαχείριση ασύγχρονων συμβάντων

Τα τρία πιο γνωστά μοντέλα εφαρμογών που μπορούν να υλοποιηθούν χρησιμοποιώντας τις συναρτήσεις της βιβλιοθήκης pthreads, είναι τα εξής (δείτε και την Διαφάνεια 20 του Κεφαλαίου 2)

- Μοντέλο ανάθεσης / διεκπεραίωσης: υπάρχει ένα νήμα - συντονιστής που διαβάζει δεδομένα εισόδου από διάφορες πηγές και τα διανέμει στα υπόλοιπα νήματα για διεκπεραίωση.
- Μοντέλο γραμμής παραγωγής: μία λειτουργία διασπάται σε ένα πλήθος υπολειτουργιών, η καθεμία από τις οποίες εξυπηρετείται και από ένα διαφορετικό νήμα.
- Μοντέλο ομότιμων νημάτων: λειτουργεί όπως το μοντέλο ανάθεσης / διεκπεραίωσης, με τη διαφορά πως το νήμα συντονιστής πραγματοποιεί και το ίδιο κάποιο ποσοστό του συνολικού πλήθους των υπολογισμών.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Η βιβλιοθήκη pthreads περιέχει αρκετές δεκάδες συναρτήσεις, οι οποίες κατατάσσονται σε αρκετές κατηγορίες, όπως

- συναρτήσεις διαχείρισης νημάτων (pthread_*****)
- συναρτήσεις διαχείρισης μεταβλητών αμοιβαίου αποκλεισμού (pthread_mutex_*****)
- συναρτήσεις διαχείρισης μεταβλητών συνθήκης (pthread_cond_*****)

Αντίστοιχες συναρτήσεις υπάρχουν και για τον χειρισμό των ιδιοτήτων των παραπάνω αντικειμένων, δηλαδή

- συναρτήσεις διαχείρισης ιδιοτήτων νημάτων (pthread_attr*****)
- συναρτήσεις διαχείρισης ιδιοτήτων μεταβλητών αμοιβαίου αποκλεισμού (pthread_mutexattr_*****)
- συναρτήσεις διαχείρισης μεταβλητών συνθήκης (pthread_condattr_*****)

Ιδιαίτερο ενδιαφέρον παρουσιάζει επίσης και μία ομάδα συναρτήσεων (pthread_key_*****), οι οποίες επιτρέπουν τον χειρισμό πρόσθετων ιδιοτήτων που σχετίζονται με τα νήματα της εφαρμογής.

Σε αντίθεση με τις κλήσεις συστήματος του λειτουργικού που επιστρέφουν την τιμή 0 σε περίπτωση επιτυχίας και την τιμή -1 σε περίπτωση σφάλματος, οι συναρτήσεις της βιβλιοθήκης pthread επιστρέφουν την τιμή 0 σε περίπτωση επιτυχίας και θετική τιμή σε περίπτωση εμφάνισης κάποιου σφάλματος, η οποία δεν είναι παρά μία από τις τιμές της μεταβλητής errno.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
#define PTHREAD_CREATE_JOINABLE      0
#define PTHREAD_INHERIT_SCHED        0
#define PTHREAD_ONCE_INIT            0
#define PTHREAD_PROCESS_PRIVATE       0
#define PTHREAD_CREATE_DETACHED      1
#define PTHREAD_EXPLICIT_SCHED        1
#define PTHREAD_PROCESS_SHARED        1
#define PTHREAD_SCOPE_SYSTEM          0
#define PTHREAD_SCOPE_PROCESS        1
```

```
typedef unsigned int pthread_key_t; /* Used for thread-specific data keys */
typedef int pthread_once_t; /* Used for dynamic package initialisation */
typedef unsigned long int pthread_t; /* Used to identify a thread */
```

```
typedef struct { /* Used to identify a thread attribute object */
    int __detachstate;
    int __schedpolicy;
    struct sched_param __schedparam;
    int __inheritsched;
    int __scope;
    size_t __guardsize;
    int __stackaddr_set;
    void *__stackaddr;
    unsigned long int __stacksize;
} pthread_attr_t;
```

```
#include <pthread.h>
```

```
int pthread_create (pthread_t *restrict thread, const pthread_attr_t *restrict attr,
    void *(*start_routine)(void *), void *restrict arg); /* Creates a new thread */
noreturn void pthread_exit (void *retval); /* Terminates the calling thread */
int pthread_cancel(pthread_t thread); /* Sends a cancellation request to a thread */
pthread_t pthread_self (void); /* Obtains the TID of the calling thread */
int pthread_equal (pthread_t t1, pthread_t t2); /* Compares thread TIDs */
int pthread_join (pthread_t thread, void **retval); /* Joins with a terminated thread */
int pthread_detach (pthread_t thread); /* Detaches a thread */
int pthread_attr_init (pthread_attr_t *attr); /* Initializes a thread attributes object (TAO) */
int pthread_attr_destroy (pthread_attr_t *attr); /* Destroys a thread attributes object (TAO) */
int pthread_attr_setdetachstate (pthread_attr_t *attr, int detachstate); /* Sets detach state attribute in TAO */
int pthread_attr_getdetachstate (const pthread_attr_t *attr, int *detachstate); /* Gets detach state attribute in TAO */
```

Οι πιο σημαντικές σταθερές, συναρτήσεις και δομές δεδομένων της βιβλιοθήκης pthreads



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Η συνάρτηση `pthread_create` Η δημιουργία ενός νήματος με την βιβλιοθήκη `pthreads` πραγματοποιείται από την συνάρτηση `pthread_create`, το πρωτότυπο της οποίας έχει τη μορφή

```
int pthread_create (pthread_t *restrict thread, const pthread_attr_t *restrict attr,  
void *(*start)(void*), void *restrict arg);
```

Αυτή η συνάρτηση **δεν χρησιμοποιείται** για την δημιουργία του **κυρίου** νήματος της εφαρμογής, αφού αυτό δημιουργείται από το **λειτουργικό σύστημα** κατά την φάση της **εκκίνησης** της, αλλά αντίθετα, δημιουργεί **νέα πρόσθετα** νήματα.

Αυτή η συνάρτηση περιλαμβάνει τα ακόλουθα ορίσματα:

- **thread**: δείκτης τύπου `pthread_t` που περιέχει το **αναγνωριστικό TID** του νήματος που δημιουργείται
- **attr**: επιτρέπει τον ορισμό **πρόσθετων χαρακτηριστικών** και **ιδιοτήτων** για το νήμα που δημιουργείται
- **start**: προσδιορίζει τη **συνάρτηση** που **καλείται** και **εκτελείται** κατά την **εκτέλεση** του νέου νήματος
- **arg**: προσδιορίζει τα ορίσματα που δέχεται η συνάρτηση **start** εφόσον χρησιμοποιούνται τέτοια ορίσματα

Σε **χαμηλό επίπεδο**, η δημιουργία του νέου νήματος πραγματοποιείται από την **κλήση συστήματος clone** η οποία καλείται με τις σημαίες `CLONE_VM`, `CLONE_FS`, `CLONE_FILES`, `CLONE_SIGHAND`, `CLONE_THREAD`, `CLONE_SYSVSEM`, `CLONE_SETTLS`, `CLONE_PARENT_SETTID`, καθώς και τις `CLONE_CHILD_CLEARTID` και `CLONE_DETACHED`



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Ο καθορισμός **πρόσθετων χαρακτηριστικών** και **ιδιοτήτων** για το νέο νήμα γίνεται μέσω του ορίσματος **attr**, που αποτελεί έναν **δείκτη** σε μία **δομή** τύπου **pthread_attr_t** μέσω της οποίας ορίζονται τα ακόλουθα:

- ο **τρόπος διαχείρισης των πόρων του** μετά από τον **τερματισμό** του (το πεδίο `__detachstate`)
- η **εμβέλεια** του νήματος (το πεδίο `scope`) όσον αφορά στον **ανταγωνισμό** με τα άλλα νήματα
- ο **τρόπος ορισμού των ιδιοτήτων** που σχετίζονται με την **χρονοδρομολόγηση** (το πεδίο `__inheritsched`)
- η **πολιτική χρονοδρομολόγησης** (το πεδίο `__schedpolicy`)
- η **προτεραιότητα** του νήματος (το πεδίο `__schedparam`)

Η **συνάρτηση pthread_exit** Ο **τερματισμός της λειτουργίας** ενός νήματος, πραγματοποιείται με αρκετούς τρόπους, όπως:

- μέσω της επιστροφής της **συνάρτησης** του ορίσματος **start** της **pthread_create**
- δια της κλήσεως της συνάρτησης **pthread_exit**, η οποία **τερματίζει ρητά** την λειτουργία του νήματος
- δια της κλήσεως της συνάρτησης **pthread_cancel**, η οποία **ακυρώνει** την εκτέλεσή του νήματος
- δια της χρήσεως της **κλήσεως συστήματος exit**, από **οποιοδήποτε** από τα νήματα της εφαρμογής
- δια της **επιστροφής κάποιας τιμής** από το **κεντρικό νήμα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Το πρωτότυπο της συνάρτησης `pthread_exit` χαρακτηρίζεται από μία σύνταξη της μορφής

```
noreturn void pthread_exit (void *retval);
```

Το όρισμα `retval` αποτελεί έναν δείκτη τύπου `void` που χρησιμοποιείται για την **επιστροφή τιμής** από τη συνάρτηση, η οποία μπορεί να **ανακτηθεί** από ένα **άλλο** νήμα, δια της χρήσεως της συνάρτησης `pthread_join`.

Η συνάρτηση `pthread_self` επιτρέπει την **ανάκτηση** του **αναγνωριστικού** του νήματος **TID** και το πρωτότυπό της έχει τη μορφή

```
pthread_t pthread_self (void);
```

Η δυνατότητα **ανάκτησης** αυτών των **αναγνωριστικών**, είναι **ιδιαίτερα σημαντική**, διότι αυτές οι παράμετροι προσδιορίζουν με **μοναδικό τρόπο** τα **νήματα** μέσα στην εφαρμογή, ενώ χρησιμοποιούνται ως **ορίσματα** σε αρκετές **συναρτήσεις διαχείρισης νημάτων**, όπως είναι οι συναρτήσεις `pthread_join` και `pthread_detach`.

Η συνάρτηση `pthread_equal` χαρακτηρίζεται από ένα **πρωτότυπο** της μορφής

```
int pthread_equal (pthread_t t1, pthread_t t2);
```

Αυτή η συνάρτηση δέχεται ως ορίσματα **δύο αναγνωριστικά νημάτων** και προχωρεί στη **σύγκριση** τους, επιστρέφοντας **μη μηδενική τιμή** εάν τα δύο αναγνωριστικά έχουν **την ίδια τιμή** και **μηδενική** τιμή, στην **αντίθετη** περίπτωση.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

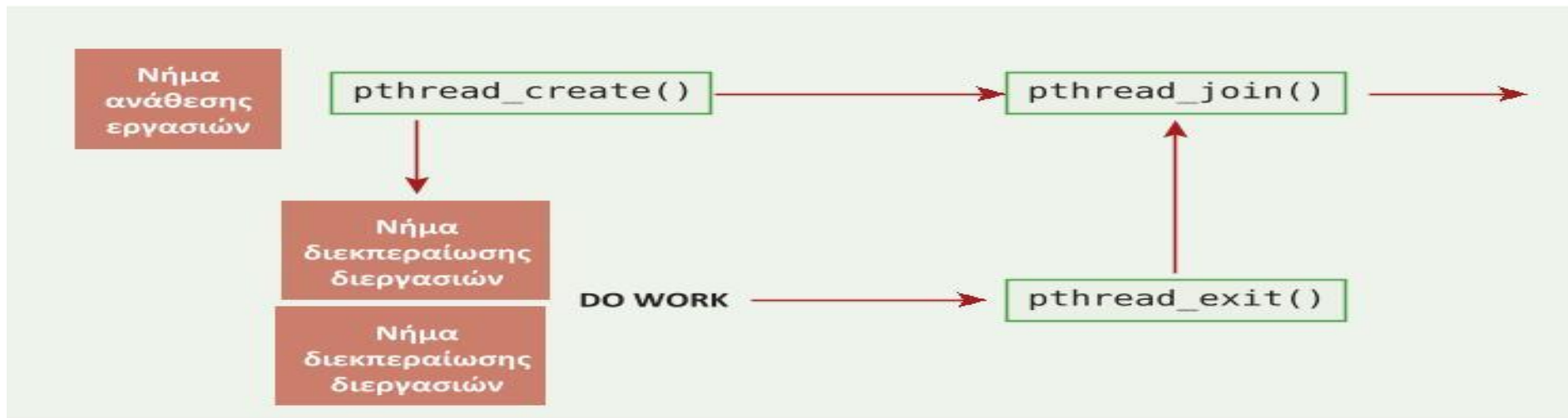
Η βιβλιοθήκη pthreads

Η συνάρτηση **pthread_join** Αυτή η συνάρτηση που χαρακτηρίζεται από ένα **πρωτότυπο** της μορφής

```
int pthread_join (pthread_t thread, void **retval);
```

δέχεται ως **πρώτο** όρισμα ένα **TID** που περιγράφεται από το όνομα **thread** και **αναγκάζει** το νήμα που την έχει καλέσει, να **αναστείλει** τη λειτουργία του, αναμένοντας τον **τερματισμό** του νήματος που προσδιορίζεται από αυτό το όρισμα, εκτός βέβαια, εάν αυτό το νήμα έχει **ήδη τερματίσει τη λειτουργία** του, οπότε η συνάρτηση **επιστρέφει άμεσα**.

Αυτή η διαδικασία περιγράφεται από το όνομα **σύνδεση** ή σύζευξη **νήματος** και απεικονίζεται στη συνέχεια



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Η συνάρτηση **pthread_detach** Επιτρέπει τον **χαρακτηρισμό** ενός νήματος ως **μη δεκτικό σύνδεσης** και κατά συνέπεια, κατά τον **τερματισμό** του θα **αποδεσμευτούν** οι πόροι που χρησιμοποίησε και θα **απομακρυνθεί** από το σύστημα **χωρίς να συνδεθεί** με κάποιο άλλο νήμα (κάτι που σημαίνει πως **δεν είναι δυνατό** να χρησιμοποιηθεί στην συνάρτηση **pthread_join**)

Αυτή η συνάρτηση χαρακτηρίζεται από ένα **πρωτότυπο** της μορφής

```
int pthread_detach (pthread_t thread);
```

και δέχεται ως όρισμα το **αναγνωριστικό του νήματος** που επιθυμούμε να χαρακτηρίσουμε με αυτόν τον τρόπο

Η συνάρτηση **pthread_cancel** Επιτρέπει την **ακύρωση της λειτουργίας** ενός νήματος ως αποτέλεσμα της εμφάνισης ενός **σφάλματος** που καθιστά **άνευ νοήματος** την συνέχισή του και χαρακτηρίζεται από ένα **πρωτότυπο** της μορφής

```
int pthread_cancel (pthread_t thread);
```

Η συνάρτηση **pthread_cancel** δεν προκαλεί την **άμεση ακύρωση** της λειτουργίας του νήματος, αλλά **αποστέλλει** στο νήμα ένα **αίτημα ακύρωσης**. Ωστόσο, το **εάν θα λάβει χώρα** αυτή η ακύρωση, ή όχι, καθώς και η **χρονική στιγμή** κατά την οποία αυτή θα πραγματοποιηθεί, **δεν είναι** κάτι που ορίζεται από την συνάρτηση **pthread_cancel**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Αυτό εξαρτάται από τις τιμές **δύο παραμέτρων** που **σχετίζονται** με αυτή την διαδικασία **ακύρωσης** και πιο συγκεκριμένα:

- την **κατάσταση** της ακύρωσης **state**
- τον **τύπο** της ακύρωσης **type**

Οι τιμές αυτών των **παραμέτρων** προσδιορίζονται μέσω των συναρτήσεων **pthread_setcancelstate** και **pthread_setcanceltype** τα **πρωτότυπα** των οποίων ακολουθούν στη συνέχεια.

```
int pthread_setcancelstate (int state, int *oldstate);  
pthread_setcanceltype (int type, int *oldtype);
```

Οι τιμές του ορίσματος **state** και η **λειτουργία** που υπαγορεύουν, ακολουθούν στη συνέχεια

PTHREAD_CANCELLED	
PTHREAD_CANCEL_DEFERRED	→ η ακύρωση πραγματοποιείται όταν προσεγγιστεί το πρώτο σημείο ακύρωσης
PTHREAD_CANCEL_ENABLE	
PTHREAD_CANCEL_ASYNCHRONOUS	→ η ακύρωση μπορεί να γίνει ανά πάσα χρονική στιγμή
PTHREAD_CANCEL_DISABLE	→ η ακύρωση δεν είναι δυνατή

Ως **σημεία ακύρωσης**, μπορούν να χρησιμοποιηθούν **γνωστές κλήσεις συστήματος**, όπως είναι οι **accept, close, connect, creat, fcntl, open, pause, read, send, write, sleep, wait, system**, ή η **pthread_testcancel** που λειτουργεί ως **σημείο ακύρωσης**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define THREADS 5

static void * threadFunc (void *arg) {
    char *s = (char *) arg;
    printf("%s", s);
    return (void *) strlen(s); }

int main(int argc, char *argv[]) {
    pthread_t threads [THREADS];
    void *res; int i, ret;
    for (i=0;i<THREADS;i++) {
        ret = pthread_create (&threads[i], NULL, threadFunc, "Hello world\n");
        if (!ret)
            printf ("Thread with TID %ld has been created succesfully\n", threads[i]); }
    printf ("Hello from the main thread !!\n");
    for (i=0;i<THREADS;i++) {
        ret = pthread_join (threads[i], &res);
        if (!ret)
            printf("Thread no %d returned %ld\n", i, (long) res); }
    exit(EXIT_SUCCESS); }
```

```
amarg@amarg-vbox:~$ gcc -c pthread2.c
amarg@amarg-vbox:~$ gcc -o pthread2 pthread2.o -pthread
amarg@amarg-vbox:~$ ./pthread2
Thread with TID 140395550988032 has been created succesfully
Thread with TID 140395542595328 has been created succesfully
Thread with TID 140395534202624 has been created succesfully
Thread with TID 140395525809920 has been created succesfully
Thread with TID 140395517417216 has been created succesfully
Hello from the main thread !!
Hello world
Thread no 0 returned 12
Hello world
Thread no 1 returned 12
Hello world
Thread no 2 returned 12
Hello world
Thread no 3 returned 12
Hello world
Thread no 4 returned 12
amarg@amarg-vbox:~$
```

Παράδειγμα χρήσης των βασικών συναρτήσεων της βιβλιοθήκης pthreads και η αντίστοιχη έξοδος της εφαρμογής



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Στην περίπτωση ταυτόχρονης προσπέλασης κοινόχρηστων πόρων από πολλά νήματα, για την πραγματοποίηση διαδικασιών εγγραφής, ανακύπτουν συνθήκες ανταγωνισμού που καθιστούν επιτακτική την ανάγκη συγχρονισμού της λειτουργίας τους.

Ειδικότερα, κατά την χρονική στιγμή που ένα νήμα τροποποιεί την τιμή μιας κοινόχρηστης μεταβλητής, τα υπόλοιπα δεν θα πρέπει να έχουν το δικαίωμα προσπέλασης αυτής της τιμής κάτι που διασφαλίζεται με μεταβλητές αμοιβαίου αποκλεισμού.

Ισοδύναμα, κατά τη διάρκεια εκτέλεσης από ένα νήμα του κρίσιμου τμήματός του, η λειτουργία του θα πρέπει να είναι αδιάλειπτη και χωρίς να διακόπτεται από τα υπόλοιπα νήματα της εφαρμογής.

Η χρήση μίας μεταβλητής αμοιβαίου αποκλεισμού, περιλαμβάνει τα επόμενα στάδια:

1. Μια μεταβλητή αμοιβαίου αποκλεισμού δεσμεύεται και αρχικοποιείται.
2. Όλα τα νήματα προσπαθούν να δεσμεύσουν αυτή τη μεταβλητή. Ωστόσο, αυτή η διαδικασία θα πραγματοποιηθεί με επιτυχία μόνο από ένα νήμα, το οποίο χαρακτηρίζεται ως ο κάτοχός της.
3. Το νήμα που κατέχει την μεταβλητή, προχωρεί σε προσπέλαση της κοινόχρηστης θέσης μνήμης που σχετίζεται με αυτή.
4. Στο τέλος της διαδικασίας, το νήμα αποδεσμεύει την μεταβλητή, η οποία πλέον είναι διαθέσιμη και μπορεί να δεσμευθεί από το ίδιο ή κάποιο άλλο νήμα.
5. Η παραπάνω διαδικασία επαναλαμβάνεται όσες φορές αυτό είναι αναγκαίο.
6. Στο τελευταίο στάδιο, η μεταβλητή αποδεσμεύεται εφόσον δεν χρειάζεται πλέον.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Οι πιο συνηθισμένοι τύποι **μεταβλητών αμοιβαίου αποκλεισμού** είναι οι εξής:

- Οι **συνήθεις** μεταβλητές, που **δεν προσφέρουν** προστασία από την εμφάνιση **αδιέξοδου**.
- Μεταβλητές που προσφέρουν **έλεγχο σφαλμάτων**.
- Μεταβλητές **αναδρομικού τύπου**, όπου **καταμετρείται** το πλήθος των **διαδικασιών κλειδώματος** και **ξεκλειδώματος**.

Η **αρχικοποίηση** μιας **μεταβλητής mutex** πραγματοποιείται είτε **στατικά** μέσω της **μακροεντολής**

```
#define PTHREAD_MUTEX_INITIALIZER {0,0,0,PTHREAD_MUTEX_NORMAL, __LOCK_INITIALIZER}
```

είτε **δυναμικά** μέσω της συνάρτησης **pthread_mutex_init** οπότε θα πρέπει να **αποδεσμευτεί** με την **pthread_mutex_destroy**

```
typedef struct {  
    int __m_reserved;  
    int __m_count;  
    pthread_descr __m_owner;  
    int __m_kind;  
    struct pthread_fastlock __m_lock;  
} pthread_mutex_t;
```

```
int pthread_mutex_init (pthread_mutex_t *restrict mutex,  
                        const pthread_mutexattr_t *restrict attr);  
int pthread_mutex_destroy (pthread_mutex_t *mutex);
```

```
int pthread_mutex_lock (pthread_mutex_t *mutex);  
int pthread_mutex_trylock (pthread_mutex_t *mutex);  
int pthread_mutex_unlock (pthread_mutex_t *mutex);
```

Η βασική **δομή δεδομένων** και οι **συναρτήσεις** **διαχείρισης** των **mutexes**

```
int pthread_mutex_timedlock (pthread_mutex_t *restrict mutex,  
                             const struct timespec *restrict abstime);
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define THREADS 5
#define SAMPLES 1000

typedef struct dotProductStruct {
    double * aVector, * bVector;
    double product;
    int samples; } dotProductStruct;

dotProductStruct dStruct;
pthread_t threads [THREADS];
pthread_mutex_t mutexSum;

void * dotProduct (void * arg) {
    double sum=0.0, * xValues, * yValues;
    int start, end, length, counter;
    long offset = (long)arg;
    length = dStruct.samples;
    start = offset*length;
    end = start+length;
    xValues = dStruct.aVector;
    yValues = dStruct.bVector;
    for (counter=start; counter<end; counter++)
        sum += (xValues[counter]*yValues[counter]);
    pthread_mutex_lock (&mutexSum);
    dStruct.product += sum;
    pthread_mutex_unlock (&mutexSum);
    pthread_exit ((void *)0);
    return (0); }

int main(int argc, char * argv[]) {
    int counter, status;
    double * xVector, * yVector;
    pthread_attr_t attribute;
    xVector = (double *) malloc (SAMPLES*THREADS*sizeof(double));
    yVector = (double *) malloc (SAMPLES*THREADS*sizeof(double));
    for (counter=0; counter<SAMPLES*THREADS; counter++) {
        xVector[counter]=(double)rand()/RAND_MAX;
        yVector[counter]=(double)rand()/RAND_MAX; }

    dStruct.aVector = xVector;
    dStruct.bVector = yVector;
    dStruct.product = 0.0;
    dStruct.samples = SAMPLES;
    pthread_mutex_init (&mutexSum, NULL);
    pthread_attr_init (&attribute);
    pthread_attr_setdetachstate (&attribute, PTHREAD_CREATE_JOINABLE);
    for (counter=0; counter<THREADS; counter++)
        pthread_create (&threads[counter],
                        &attribute, dotProduct, (void *)((long)counter));
    pthread_attr_destroy (&attribute);
    for (counter=0; counter<THREADS; counter++)
        pthread_join (threads[counter], (void **)&status);
    printf ("Vector product has a value of %.6f\n", dStruct.product);
    free (xVector); free (yVector);
    pthread_mutex_destroy (&mutexSum);
    pthread_exit (NULL);
    return (0); }
```

Υπολογισμός εσωτερικού γινομένου
διανυσμάτων με τις συναρτήσεις της
βιβλιοθήκης pthreads

```
amarg@amarg-vbox:~/shared/OpenMP$ gcc -c dotThread.c
amarg@amarg-vbox:~/shared/OpenMP$ gcc -o dotThread dotThread.o -pthread
amarg@amarg-vbox:~/shared/OpenMP$ ./dotThread
Vector product has a value of 1235.338209
amarg@amarg-vbox:~/shared/OpenMP$
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Εκτός από τις **μεταβλητές αμοιβαίου αποκλεισμού**, ο **συγχρονισμός** στηρίζεται και στις **μεταβλητές συνθήκης**, οι οποίες χρησιμοποιούνται από ένα νήμα για την **ενημέρωση** των υπολοίπων νημάτων της εφαρμογής, σχετικά με **μεταβολές της κατάστασης** μιας **κοινόχρηστης μεταβλητής mutex**, ή γενικότερα, ενός **κοινόχρηστου** πόρου.

Αυτή η ενημέρωση έχει τη μορφή μιας **ειδοποίησεως** και προκειμένου ένα νήμα να **παραλάβει** μία τέτοια **ειδοποίηση**, μπορεί να **αναστείλει** την λειτουργία του **με τον ίδιο τρόπο που το κάνει** στην περίπτωση των **μεταβλητών αμοιβαίου αποκλεισμού**.

Η χρήση των **μεταβλητών συνθήκης**, οδηγεί σε **επιτάχυνση** της διαδικασίας, αφού ένα νήμα, αντί να **ελέγχει επαναληπτικά** την κατάσταση της **mutex**, περιέρχεται, αντίθετα, σε **κατάσταση υπνώσεως**, αναμένοντας μία **ειδοποίηση** από ένα άλλο νήμα, πως **η κατάσταση της μεταβλητής έχει αλλάξει** και κατά συνέπεια, μπορεί να **συνεχίσει** την εκτέλεσή του.

Μία **μεταβλητή συνθήκης**, μπορεί να δεσμευτεί τόσο με **στατικό**, όσο και με **δυναμικό τρόπο**, ενώ **πριν** από την χρήση της, θα πρέπει να **αρχικοποιηθεί**. Εάν η μεταβλητή είναι **δυναμική**, η **αρχικοποίηση** γίνεται με τη συνάρτηση **pthread_cond_init** ενώ για την περίπτωση των **στατικών μεταβλητών**, χρησιμοποιείται η **μακροεντολή**

```
#define PTHREAD_COND_INITIALIZER {__LOCK_INITIALIZER,0}
```

Η **αποδέσμευση** μιας **δυναμικής μεταβλητής συνθήκης**, πραγματοποιείται από την συνάρτηση **pthread_cond_destroy**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
#define __LOCK_INITIALIZER { 0, 0 }
#define PTHREAD_COND_INITIALIZER {__LOCK_INITIALIZER,0}
```

```
typedef struct {
    struct _pthread_fastlock __c_lock;
    _pthread_descr __c_waiting;
    char __padding [48 - sizeof(struct _pthread_fastlock) -
                    sizeof(_pthread_descr) - sizeof(__pthread_cond_align_t)];
} pthread_cond_t;
```

```
typedef struct {
    int __dummy;
} pthread_condattr_t;
```

```
#include <pthread.h>
```

```
int pthread_cond_init (pthread_cond_t *restrict cond, const pthread_condattr_t *restrict attr);
int pthread_cond_destroy (pthread_cond_t *cond);
```

```
int pthread_cond_signal (pthread_cond_t *cond);
int pthread_cond_broadcast (pthread_cond_t *cond); }
```

```
int pthread_cond_wait (pthread_cond_t *restrict cond, pthread_mutex_t *restrict mutex);
int pthread_cond_timedwait (pthread_cond_t *restrict cond,
                             pthread_mutex_t *restrict mutex, const struct timespec *restrict abstime);
```

Οι πιο σημαντικές συναρτήσεις διαχείρισης των μεταβλητών συνθήκης.

Ενημερώνουν ένα νήμα ή ομάδα νημάτων που έχει μπλοκαριστεί αναμένοντας την ικανοποίηση κάποιας συνθήκης, πως αυτή η συνθήκη ικανοποιείται πλέον.

Αναστολή
λειτουργίας
νήματος



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Οι **μεταβλητές συνθήκης** χρησιμοποιούνται **από κοινού** με τις **μεταβλητές αμοιβαίου αποκλεισμού**. Ειδικότερα:

Αρχικά, ένα νήμα **κλειδώνει** μία **μεταβλητή αμοιβαίου αποκλεισμού**, η οποία σχετίζεται με μία **μεταβλητή συνθήκης**. Στην περίπτωση αυτή, ενδέχεται να εμφανιστούν **δύο** καταστάσεις:

- η συνθήκη **ικανοποιείται** οπότε το νήμα **συνεχίζει την εκτέλεση του** και μετά από την **ολοκλήρωση** της διαδικασίας που πραγματοποιεί, **ξεκλειδώνει** τη μεταβλητή **mutex**
- η συνθήκη **δεν ικανοποιείται** οπότε το νήμα **αναστέλλει την λειτουργία** του καλώντας τη συνάρτηση **pthread_cond_wait**, με την μεταβλητή **mutex** να **ξεκλειδώνεται αυτόματα** από το σύστημα.

Σε κάποια **μεταγενέστερη χρονική στιγμή**, κάποιο άλλο νήμα καλεί τη συνάρτηση **pthread_cond_signal** για να εκχωρήσει στη μεταβλητή συνθήκης αληθή τιμή, γεγονός που υποδηλώνει πως η εν λόγω συνθήκη πλέον ικανοποιείται.

Στην περίπτωση αυτή, λαμβάνει χώρα **αφύπνιση** του νήματος, με την **μεταβλητή mutex** να έχει κλειδωθεί **αυτόματα** από το σύστημα και αφού το νήμα **ολοκληρώσει** την διαδικασία του, προχωρεί στο **ξεκλείδωμα** της μεταβλητής **mutex**.

Με τον τρόπο αυτό, όλα τα νήματα **συνεργάζονται σωστά μεταξύ τους**, χωρίς να ανακύπτουν τα προβλήματα που σχετίζονται με την συνθήκη του **ταυτοχρονισμού**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
void *consumer(void *arg) {
    int i;
    while (1) {
        pthread_mutex_lock (&mut);
        if (items < 0) exit(1);
        while (items == 0) pthread_cond_wait (&var_cons, &mut);
        i = buffer[rem];
        rem = (rem+1) % BUF_SIZE;
        items--;
        pthread_mutex_unlock (&mut);
        pthread_cond_signal (&var_prod);
        printf ("Consume value %d\n", i);}}}
```

Προκειμένου οι διεργασίες να λειτουργήσουν **ταυτόχρονα**, απαιτείται η χρήση **κοινόχρηστου αποθηκευτικού χώρου**, στον οποίο ο παραγωγός θα **προσθέτει** αντικείμενα, τα οποία στη συνέχεια θα **απομακρύνονται** από τον καταναλωτή.

Στο πρόβλημα **παραγωγού / καταναλωτή**, συμμετέχουν **δύο διεργασίες**, η μία εξ αυτών **δημιουργεί** αντικείμενα, τα οποία, σε κάποια μεταγενέστερη χρονική στιγμή, **αναλώνονται** από την **άλλη** διεργασία.

Απαιτείται **έλεγχος πληρότητας** από τον **παραγωγό** και **έλεγχος διαθεσιμότητας** από τον **καταναλωτή**, ο οποίος πραγματοποιείται χρησιμοποιώντας **μεταβλητές συνθήκης** και **αμοιβαίου αποκλεισμού**

```
int main (int argc, char *argv[]) {
    pthread_t tid1, tid2;
    if (pthread_create (&tid1, NULL, producer, NULL) != 0) {
        printf ("Unable to create producer!\n");
        exit (EXIT_FAILURE); }
    if (pthread_create (&tid2, NULL, consumer, NULL) != 0) {
        fprintf (stderr, "Unable to create consumer!\n");
        exit (EXIT_FAILURE); }
    pthread_join (tid1, NULL);
    pthread_join (tid2, NULL);
    printf ("Manager thread is terminating now ...\n");
    return (0); }
```

Το πρόβλημα παραγωγού καταναλωτή – ο κώδικας του καταναλωτή και η συνάρτηση main



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

```
#include<pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUF_SIZE 3
#define OPERATIONS 8

int buffer[BUF_SIZE];
int insert=0, rem=0, items=0;

pthread_mutex_t mut=PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t var_cons=PTHREAD_COND_INITIALIZER;
pthread_cond_t var_prod=PTHREAD_COND_INITIALIZER;

void *producer (void *arg) {
    for (int i=1; i<=OPERATIONS; i++) {
        pthread_mutex_lock (&mut);
        if (items > BUF_SIZE) exit(EXIT_FAILURE);
        while (items == BUF_SIZE)
            pthread_cond_wait (&var_prod, &mut);
        buffer[insert] = i;
        insert = (insert+1) % BUF_SIZE;
        items++;
        pthread_mutex_unlock (&mut);
        pthread_cond_signal (&var_cons);
        printf ("producer: inserted %d\n", i); }
    printf ("producer quitting\n"); }
```

```
amarg@amarg-vbox:~$ gcc -c prodCons.c
amarg@amarg-vbox:~$ gcc -o prodCons prodCons.o -pthread
amarg@amarg-vbox:~$ ./prodCons
producer: inserted 1
producer: inserted 2
producer: inserted 3
producer: inserted 4
Consume value 1
Consume value 2
Consume value 3
Consume value 4
producer: inserted 5
producer: inserted 6
producer: inserted 7
producer: inserted 8
producer quitting
Consume value 5
Consume value 6
Consume value 7
Consume value 8
```

Το πρόβλημα παραγωγού καταναλωτή – ο κώδικας
του παραγωγού και η έξοδος της εφαρμογής



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Η βιβλιοθήκη pthreads

Μία δυσκολία που σχετίζεται με την ανάπτυξη **πολυνηματικών εφαρμογών**, είναι η **απροσδιοριστία** που χαρακτηρίζει το αποτέλεσμα της **κλήσης** μιας **συνάρτησης** από μία **ομάδα νημάτων**, εάν **δεν ληφθούν** μέτρα για την αντιμετώπιση της, ιδιαίτερα στην περίπτωση χρήσης **κοινόχρηστων καθολικών μεταβλητών**, οι οποίες διαμοιράζονται σε όλα τα νήματα.

Εάν μία τέτοια συνάρτηση χρησιμοποιείται από **ένα νήμα της εφαρμογής**, τότε η **κλήση** της από τα **υπόλοιπα νήματα**, **δεν είναι επιτρεπτή**, κάτι που συνιστά **περιοριστικό παράγοντα** που θα είναι καλό να **απομακρυνθεί**.

Αυτό μπορεί να γίνει με μία **μεταβλητή αμοιβαίου αποκλεισμού** η οποία μπορεί να **συσχετιστεί** είτε με την **συνάρτηση** είτε με την **κοινόχρηστη μεταβλητή** που αποτελεί και την **προτιμητέα λύση**, αφού επιτρέπει σε **περισσότερα του ενός νήματα** να εκτελέσουν **παράλληλα** τη συνάρτηση, εκτός εάν υπάρχουν **πολλά νήματα** που θέλουν να **εκτελέσουν** το **κρίσιμο τμήμα**.

Περαιτέρω **βελτίωση** μπορεί να προκύψει μέσω χρήσεως **συναρτήσεων επανεισόδου** οι οποίες **δεν** χρησιμοποιούν **καθολικές** και **στατικές** μεταβλητές, οπότε **δεν υπάρχει** λόγος χρήσης **αμοιβαίου αποκλεισμού**.

Αντίθετα, η **πάσης φύσεως πληροφορία** που θα πρέπει να **επιστραφεί** από αυτές τις συναρτήσεις, αποθηκεύεται σε **περιοχές μνήμης**, οι οποίες ήδη έχουν **δεσμευτεί** από την εν λόγω διεργασία, **ειδικά** για αυτό το σκοπό.

Προκειμένου να χρησιμοποιηθεί αυτή η **λειτουργικότητα** με **παλαιότερες συναρτήσεις** διατηρείται για κάθε μεταβλητή ένα **ξεχωριστό αντίγραφο** για το κάθε **νήμα** που την χρησιμοποιεί με τις τιμές που **αποθηκεύονται** σε αυτά τα αντίγραφα, διατηρούνται ανάμεσα στις **διαδοχικές κλήσεις** αυτών των συναρτήσεων από τα **εμπλεκόμενα νήματα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Το περιβάλλον του **OpenMP** (**Open MultiProcessing**) αποτελεί ένα **εργαλείο προγραμματισμού υψηλού επιπέδου** και ειδικότερα, ένα **API (Application Programming Interface)** που επιτρέπει την **εύκολη** ανάπτυξη εφαρμογών **κοινόχρηστης μνήμης**.

Η ανάπτυξή του ξεκίνησε την δεκαετία του **1990**, με την εμφάνιση των αρχιτεκτονικών **συμμετρικών πολυεπεξεργαστών κοινόχρηστης μνήμης**, οπότε η ανάγκη διατύπωσης ενός **πρότυπου** το οποίο θα ακολουθούσαν **όλες οι υλοποιήσεις** εργαλείων ανάπτυξης **πολυνηματικών εφαρμογών** για τα συστήματα αυτού του τύπου κατέστη **επιτακτική**. Γνωστές υλοποιήσεις είναι οι

- **GOMP** (GNU Offloading and Multi-Processing Project)
- **IOMP** (Intel OpenMP)

Το **OpenMP** δεν αποτελεί **ξεχωριστή βιβλιοθήκη** που εγκαθίσταται **αυτόνομα**, αλλά **αναπόσπαστο τμήμα** του μεταγλωττιστή **gcc** και πιο συγκεκριμένα, μία **βιβλιοθήκη** με όνομα **libgomp**, οι συναρτήσεις της οποίας νοούνται ως **επέκταση της γλώσσας C**.

Με άλλα λόγια, η υλοποίηση του **OpenMP** είναι **ενσωματωμένη στον μεταγλωττιστή** και κατά συνέπεια, κάθε σύστημα στο οποίο έχει **εγκατασταθεί** αυτός ο μεταγλωττιστής είναι σε θέση να χρησιμοποιηθεί για την **ανάπτυξη εφαρμογών OpenMP**, **χωρίς να χρειαστεί να εγκατασταθεί απολύτως τίποτε άλλο**.

Επομένως, η έκδοση του OpenMP **εξαρτάται** από την έκδοση του μεταγλωττιστή **gcc**. Η **τελευταία έκδοση** του OpenMP τον **Ιούνιο** του **2024** είναι η έκδοση **5.2** η οποία θα υποστηρίζεται από την **Έκδοση 13** του **gcc** που βρίσκεται **υπό ανάπτυξη**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Το γεγονός πως το OpenMP αποτελεί **επέκταση της γλώσσας C** και όχι μία **αυτόνομη** βιβλιοθήκη, προσφέρει την δυνατότητα **ταχύτατης παραλληλοποίησης** των υφιστάμενων σειριακών προγραμμάτων.

Προκειμένου να **παραλληλοποιήσουμε** τον **σειριακό** μας κώδικα, θα πρέπει να **εντοπίσουμε** τα σημεία που μπορούν να εκτελεστούν **παράλληλα** και να καταχωρήσουμε **οδηγίες** προς τον **μεταγλωττιστή** που υπαγορεύουν τον τρόπο της **παραλληλοποίησης** έτσι ώστε ο κώδικας να εκτελεστεί ως **πολυνηματική εφαρμογή** σε ένα **περιβάλλον κοινόχρηστης μνήμης**.

Η διαδικασία της **μεταγλώττισης**, που θα δημιουργήσει το **εκτελέσιμο αρχείο**, απαιτεί την χρήση κατά την κλήση του **μεταγλωττιστή gcc** του διακόπτη **-fopenmp** και την συμπερίληψη στο πηγαίο αρχείο του αρχείου **omp.h** που περιλαμβάνει:

1. Οδηγίες προς τον **μεταγλωττιστή**, που επιτρέπουν την **διαχείριση** και τον **συγχρονισμό** νημάτων.
2. **Μεταβλητές περιβάλλοντος**, που επιτρέπουν την **αρχικοποίηση παραμέτρων διαμόρφωσης**.
3. Ρουτίνες **χρόνου εκτέλεσης** (Run Time Library - RTL) που επιτρέπουν
 - την **αποκατάσταση** συνθηκών **αμοιβαίου αποκλεισμού** ανάμεσα στα νήματα
 - τον ορισμό των **παραμέτρων λειτουργίας** των παράλληλων εφαρμογών
 - τον υπολογισμό **μεγεθών**, όπως είναι η **επιτάχυνση** της εφαρμογής λόγω της παραλληλοποίησης



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η **αρχιτεκτονική** της **υλοποίησης GOMP** του **OpenMP** περιλαμβάνει **δύο επίπεδα** και παρουσιάζεται στο επόμενο σχήμα.

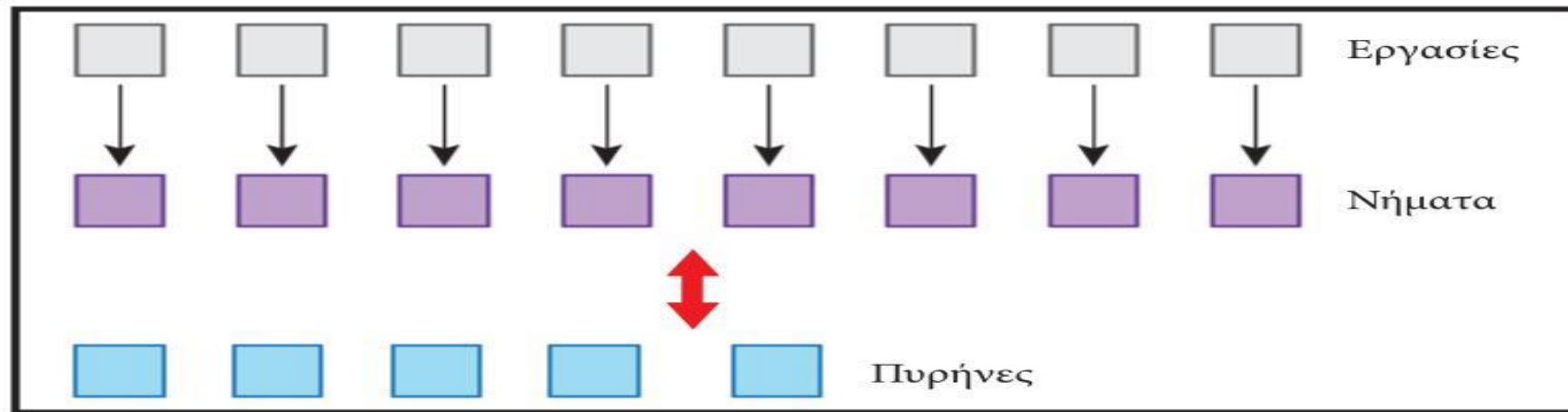


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

- το **πρώτο επίπεδο** περιλαμβάνει τον **μεταγλωττιστή** ο οποίος πραγματοποιεί
 - την **ανάλυση του πηγαίου κώδικα**
 - τον προσδιορισμό των **οδηγιών προς τον προεπεξεργαστή** που υπαγορεύουν τον **τρόπο** της παραλληλοποίησης
 - την **μετατροπή** αυτών των οδηγιών σε **κλήσεις συναρτήσεων**
- το δεύτερο επίπεδο περιλαμβάνει το **περιβάλλον χρόνου εκτέλεσης** στο οποίο υλοποιούνται οι **μηχανισμοί** και **δυναμικές δομές δεδομένων** και υλοποιείται ως η βιβλιοθήκη **libgomp**

Η **θεμελιώδης δομική μονάδα εκτέλεσης** στο **OpenMP** είναι το **νήμα** (thread), με το κάθε **ενεργό νήμα** μιας ομάδας νημάτων, να σχετίζεται με **μία και μοναδική εργασία** προς εκτέλεση, αποκαθιστώντας μία **συσχέτιση 1-προς-1** νημάτων / εργασιών.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η διαχείριση των νημάτων στο OpenMP ακολουθεί το μοντέλο παράλληλης εκτέλεσης **fork/join** και στηρίζεται στην **δυναμική δημιουργία** και **καταστροφή νημάτων** με τον κατάλληλο τρόπο, κάθε φορά που αυτό **καθίσταται αναγκαίο**.

Ο **υποκείμενος μηχανισμός** διαχείρισης νημάτων που χρησιμοποιείται από το OpenMP, είναι η **βιβλιοθήκη pthreads** με το OpenMP να προσφέρει την **δυνατότητα χρήσης** όλων των δυνατοτήτων που προσφέρει αυτή η βιβλιοθήκη αλλά με έναν **πολύ πιο εύκολο και αποδοτικό τρόπο** χωρίς τις **πολυπλοκότητες** που χαρακτηρίζουν την **απευθείας χρήση** αυτής της βιβλιοθήκης.

Η προσθήκη **οδηγιών παραλληλοποίησης** στηρίζεται σε **οδηγίες τύπου #pragma** οι οποίες μπορεί να συνοδεύονται από **προαιρετικά πεδία** που είναι χωρισμένα με **κόμμα** ή με τον χαρακτήρα του **κενού διαστήματος**.

Αυτές οι οδηγίες προσφέρουν **πρόσθετες οδηγίες** στον **μεταγλωττιστή**, πέρα από αυτές οι οποίες μεταφέρονται **από την ίδια την γλώσσα** οι οποίες εφόσον **δεν είναι γνωστές** στον μεταγλωττιστή, απλά **αγνοούνται**.

Αυτή η δυνατότητα επιτρέπει σε ένα **πρόγραμμα OpenMP** να **μεταγλωττιστεί** και να **εκτελεστεί** σε **οποιοδήποτε** σύστημα διαθέτει έναν **μεταγλωττιστή** της γλώσσας C, ανεξάρτητα από το εάν αυτός **υποστηρίζει** το **OpenMP** ή **όχι**.

Σε ένα πρόγραμμα που χρησιμοποιεί το **περιβάλλον του OpenMP** οι **οδηγίες παραλληλισμού** εντοπίζονται εύκολα ως οι γραμμές εκείνες οι οποίες ξεκινούν με την συμβολοσειρά **#pragma omp**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Μία **εφαρμογή OpenMP** ακολουθεί το **πρότυπο GSLP** (Globally Sequential Locally Parallel) – Διαφάνεια 6, Κεφάλαιο 2

Κατά την **εκτέλεση** της εφαρμογής, λαμβάνει χώρα η δημιουργία ενός **αρχικού νήματος** που εκτελεί **ακολουθιακά** τις γραμμές του κώδικα **την μία μετά την άλλη**, με τον ίδιο τρόπο με τον οποίο αυτό γίνεται στις **σειριακές** εφαρμογές

Όταν το **κεντρικό νήμα** συναντήσει την οδηγία **#pragma omp parallel** αντιλαμβάνεται πως έχει συναντήσει κώδικα ο οποίος έχει χαρακτηριστεί από τον χρήστη ως **(δυνητικά) παραλληλοποιήσιμος**.

Ως απόκριση, **αναστέλλει** την εκτέλεση της διαδικασίας που εκτελούσε μέχρι εκείνο το σημείο και δημιουργεί μία **ομάδα νημάτων**, η οποία περιλαμβάνει **το ίδιο αυτό νήμα** καθώς και το **κατάλληλο πλήθος νημάτων** και με το νήμα που δημιούργησε αυτή την ομάδα, να αναγορεύεται στο **ηγετικό νήμα** αυτής της ομάδας

Αυτά τα **πρόσθετα νήματα** που νοούνται ως **θυγατρικά νήματα**, διαθέτουν το **καθένα** το δικό του **TID** και την δική του **στοίβα** και χρησιμοποιώντας τον δικό τους **πυρήνα** εκτελούν **συλλογικά** τον **παράλληλο** κώδικα, μία εργασία το κάθε νήμα.

Το **ηγετικό νήμα** της ομάδας **δεν παραμένει ανενεργό**, αλλά **εκτελεί** και αυτό **το δικό του** ποσοστό **υπολογιστικού έργου**

Εάν το **πλήθος** της ομάδας νημάτων είναι ίσο με **N**, τότε το **TID** που εκχωρείται από το OpenMP σε **κάθε νήμα**, είναι ένας **ακέραιος** με τιμή **από 0 έως N-1** με το TID του **ηγετικού** νήματος να είναι ίσο με το **μηδέν**.



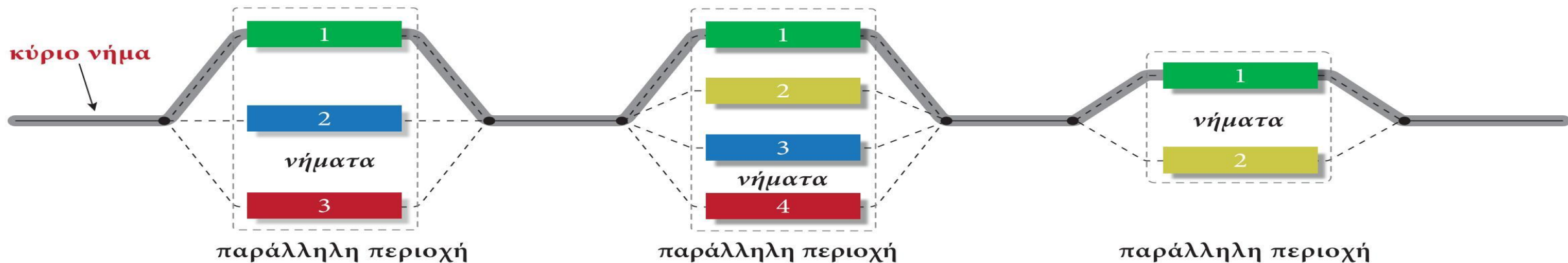
Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Όταν **όλα τα νήματα** της ομάδας νημάτων **ολοκληρώσουν** την λειτουργία τους και **τερματιστούν**, ο έλεγχος επιστρέφει στο **κεντρικό νήμα** το οποίο **συνεχίζει** τη λειτουργία του, εκτελώντας τον κώδικα που βρίσκεται **μετά από τον κώδικα της παράλληλης περιοχής**, σε πλήρη εφαρμογή του μοντέλου προγραμματισμού **fork / join**

Αυτή η διαδικασία χαρακτηρίζεται από μηχανισμό **χρονικής φραγής**, σύμφωνα με τον οποίο όταν ένα νήμα **ολοκληρώσει** την εκτέλεση του τμήματος κώδικα που του ανατέθηκε, **δεν θα τερματίσει**, αλλά αντίθετα, **θα περιμένει**, μέχρι **όλα τα υπόλοιπα νήματα** της ομάδας να **ολοκληρώσουν** και εκείνα τη λειτουργία τους και στη συνέχεια θα τερματιστούν **όλα μαζί**.

Εάν το **κεντρικό νήμα** συναντήσει **νέο παράλληλο κώδικα**, λειτουργεί **με τον ίδιο** τρόπο και αυτό γίνεται **όσες φορές απαιτείται**.



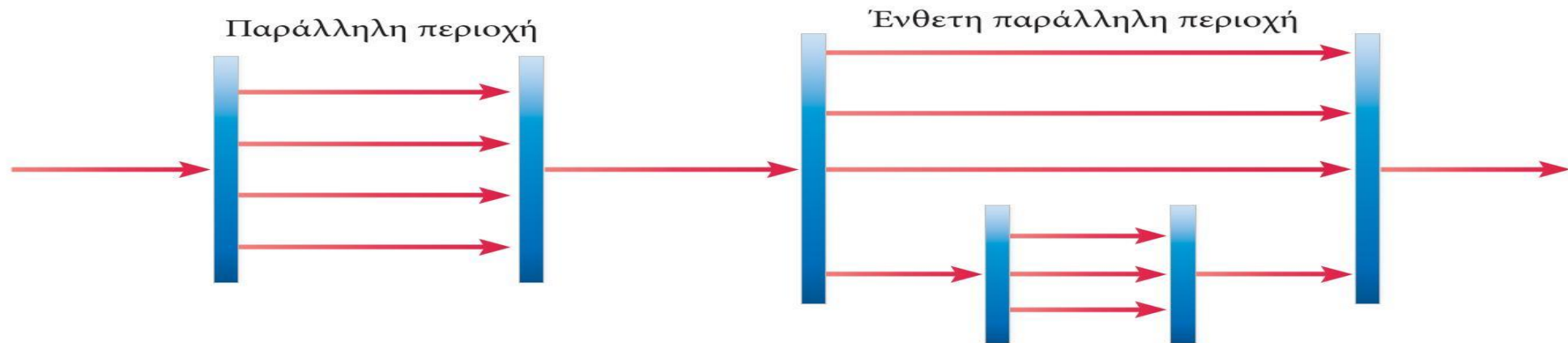
Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Στο **OpenMP**, οι εργασίες που **δημιουργούνται αυτόματα** από το **περιβάλλον χρόνου εκτέλεσης** ως απόκριση στην οδηγία **parallel** χαρακτηρίζονται ως **implicit tasks** προκειμένου να **διακριθούν** από τα **explicit tasks** που δημιουργούνται από τον **προγραμματιστή**, δια της χρήσεως της οδηγίας **task** και χαρακτηρίζονται από πολύ περισσότερες δυνατότητες

Ανάμεσα στα νήματα υφίσταται **σχέση γονέα / παιδιού**, υπό την έννοια πως το **καθένα** από τα νήματα της ομάδας αποτελεί **θυγατρικό νήμα** του νήματος που δημιούργησε αυτή την ομάδα, το οποίο με την σειρά του, αποτελεί το **γονικό νήμα**.

Το **γονικό** μαζί με τα **θυγατρικά νήματα** συνιστούν μία **ομάδα ανταγωνισμού**, αφού **ανταγωνίζονται** μεταξύ τους για την χρήση **κοινόχρηστων** πόρων, κάτι που απαιτεί την χρήση **μηχανισμών συγχρονισμού**. Υποστηρίζεται ο **ένθετος** παραλληλισμός.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ο **επεξεργαστής** στον οποίο εκτελείται το **κεντρικό νήμα** της εφαρμογής, είναι γνωστός ως **μητρική συσκευή** και στα πιο συνηθισμένα υπολογιστικά συστήματα, αποτελεί κάποιον **πολυπύρρηνο επεξεργαστή**.

Εάν το σύστημα διαθέτει **πρόσθετα στοιχεία επεξεργασίας**, όπως είναι οι **μονάδες GPU** μιας κάρτας γραφικών, αυτά αποτελούν **απομακρυσμένες συσκευές** ή **συσκευές στόχους** (target devices), στις οποίες ο προγραμματιστής μπορεί να **αναθέσει** υπολογιστικό έργο μέσω των κατάλληλων **οδηγιών** και **συναρτήσεων** του OpenMP.

Κατά την υλοποίησης αυτής της **διαδικασίας μεταφοράς υπολογιστικού έργου** και των σχετικών **δεδομένων** σε άλλες συσκευές **εκτός** της μητρικής, η κάθε συσκευή εκτελεί τα νήματα της εφαρμογής που έχουν ανατεθεί σε αυτή και τα οποία είναι **διαφορετικά** από τα νήματα της εφαρμογής που εκτελούνται σε **διαφορετικές** συσκευές.

Η **μετακίνηση** των νημάτων **ανάμεσα** σε αυτές τις συσκευές (για παράδειγμα από την **CPU** στην **GPU**) **δεν είναι δυνατή**.

Η κάθε **συσκευή** περιγράφεται από ένα **μοναδικό αναγνωστικό**, το οποίο την **ταυτοποιεί** μέσα στο σύστημα.

Η τιμή του **αναγνωριστικού** της **μητρικής συσκευής**, είναι ίση με το **συνολικό πλήθος** των **υπόλοιπων** (μη μητρικών) συσκευών, η καθεμία των οποίων περιγράφεται από το δικό της **μοναδικό αναγνωριστικό**, η τιμή του οποίου είναι **μεγαλύτερη ή ίση με το μηδέν**, αλλά **μικρότερη** από την τιμή του αναγνωριστικού της **μητρικής** συσκευής.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Καθορισμός του πλήθους των νημάτων στο OpenMP

Στο OpenMP, το πλήθος των νημάτων της παράλληλης εφαρμογής μπορεί να οριστεί με τρεις διαφορετικούς τρόπους:

1. Αρχικοποιώντας την μεταβλητή περιβάλλοντος **OMP_NUM_THREADS**, για παράδειγμα, γράφοντας

```
export OMP_NUM_THREADS=4    ή    setenv OMP_NUM_THREADS=4
```

2. Χρησιμοποιώντας την συνάρτηση **void omp_set_num_threads (int num);** για παράδειγμα, γράφοντας

```
omp_set_num_threads(4);
```

3. Χρησιμοποιώντας τον όρο **num_threads** που συνοδεύει την οδηγία **#pragma omp parallel**, για παράδειγμα, γράφοντας

```
#pragma omp parallel num_threads (threads)
```

Μιλώντας γενικά, το πλήθος των νημάτων προσδιορίζεται κατά σειρά προτεραιότητας

- από την τιμή του όρου **if**, εάν χρησιμοποιείται
- από την τιμή του όρου **num_threads**
- από τη συνάρτηση **omp_set_num_threads**
- από την μεταβλητή περιβάλλοντος **OMP_NUM_THREADS**



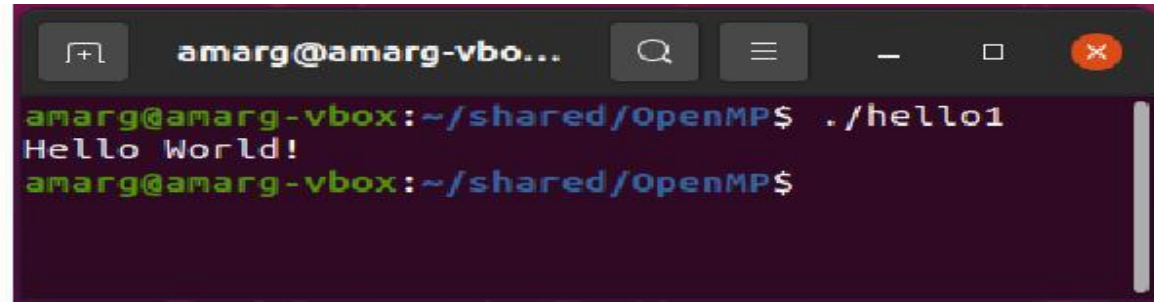
Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η **σειριακή** και η **παράλληλη** έκδοση μιας απλής εφαρμογής που εκτυπώνει ένα μήνυμα στην οθόνη του χρήστη

```
#include <omp.h>
#include <stdio.h>

int main (int argc, char **argv) {
    printf ("Hello World!\n");
    return 0; }
```

A terminal window titled 'amarg@amarg-vbo...' showing the execution of a program. The prompt is 'amarg@amarg-vbox:~/shared/OpenMP\$' and the command is './hello1'. The output is 'Hello World!' followed by a new prompt 'amarg@amarg-vbox:~/shared/OpenMP\$'.

Το **πλήθος των νημάτων** ορίζεται μέσω της μεταβλητής περιβάλλοντος **OMP_NUMTHREADS**

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv) {

    #pragma omp parallel
    {
        printf ("Hello World!\n");
        printf (" Have a nice day\n"); }
    printf ("\nHello from main thread\n");
    return 0; }
```

A terminal window titled 'amarg@amarg-vbox:~\$' showing the execution of a parallel program. The commands are 'export OMP_NUM_THREADS=4', 'gcc -o omp_hello2 -fopenmp omp_hello2.c', and './omp_hello2'. The output shows four parallel threads printing 'Hello World!' and 'Have a nice day' simultaneously, followed by the main thread printing 'Hello from main thread'.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Επίδειξη της διαδικασίας ορισμού του **πλήθους των νημάτων** της εφαρμογής από τη **γραμμή εντολών**.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char **argv) {
    int threads;
    if (argc==1) threads = omp_get_num_procs();
    → else threads = strtol (argv[1],NULL,10);

    #pragma omp parallel num_threads(threads)
    {
        printf ("Hello World!\n");
        printf (" Have a nice day\n"); }
    printf ("\nHello from main thread\n");
    return 0; }
```

```
amarg@amarg-vbox:~$ gcc -o omp_hello1 -fopenmp omp_hello1.c
amarg@amarg-vbox:~$ ./omp_hello1
Hello World!
Have a nice day
↓
Hello from main thread
amarg@amarg-vbox:~$ ./omp_hello1 2
Hello World!
Hello World!
Have a nice day
Have a nice day
Hello from main thread
amarg@amarg-vbox:~$ ./omp_hello1 3
Hello World!
Have a nice day
Hello World!
Have a nice day
Hello World!
Have a nice day
Hello from main thread
amarg@amarg-vbox:~$
```

Εάν **δεν καθοριστεί** πλήθος νημάτων, αυτό τίθεται αυτόματα στην τιμή του **πλήθους των πυρήνων του πολυπύρηνου επεξεργαστή του συστήματος**, έτσι ώστε ο **κάθε πυρήνας** να εκτελέσει **ένα και μοναδικό** νήμα

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάκτηση του αναγνωριστικού του νήματος και του συνολικού πλήθους των νημάτων

Η **ανάκτηση του αναγνωριστικού** για το **κάθε νήμα** της παράλληλης εφαρμογής, στηρίζεται στην κλήση της συνάρτησης

```
int omp_get_thread_num (void);
```

ενώ το **συνολικό πλήθος** των **διαθέσιμων νημάτων** επιστρέφεται από την συνάρτηση

```
int opt_get_num_threads (void);
```

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv) {
    int rank, size;

    #pragma omp parallel private(rank)
    {
        rank = omp_get_thread_num();
        size = omp_get_num_threads();
        printf("Hello World! from thread %d of total %d\n", rank, size);
    }
    return 0; }
```

```
amarg@amarg-vbox:~$ gcc -o omp_hello3 -fopenmp omp_hello3.c
amarg@amarg-vbox:~$ ./omp_hello3
Hello World! from thread 3 of total 4
Hello World! from thread 0 of total 4
Hello World! from thread 1 of total 4
Hello World! from thread 2 of total 4
amarg@amarg-vbox:~$
```

Το κάθε νήμα εκτυπώνει το **αναγνωριστικό του** και το **συνολικό πλήθος** των νημάτων



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η **γνώση του αναγνωριστικού** του κάθε νήματος μας επιτρέπει να το **διακρίνουμε** από τα υπόλοιπα νήματα και να του αναθέσουμε **το δικό του** υπολογιστικό έργο

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv) {
    int rank, size;
    float a=5.6, b=6.8;

    #pragma omp parallel private(rank)
    {
        rank = omp_get_thread_num();
        size = omp_get_num_threads();
        if (rank==0) printf ("I am the main thread with thread id = %d\n", rank);
        if (rank==1) {
            printf ("My thread id is %d and I perform addition\n", rank);
            printf ("Thread %d ==> The sum of a and b is %f\n", rank, a+b); }
        if (rank==2) {
            printf ("My thread id is %d and I perform subtraction\n", rank);
            printf ("Thread %d ==> The difference of a and b is %f\n", rank, a-b); }
        if (rank==3) {
            printf ("My thread id is %d and I perform multiplication\n", rank);
            printf ("Thread %d ==> The product of a and b is %f\n", rank, a*b); } }

    return 0; }
```

```
amarg@amarg-vbox:~$ gcc -o omp_hello4 -fopenmp omp_hello4.c
amarg@amarg-vbox:~$ ./omp_hello4
My thread id is 3 and I perform multiplication
Thread 3 ==> The product of a and b is 38.080002
My thread id is 1 and I perform addition
Thread 1 ==> The sum of a and b is 12.400000
My thread id is 2 and I perform subtraction
Thread 2 ==> The difference of a and b is -1.200000
I am the main thread with thread id = 0
amarg@amarg-vbox:~$
```

- Το **νήμα 0** είναι το **ηγετικό** νήμα
- Το **νήμα 1** πραγματοποιεί **πρόσθεση**
- Το **νήμα 2** πραγματοποιεί **αφαίρεση**
- Το **νήμα 3** πραγματοποιεί **πολλαπλασιασμό**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι οδηγίες περιβάλλοντος του OpenMP

Το **OpenMP** υποστηρίζει **δεκάδες οδηγίες** προς τον **μεταγλωττιστή** οι οποίες διατυπώνονται μέσω **μηχανισμών** της υποκείμενης γλώσσας προγραμματισμού (**C**, **C++** και **Fortran**) και ανήκουν σε κάποιον από τους επόμενους τύπους:

- **Δηλωτικές οδηγίες** (π.χ. **declare variant**, **declare reduction**, **declare target**, **declare mapper**): εμπλέκονται στην **δήλωση αντικειμένων** του OpenMP (**μεταβλητές**, **υπορουτίνες** και **συναρτήσεις**) και **δεν σχετίζονται** με την εκτέλεση κώδικα.
- **Εκτελέσιμες οδηγίες** (π.χ. **parallel**, **teams**, **sections**, **task**, **nowait**, **critical**): Σχετίζονται με **εκτελέσιμα τμήματα κώδικα** και υπαγορεύουν τον **τρόπο** με τον οποίο θα εκτελεστεί ο κώδικας του χρήστη.
- **Ενημερωτικές οδηγίες** (π.χ. **requires**, **assume**): δεν είναι **ούτε** δηλωτικές **ούτε** εκτελέσιμες, αλλά αντίθετα, ο ρόλος τους είναι, η **προώθηση** προς τον μεταγλωττιστή, ιδιοτήτων που σχετίζονται με τον **κώδικα του χρήστη**.
- **Βοηθητικές οδηγίες** (π.χ. **nothing**, **error**): διευκολύνουν την **αλληλεπίδραση με τον μεταγλωττιστή** καθώς και την **αναγνωσιμότητα του πηγαίου κώδικα**. Μπορεί να είναι είτε **ενημερωτικές** είτε **εκτελέσιμες** οδηγίες.
- **Αυτοδύναμες οδηγίες** (π.χ. **deprobj**, **ordered**): εκτελέσιμες οδηγίες που **δεν σχετίζονται** με κώδικα του χρήστη αλλά αντίθετα, με τον κώδικα του **περιβάλλοντος** του OpenMP.
- **Δευτερεύουσες οδηγίες** (π.χ. **scan**, **section**): **δεν είναι** εκτελέσιμες, αλλά αντίθετα εμφανίζονται **μόνο ως τμήματα εκτελέσιμων δομών** του OpenMP.

Εκτός από τους παραπάνω τύπους οδηγιών, το OpenMP υποστηρίζει και τις επονομαζόμενες **μετα-οδηγίες (metadirectives)** οι οποίες δεν είναι παρά **οδηγίες**, οι οποίες κάτω από τις **κατάλληλες συνθήκες** αναλύονται σε **άλλες** οδηγίες.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Εμπλουτίζοντας τις οδηγίες με λίστες όρων και ορισμάτων

Οι **όροι** αποτελούν **τμήμα του ορισμού** μιας οδηγίας του **OpenMP** και η χρήση τους είναι γενικά **προαιρετική**.

Εάν μία οδηγία δέχεται **περισσότερους από έναν όρους**, γενικά η σειρά με την οποία αναγράφονται αυτοί οι όροι **δεν έχει καμία σημασία**, εκτός εάν έχει οριστεί **κάτι διαφορετικό**.

Εάν ένας όρος δέχεται **ορίσματα**, αυτά γράφονται **κεκλεισμένα εντός παρενθέσεων**, το ένα **μετά** το άλλο και **χωρισμένα με κόμμα**, ενώ στην αντίθετη περίπτωση, αναγράφουμε απλά **το όνομά τους**, ενώ εάν **δύο ή περισσότεροι** όροι που σχετίζονται με **την ίδια** οδηγία **δεν δέχονται** ορίσματα, μπορούν να **ομαδοποιηθούν** στην **ίδια** απλή οδηγία.

Ανάλογα με τον **ρόλο του** και **τον τρόπο λειτουργίας** της οδηγίας με την οποία σχετίζεται

- η ύπαρξη ενός όρου σε μία οδηγία μπορεί να είναι **υποχρεωτική** αλλά **μπορεί και όχι**
- ο όρος μπορεί να εμφανίζεται στην οδηγία **πολλές φορές** ή **το πολύ μία φορά**
- ο όρος μπορεί να χρησιμοποιείται **κατ' αποκλειστικότητα** ή **όχι**
- ο όρος μπορεί να καταχωρείται **τελευταίος** στην γραμμή της δήλωσης αλλά **μπορεί και όχι**

Αναφορικά με τις **λίστες των ορισμάτων** που χρησιμοποιούνται με αυτούς τους όρους, αυτές μπορεί να περιλαμβάνουν, ονόματα **μεταβλητών** και **πινάκων**, ονόματα **συναρτήσεων** και **οδηγιών**, **παραμέτρους συναρτήσεων** και **τύπους δεδομένων**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση υπό συνθήκη – ο όρος `if`

Το **OpenMP** υποστηρίζει **παραλληλισμό υπό συνθήκη**, μέσω του οποίου είναι δυνατός ο καθορισμός **σημαντικών παραμέτρων λειτουργίας** της εφαρμογής, όπως είναι το **πλήθος των νημάτων** που θα δημιουργηθούν μέσω της **`parallel`**, καθώς και το εάν οι νέες εργασίες που θα δημιουργηθούν μέσω της οδηγίας **`task`**, θα εκτελεστούν **αμέσως**, ή θα δρομολογηθούν για **αργότερα**.

Αυτή η λειτουργικότητα **δεν υποστηρίζεται** από τις συμβατικές δομές **`if`**, αφού περιλαμβάνονται σε κώδικα που **θα εκτελεστεί από κάποιο νήμα**, ενώ εδώ αναφερόμαστε σε **αποφάσεις** που θα πρέπει να ληφθούν **πιο πριν** και ανήκουν σε **διαφορετικό επίπεδο αφαίρεσης**, όπως είναι το **εάν** θα δημιουργηθεί ένα νήμα, καθώς και το **πότε** θα εκτελεστεί

Η οδηγία **`masked`** και ο όρος **`filter`**

Η οδηγία **`masked`** επιτρέπει την **εκτέλεση** του σχετιζόμενου κώδικα, **όχι από όλα τα νήματα** της τρέχουσας ομάδας, αλλά από **ένα μόνο υποσύνολο** αυτών των νημάτων, ενώ τα υπόλοιπα νήματα **δεν συμμετέχουν** σε αυτή την διαδικασία.

Το **υποσύνολο** των νημάτων της ομάδας που **θα εκτελέσει** τον κώδικα, προσδιορίζεται από τον όρο **`filter`** της οδηγίας **`masked`**.

Εάν το όρισμα αυτού του όρου είναι το **αναγνωριστικό του τρέχοντος νήματος στην τρέχουσα ομάδα**, τότε ο όρος **`filter`** επιλέγει αυτό το **συγκεκριμένο** νήμα, ενώ εάν **δεν χρησιμοποιηθεί** αυτός ο όρος, τότε το σύστημα επιλέγει το **κύριο νήμα** της ομάδας, δηλαδή συμπεριφέρεται όπως θα το έκανε εάν χρησιμοποιούσαμε τον όρο **`filter`** με τιμή ορίσματος **ίση με το μηδέν**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Η κάθε **μεταβλητή** που δηλώνεται και χρησιμοποιείται σε ένα πρόγραμμα, χαρακτηρίζεται και από την **εμβέλειά** της, η οποία ορίζεται ως τα **τμήματα εκείνα του προγράμματος** στα οποία η μεταβλητή **είναι γνωστή** και **μπορεί να χρησιμοποιηθεί**.

Υπάρχουν **δύο τύποι μεταβλητών** που χρησιμοποιούνται σε ένα πρόγραμμα, και πιο συγκεκριμένα

- **τοπικές μεταβλητές**, που δηλώνονται και χρησιμοποιούνται **μέσα στον κώδικα** μίας συνάρτησης
- **καθολικές μεταβλητές**, που δηλώνονται **στην αρχή** του αρχείου και **έξω από κάθε συνάρτηση**.

Οι μεταβλητές της **πρώτης κατηγορίας** είναι προσπελάσιμες **μόνο μέσα από τον κώδικα** που υλοποιείται από την συνάρτηση, ενώ οι μεταβλητές της **δεύτερης κατηγορίας**, μπορούν να προσπελαστούν **από κάθε συνάρτηση** του προγράμματος.

Επομένως, οι **τοπικές** μεταβλητές αποτελούν **ιδιωτικές** μεταβλητές, ενώ οι **καθολικές** μεταβλητές μπορούν να θεωρηθούν **κοινόχρηστες** μεταβλητές η προσπέλαση των οποίων θα πρέπει να γίνεται με **ιδιαίτερη προσοχή**.

Στο **OpenMP** η κατάσταση είναι παρόμοια, με την **διαφορά** πως η **ασάφεια** που χαρακτηρίζει τον ρόλο των μεταβλητών εκείνων που **δεν περιλαμβάνονται εξ ολοκλήρου στον κώδικα που σχετίζεται με κάποια εργασία**, καθιστά **επιτακτική** την υποβολή **οδηγιών προς τον μεταγλωττιστή** σχετικά με την **εμβέλεια** και τον **τρόπο αντιμετώπισης** αυτών των μεταβλητών.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Μία μεταβλητή που βρίσκεται **εκτός της main** είναι προφανώς, **κοινόχρηστη** και **ορατή** από **όλα** τα νήματα

Μία μεταβλητή που βρίσκεται **εντός ενός τμήματος κώδικα**, είναι προφανώς, **τοπική** και **ορατή** μόνο από **το τρέχον** νήμα

Τι γίνεται με τις μεταβλητές στις οποίες γίνεται αναφορά τόσο **στο εσωτερικό** όσο και **στο εξωτερικό** ενός τμήματος παράλληλου κώδικα; θα υφίσταται **ένα και μοναδικό κοινόχρηστο αντίγραφο** το οποίο θα **διαμοιράζεται σε όλα τα νήματα** της εφαρμογής, ή μήπως το **κάθε νήμα**, θα διαθέτει το **δικό του τοπικό αντίγραφο** για αυτή την μεταβλητή;

Στο **OpenMP** υπάρχουν οι επόμενοι τύποι μεταβλητών οι οποίες δηλώνονται με οδηγίες τύπου **#pragma**

Ιδιωτικές μεταβλητές (private): δημιουργείται **ένα και μοναδικό τοπικό αντίγραφο** στην **στοίβα** του κάθε νήματος

Κοινόχρηστες μεταβλητές (shared): **όλα** τα νήματα που εκτελούν τον **παράλληλο** κώδικα έχουν το **δικαίωμα** να προσπελάσουν το **αυθεντικό αντίγραφο** της μεταβλητής το οποίο βρίσκεται αποθηκευμένο στην **στοίβα** του γονικού νήματος

```
#pragma omp parallel private (x,y,z) shared (a,b,c)
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Η προεπιλεγμένη εμβέλεια για όλες τις μεταβλητές που δηλώνονται στην συνάρτηση `main` και πριν από ένα τμήμα παράλληλου κώδικα, είναι η κοινόχρηστη εμβέλεια, έτσι ώστε αυτές να είναι προσπελάσιμες από όλα τα νήματα της ομάδας που ξεκινούν την λειτουργία τους όταν ο μεταγλωττιστής συναντήσει κάποια οδηγία `parallel`.

Οι μεταβλητές που δηλώνονται εντός του τμήματος του παράλληλου κώδικα έχουν ιδιωτική εμβέλεια, όπως συμβαίνει, για παράδειγμα, με τις τοπικές μεταβλητές συναρτήσεων.

Η τιμή μιας κοινόχρηστης μεταβλητής στην αρχή ενός τμήματος παράλληλου κώδικα, είναι η ίδια με την τιμή πριν από αυτό το τμήμα, ενώ μετά από την ολοκλήρωση του κώδικα, η τιμή της μεταβλητής είναι αυτή που είχε στο τέλος αυτού του τμήματος.

Ο ορισμός της εμβέλειας για την κάθε μεταβλητή που χρησιμοποιείται σε ένα τμήμα παράλληλου κώδικα και έχει δηλωθεί έξω από αυτό είναι καλό να γίνεται από τον προγραμματιστή και όχι από το OpenMP.

Χρησιμοποιώντας τον όρο `default (none)`, δηλώνουμε πως δεν υφίσταται κάποιος προεπιλεγμένος τύπος εμβέλειας και κατά συνέπεια, για την κάθε μεταβλητή που χρησιμοποιείται μέσα στον παράλληλο κώδικα, θα πρέπει να δηλώσουμε ρητά την εμβέλειά της χρησιμοποιώντας τους όρους `private` ή `shared` με τον τρόπο που προαναφέραμε.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Μεταβλητές `firstprivate`: συμπεριφέρονται όπως οι **ιδιωτικές μεταβλητές**, με την διαφορά πως **αρχικοποιούνται** στην τιμή μίας **κοινόχρηστης** μεταβλητής που βρίσκεται **έξω** από ένα τμήμα **εκτελέσιμου παράλληλου κώδικα**.

η `x[0]` αρχικοποιείται **εκτός**
του παράλληλου βρόχου



```
x[0]=my_function()
#pragma omp parallel for private(j) firstprivate(x)
for (i=0;i<M;i++)
    for (j=1;j<N;j++)
        x[j]=g(i,x[j-1])
```



για να χρησιμοποιηθεί **με τον σωστό τρόπο** ενός του εσωτερικού βρόχου (**για `j=1`**) θα πρέπει να δηλωθεί ως **`firstprivate`**

Μεταβλητές `lastprivate`: συμπεριφέρονται όπως οι **ιδιωτικές μεταβλητές**, με την διαφορά πως το νήμα που **ολοκληρώνει** την **τελευταία επανάληψη** ενός **σειριακού** κώδικα, **αντιγράφει** την τιμή της μεταβλητής στο **εξωτερικό περιβάλλον** του παράλληλου κώδικα, έτσι ώστε αυτή να χρησιμοποιηθεί στον **σειριακό** κώδικα που βρίσκεται **αμέσως μετά τον παράλληλο κώδικα**.

```
#pragma omp parallel for private(j) lastprivate(x)
for (i=0;i<N;i++)
    x[0]=1.0;
    for (j=1;j<4;j++)
        x[j]=x[j-1]*(i+1);
    sum[i]=x[0]+x[1]+x[2]+x[3];
    result = x[3];
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Μεταβλητές `threadprivate`: Για την **καθεμία** από αυτές τις μεταβλητές, δημιουργείται ένα **ξεχωριστό αντίγραφο** στην μνήμη του **κάθε** νήματος, το οποίο **αρχικοποιείται** για **μία και μοναδική φορά** και σε μία χρονική στιγμή **πριν από την πρώτη αναφορά** σε αυτό το αντίγραφο, η οποία, ωστόσο, είναι **απροσδιόριστη**.

Αυτές οι μεταβλητές διατηρούνται στη μνήμη **για όλο το χρονικό διάστημα εκτέλεσης**, σε αντίθεση με μία **ιδιωτική** μεταβλητή, η οποία υφίσταται **μόνο κατά το χρονικό διάστημα εκτέλεσης** του **παράλληλου** κώδικα και στη συνέχεια **καταστρέφεται**. Αυτό σημαίνει πως οι μεταβλητές **τύπου `threadprivate`** δηλώνονται **έξω** από το τμήμα του **παράλληλου** κώδικα.

Κανένα νήμα **δεν έχει το δικαίωμα** να αναφερθεί στο αντίγραφο μίας **μεταβλητής `threadprivate`** κάποιου **άλλου** νήματος.

Οι όροι **`copyin`** και **`copyprivate`** προσφέρουν την δυνατότητα **αντιγραφής δεδομένων** ανάμεσα στα νήματα της εφαρμογής.

Ο όρος **`copyin`** επιτρέπει την **αρχικοποίηση** μεταβλητών **τύπου `threadprivate`** μιας ομάδας νημάτων, εκχωρώντας σε αυτές, την τιμή της **αντίστοιχης μεταβλητής τύπου `threadprivate`** του **κύριου** νήματος αυτής της ομάδας

Ο όρος **`copyprivate`** που σχετίζεται **αποκλειστικά** με την οδηγία **`single`** προσφέρει έναν **μηχανισμό** μέσω του οποίου η τιμή μιας **ιδιωτικής μεταβλητής τύπου `threadprivate`** που ανήκει στο περιβάλλον δεδομένων ενός **`implicit task`**, αντιγράφεται στο περιβάλλον δεδομένων **όλων των εργασιών αυτού του τύπου**, οι οποίες εκτελούν **την ίδια** παράλληλη περιοχή.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η εμβέλεια των μεταβλητών στο OpenMP

Οι **οδηγίες του OpenMP** και οι **αντίστοιχοι όροι** που καθορίζουν την **εμβέλεια** των μεταβλητών που χρησιμοποιούνται σε αυτές. Στον πίνακα απεικονίζονται **οι πιο σημαντικές** οδηγίες

	parallel	sections	task	single	for	simd	teams
private	X	X	X	X	X	X	X
shared	X		X				X
default	X		X				X
firstprivate	X	X	X	X	X		X
lastprivate		X			X	X	
copyin	X						
copyprivate				X			
reduction	X	X			X	X	X

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Εσωτερικές μεταβλητές ελέγχου (Internal Control Variables, ICVs)

Σε χαμηλό επίπεδο λειτουργίας, η συμπεριφορά του OpenMP υπαγορεύεται από τις τιμές ενός συνόλου εσωτερικών μεταβλητών ελέγχου (ICV) οι οποίες περιέχουν πληροφορίες για σημαντικές παραμέτρους του συστήματος, όπως είναι

- το πλήθος των νημάτων που θα χρησιμοποιηθούν όταν ο μεταγλωττιστής συναντήσει μία οδηγία parallel
- η ενεργοποίηση ή απενεργοποίηση της δυνατότητας χρήσης ένθετων παράλληλων περιοχών
- ο καθορισμός του μέγιστου πλήθους των νημάτων και των επιπέδων ένθεσης

Η καθεμία τις μεταβλητές ICV χαρακτηρίζεται από την εμβέλειά της με την τρέχουσα έκδοση του OpenMP, να υποστηρίζει τέσσερις δυνατότητες όσον αφορά στον τύπο αυτής της εμβέλειας, αφού μία μεταβλητή αυτού του τύπου μπορεί

- να χαρακτηρίζεται από καθολική εμβέλεια (global), οπότε η τιμή της επηρεάζει την συμπεριφορά όλων των νημάτων
- να είναι ορατή σε επίπεδο συσκευής (device) του OpenMP με διαφορετική τιμή για νήματα διαφορετικών συσκευών
- να είναι ορατή σε επίπεδο εργασίας του OpenMP (implicit task) με διαφορετική τιμή για κάθε εργασία της εφαρμογής
- να σχετίζεται με κάποιο περιβάλλον δεδομένων (data environment) το οποίο σχετίζεται με περιοχή παράλληλου κώδικα



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Εσωτερικές μεταβλητές ελέγχου (Internal Control Variables, ICVs)

Οι **μεταβλητές ICV** μπορούν να ομαδοποιηθούν σε **τρεις κατηγορίες** με κριτήριο τον **τύπο** της λειτουργίας τους:

- στις **μεταβλητές** που επηρεάζουν τον **τρόπο της εκτέλεσης** του προγράμματος
- στις **μεταβλητές** που ελέγχουν την **εκτέλεση των παράλληλων περιοχών**
- στις **μεταβλητές** που ελέγχουν την **συμπεριφορά της εφαρμογής σε καθολικό επίπεδο**.

Σημαντικές **μεταβλητές ICV** της τρέχουσας έκδοσης του **OpenMP** είναι οι εξής (για την **πλήρη λίστα**, δείτε τις σελίδες **502-504**):

- **active-levels-var** Προσδιορίζει το **βάθος της ένθεσης** του παράλληλου κώδικα που εκτελεί τη συνάρτηση
- **bind-var** Ελέγχει την **ανάθεση** των **νημάτων του OpenMP** στους **πυρήνες** του συστήματος
- **cancel-var** Ελέγχει την **συμπεριφορά της δήλωσης cancel** και των **σημείων ακύρωσης**
- **def-allocator-var** Ελέγχει την συμπεριφορά του **συστήματος δέσμευσης μνήμης**
- **max-active-levels-var** Ελέγχει το **μέγιστο πλήθος των ενεργών ένθετων παράλληλων περιοχών**
- **max-task-priority-var** Ορίζει την **μέγιστη τιμή προτεραιότητας** που μπορούμε να ορίσουμε μέσω της **οδηγίας priority**
- **ntteams-var** Ελέγχει το **πλήθος των ομάδων νημάτων** που έχουν ζητηθεί κατά την εκτέλεση της εφαρμογής
- **nthreads-var** Προσδιορίζει το **πλήθος των νημάτων** που θα ζητηθούν από το σύστημα
- **num-procs-var** Ορίζει το **πλήθος των επεξεργαστών** που είναι **διαθέσιμοι** για την **εκτέλεση νημάτων** της εφαρμογής.
- **thread-limit-var** Ελέγχει το **μέγιστο πλήθος νημάτων** που μπορούν να χρησιμοποιηθούν από μία εφαρμογή του OpenMP.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Υπολογισμός του πλήθους των νημάτων μιας παράλληλης εφαρμογής

```
let ThreadsBusy be the number of OpenMP threads currently executing in this contention group;  
if an if clause exists  
then let IfClauseValue be the value of the if clause expression;  
else let IfClauseValue = true;  
if a num_threads clause exists  
then let ThreadsRequested be the value of the num_threads clause expression;  
else let ThreadsRequested = value of the first element of nthreads-var;  
let ThreadsAvailable = (thread-limit-var - ThreadsBusy + 1);  
if (IfClauseValue = false)  
then number of threads = 1;  
else if (active-levels-var >= max-active-levels-var)  
then number of threads = 1;  
else if (dyn-var = true) and (ThreadsRequested <= ThreadsAvailable)  
then 1 <= number of threads <= ThreadsRequested  
else if (dyn-var = true) and (ThreadsRequested > ThreadsAvailable)  
then 1 <= number of threads <= ThreadsAvailable  
else if (dyn-var = false) and (ThreadsRequested <= ThreadsAvailable)  
then number of threads = ThreadsRequested;  
else if (dyn-var = false) and (ThreadsRequested > ThreadsAvailable)  
then behavior is implementation defined;
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Καταμερισμός υπολογιστικού έργου μέσω παραλληλοποίησης βρόχων

Προκειμένου η διαδικασία της **παραλληλοποίησης βρόχου** να πραγματοποιηθεί με **επιτυχία**, θα πρέπει:

- η διατύπωση του **βρόχου** θα πρέπει να είναι τέτοια ώστε να είναι δυνατός ο εκ **των προτέρων υπολογισμός** του **συνολικού πλήθους** των **επαναλήψεων** του βρόχου
- ο βρόχος θα πρέπει να είναι εκπεφρασμένος σε **κανονική μορφή** (δείτε την **Διαφάνεια 21** του **Κεφαλαίου 2**)

Κατά συνέπεια, η **αυτόματη παραλληλοποίηση** είναι δυνατή **μόνο** για τους **κανονικοποιημένους βρόχους** τύπου **for**

Αρκετοί **μεταγλωττιστές** έχουν την δυνατότητα να πραγματοποιούν **αυτόματα** αυτή την **διαδικασία κανονικοποίησης** του βρόχου, αφού αυτή η διαδικασία είναι μεταξύ άλλων **ιδιαίτερα χρήσιμη** σε διαδικασίες **βελτιστοποίησης** του βρόχου

Προκειμένου ένας **αυτόματα παραλληλοποιημένος** βρόχος να λειτουργήσει **σωστά**, θα πρέπει **να μην περιέχει** εξαρτήσεις που να **μεταφέρονται από τον βρόχο**, κάτι που **δεν ισχύει** στην αντίθετη περίπτωση.

Κατά την διάρκεια της **παράλληλης εκτέλεσης** του βρόχου, το **κεντρικό νήμα** δημιουργεί τα κατάλληλα **νήματα** με την χρήση του **μηχανισμού fork**, τα οποία λειτουργούν **από κοινού** για την **εκτέλεση** των **επαναλήψεων** του βρόχου.

Το **κάθε νήμα** έχει τον **δικό του χώρο διευθύνσεων** και τις **δικές του μεταβλητές**, καθώς και άλλα **πάσης φύσεως αντικείμενα**, όπως είναι **στατικές μεταβλητές** και **δυναμικά δεσμευμένες** δομές δεδομένων, που αποθηκεύονται στον **σωρό** και στη **στοίβα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Καταμερισμός υπολογιστικού έργου μέσω παραλληλοποίησης βρόχων

Το **OpenMP** έχει την **δυνατότητα** να **παραλληλοποιήσει αυτόματα** έναν βρόχο καταφεύγοντας σε μία **προεπιλεγμένη κατανομή** των **επαναλήψεων** του βρόχου σε αυτά τα νήματα, η οποία χαρακτηρίζεται από **ισοκατανομή των επαναλήψεων** στα νήματα

Προκειμένου η εφαρμογή να λειτουργήσει με αυτόν τον τρόπο, **ακριβώς πριν** από την εντολή του βρόχου τοποθετείται η **οδηγία**

`#pragma omp for` ή καλύτερα **σε ένα βήμα η** `#pragma omp parallel for`

```
#include <stdio.h>
#include <omp.h>
```

```
int main(int argc, char **argv) {
    int rank, size, i;
    omp_set_num_threads(4);
```

```
    #pragma omp parallel private(rank)
    {
```

```
        rank = omp_get_thread_num();
        size = omp_get_num_threads();
```

```
        #pragma omp for
```

```
        for (i=1; i<=14; i++)
```

```
            printf("Thread %02d of %d ==> Iteration No %02d\n",
                   rank, size, i); }
```

```
    return 0; }
```

εάν ανάμεσα στις
οδηγίες **parallel** και **for**
δεν παρεμβάλλονται
άλλες γραμμές κώδικα

```
amarg@amarg-vbox:~$ gcc -o omp_for1 -fopenmp omp_for1.c
amarg@amarg-vbox:~$ ./omp_for1
Thread 03 of 4 ==> Iteration No 12
Thread 03 of 4 ==> Iteration No 13
Thread 03 of 4 ==> Iteration No 14
Thread 02 of 4 ==> Iteration No 09
Thread 02 of 4 ==> Iteration No 10
Thread 02 of 4 ==> Iteration No 11
Thread 00 of 4 ==> Iteration No 01
Thread 00 of 4 ==> Iteration No 02
Thread 00 of 4 ==> Iteration No 03
Thread 00 of 4 ==> Iteration No 04
Thread 01 of 4 ==> Iteration No 05
Thread 01 of 4 ==> Iteration No 06
Thread 01 of 4 ==> Iteration No 07
Thread 01 of 4 ==> Iteration No 08
amarg@amarg-vbox:~$
```

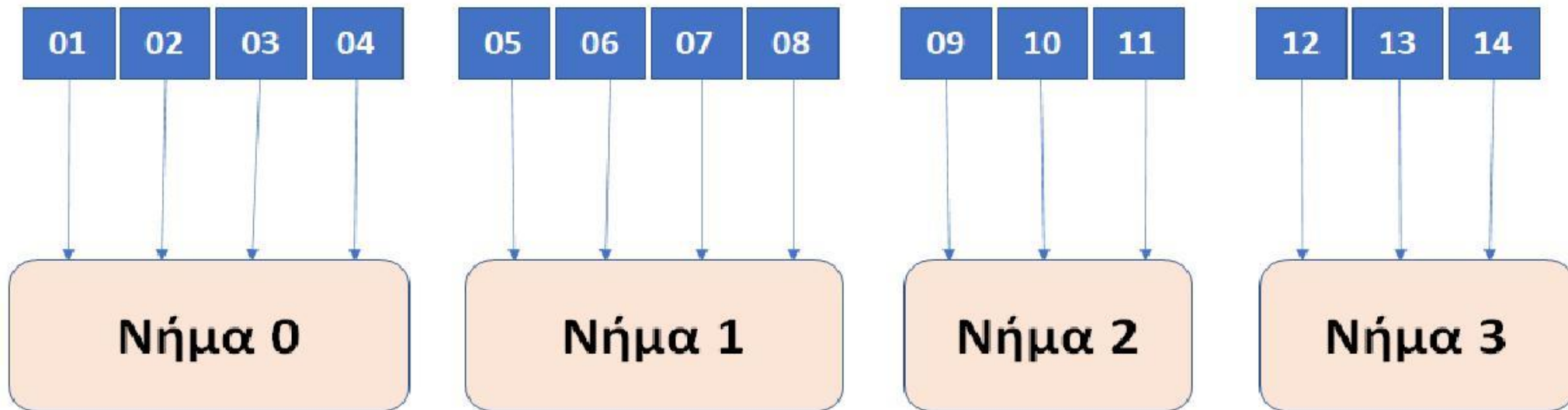


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Καταμερισμός υπολογιστικού έργου μέσω παραλληλοποίησης βρόχων

Η **ανάθεση** των **επαναλήψεων του βρόχου** στα νήματα της εφαρμογής για τον **κώδικα** της προηγούμενης διαφάνειας



Εάν **δεν χρησιμοποιηθεί** η οδηγία **#pragma omp for**, τότε ο κώδικας του βρόχου θα εκτελεστεί **ολόκληρος** από το **καθένα** από τα νήματα της εφαρμογής, το οποίο θα εκτελέσει **και τις 14 επαναλήψεις** του βρόχου, κάτι που **δεν είναι** αυτό που επιθυμούμε

Ο **μετρητής i** θα πρέπει να χαρακτηριστεί ως **ιδιωτική μεταβλητή (private)** προκειμένου ο βρόχος να εκτελεστεί με τον **σωστό τρόπο**, αφού εάν παραμείνει **κοινόχρηστη (shared)** η συμπεριφορά της εφαρμογής **είναι προβληματική**.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

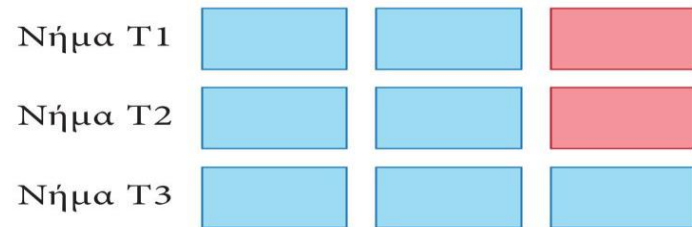
Το περιβάλλον προγραμματισμού OpenMP

Χρονοδρομολόγηση βρόχων επανάληψης

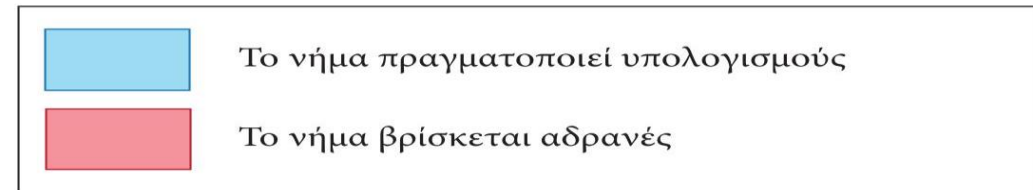
Η **απόδοση** μιας **παράλληλης εφαρμογής** που συνίσταται στην **διαμέριση των επαναλήψεων** ενός βρόχου στα διαθέσιμα **νήματα**, είναι **άρρηκτα συνδεδεμένη** με τον **τρόπο κατανομής των επαναλήψεων** στα **νήματα** που θα τις εκτελέσουν.

Εάν ορίσουμε την **απόδοση**, ως τον **λόγο** του **χρονικού διαστήματος παραγωγής έργου** από τα νήματα, προς το **συνολικό χρονικό διάστημα εκτέλεσης**, αυτή **μεγιστοποιείται** εάν κατανείμουμε τις επαναλήψεις στα νήματα έτσι ώστε το **χρονικό διάστημα** κατά το οποίο αυτά βρίσκονται σε **κατάσταση αδράνειας** να είναι το **ελάχιστο** δυνατό.

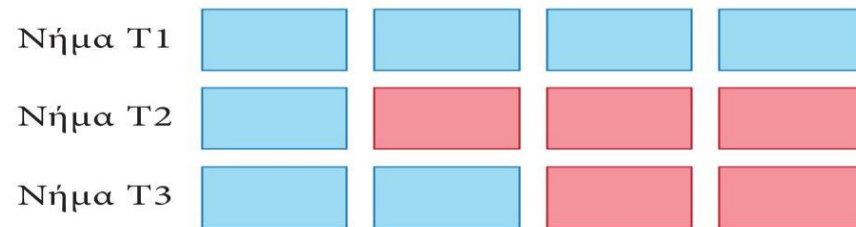
Απόδοση =
7/9 = 77.8%



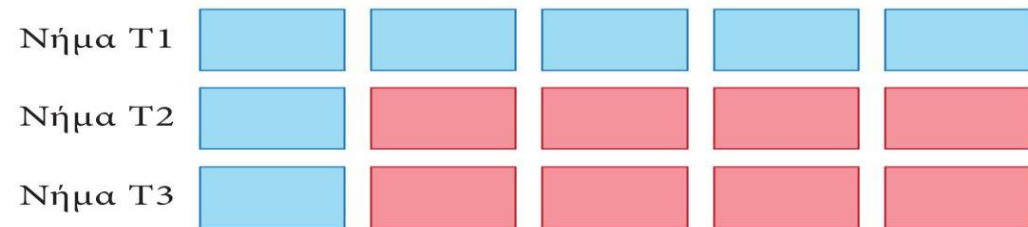
ΣΕΝΑΡΙΟ 1 (T1→2, T2→2, T3→3)



Απόδοση =
7/12 = 58.5%



ΣΕΝΑΡΙΟ 2 (T1→4, T2→1, T3→2)



ΣΕΝΑΡΙΟ 3 (T1→5, T2→1, T3→1)

Απόδοση =
7/15 = 46.7%



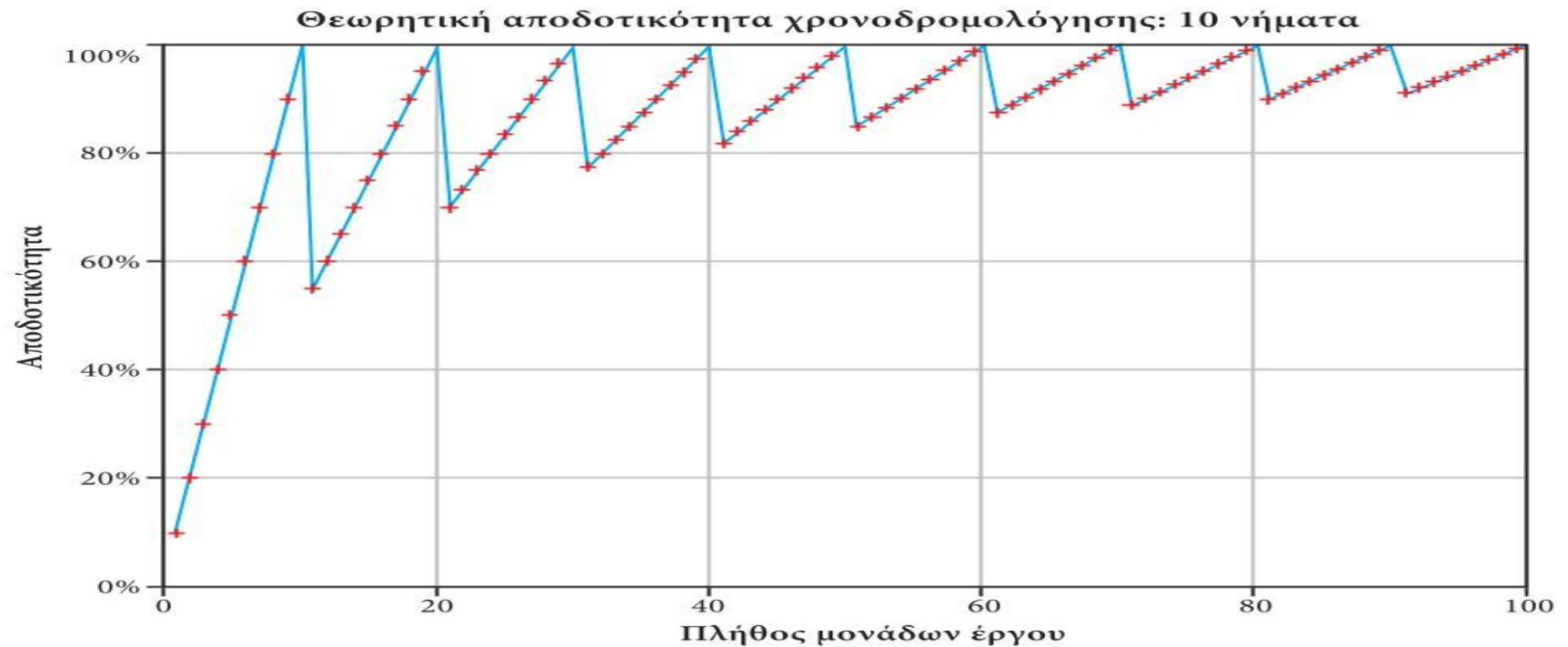
Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Χρονοδρομολόγηση βρόχων επανάληψης

Η θεωρητική απόδοση της χρονοδρομολόγησης βρόχων ως συνάρτηση του πλήθους των επαναλήψεων n που εκχωρούνται σε κάθε νήμα και ως συνάρτηση του πλήθους των νημάτων N για την τιμή $N = 10$. Παρατηρήστε πως για τιμές του n που είναι ακέραια πολλαπλάσια του 10 η τιμή της απόδοσης προσεγγίζει την τιμή 100%.

Προκειμένου να επιτύχουμε μία τιμή απόδοσης μεγαλύτερη από 0.9 (ή 90%), θα πρέπει να διαλέξουμε μία τιμή για την παράμετρο n , τέτοια ώστε να είναι $n > 10N$ ή $n/N > 10$).



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Χρονοδρομολόγηση βρόχων επανάληψης

Στο **OpenMP**, ο προσδιορισμός της βέλτιστης κατανομής επαναλήψεων στα νήματα, ονομάζεται **χρονοδρομολόγηση βρόχων**.

Η υλοποίηση των **πολιτικών χρονοδρομολόγησης** χαρακτηρίζεται από **απρόβλεπτους παράγοντες** που την καθιστούν **περίπλοκη**.

Η **ταχύτητα εκτέλεσης** των βρόχων επανάληψης, επηρεάζεται από την **χωρική συσχέτιση** που υφίσταται ανάμεσα στις **κρυφές μνήμες** που διατηρούν τα δεδομένα των νημάτων και στους **πυρήνες** που τα **εκτελούν** και μπορεί να **αυξηθεί**, εκτελώντας τα νήματα σε **πυρήνες** που βρίσκονται σε **πολύ μικρή απόσταση** από τις παραπάνω **κρυφές μνήμες**.

Ωστόσο η πληροφορία σχετικά με το **ποιοι πυρήνες εκτελούν τα διαθέσιμα νήματα**, δεν είναι **διαθέσιμη** κατά την **επιλογή της πολιτικής δρομολόγησης**, αφού **δεν είναι γνωστό** ποια δεδομένα χρησιμοποιούνται από την **κάθε επανάληψη** του βρόχου.

Επιπλέον **δεν γνωρίζουμε** το χρονικό διάστημα που απαιτείται για την **εκτέλεση** των **διάφορων εργασιών**, αφού **ενδέχεται** να υπάρξουν και **απρόβλεπτες καθυστερήσεις**, λόγω **αλληλεπίδρασης** των νημάτων και **επιβολής μηχανισμών συγχρονισμού**.

Ορίζοντας την **απόδοση E** ως

$$E = 1 - \frac{N - 1}{n} = 1 - \frac{N - 1}{N \times (n/N)}$$

και θεωρώντας ότι $N-1 \approx N$, προκύπτει ότι $n = N/(1-E)$. Για να επιτύχουμε απόδοση **τουλάχιστον ίση με E** θα πρέπει να επιλέξουμε τιμή για το **πλήθος επαναλήψεων** για το κάθε νήμα $n \geq N/(1-E)$. Για να είναι $E \rightarrow 1$ θα πρέπει να είναι $n \rightarrow \infty$.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Ο καθορισμός της **πολιτικής διαμοιρασμού των επαναλήψεων** σε έναν **βρόχο** του **OpenMP** προσδιορίζεται μέσω της οδηγίας

```
#pragma omp for schedule (mode, chunk_size)
```

Το δεύτερο όρισμα του όρου **schedule** είναι **προαιρετικό** και προσδιορίζει **το πλήθος των επαναλήψεων** που θα αναθέσουμε για εκτέλεση **στο καθένα** από τα νήματα της εφαρμογής.

Αυτή η **ομάδα επαναλήψεων** που εκχωρείται στο **κάθε νήμα** και η οποία αποτελείται από **συνεχόμενες επαναλήψεις** του βρόχου οι οποίες αντιστοιχούν σε **συνεχόμενες τιμές του μετρητή**, είναι γνωστή ως **μονάδα υπολογιστικού έργου** (chunk).

Εάν αυτό το όρισμα με **την ίδια τιμή για όλα** τα νήματα (η οποία **δεν μπορεί** να είναι **μηδέν**) **παραλειφθεί** και ο όρος **schedule** κληθεί απλά ως **schedule (mode)**, τότε **σε κάθε νήμα** ανατίθεται ένα **προεπιλεγμένο πλήθος επαναλήψεων**.

Η **αύξηση της τιμής** αυτής της παραμέτρου, οδηγεί στη **μείωση της επιβάρυνσης** που σχετίζεται με την **ανάθεση των επαναλήψεων** στις διεργασίες, αυξάνοντας όμως τον **ρυθμό ευστοχίας** που σχετίζεται με την **προσπέλαση της κρυφής μνήμης**, ενώ η **μείωση της τιμής της**, μπορεί να οδηγήσει σε **πιο ελεγχόμενη εξισορρόπηση** υπολογιστικού φόρτου.

Το όρισμα **mode** υπαγορεύει την **πολιτική ανάθεσης** και μπορεί να λάβει μία από τις τιμές **static**, **dynamic**, **guided** και **runtime**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Η πολιτική `static` αποτελεί τον πιο απλό τύπο χρονοδρομολόγησης και χαρακτηρίζεται από την μικρότερη δυνατή επιβάρυνση, αφού το κάθε νήμα μπορεί να προσδιορίσει από μόνο του το υποσύνολο των επαναλήψεων που θα πρέπει να εκτελέσει.

Η πολιτική υφίσταται σε δύο εκδοχές, με την εκδοχή `blocked` να εκχωρεί αυτόματα σε κάθε νήμα, ένα και μοναδικό σύνολο επαναλήψεων μεγάλου μεγέθους και την εκδοχή `block cyclic` να εκχωρεί σε κάθε νήμα ένα πλήθος επαναλήψεων, η τιμή του οποίου ορίζεται από το όρισμα `chunk_size`. Οι επαναλήψεις ανατίθενται πριν από την έναρξη της εκτέλεσης του βρόχου.

Στην περίπτωση της εκδοχή `blocked`, σε κάθε νήμα εκχωρείται ένα πλήθος συνεχόμενων επαναλήψεων ίσο με $\text{ceil}(M/N)$ όπου M είναι το πλήθος των επαναλήψεων και N το πλήθος των νημάτων.

Το βασικό πλεονέκτημα αυτής της πολιτικής, είναι πως οι επαναλήψεις του βρόχου που είναι γειτονικές μεταξύ τους στον χώρο των επαναλήψεων, είναι πάρα πολύ πιθανό να εκτελεστούν από το ίδιο νήμα, γεγονός που οδηγεί στην αύξηση της απόδοσης, αφού τα εμπλεκόμενα δεδομένα είναι αποθηκευμένα στην ίδια κρυφή μνήμη.

Στην εκδοχή `block cyclic`, ορίζουμε μέσω του `chunk_size` ένα πλήθος επαναλήψεων προς ανάθεση στο κάθε νήμα, με αυτή την ανάθεση να γίνεται εκ περιτροπής, δηλαδή εναλλάξ και με κυκλικό τρόπο. Χαρακτηρίζεται από μικρότερο βαθμό τοπικότητας σε σχέση με την εκδοχή `blocked` ενώ για τιμή `chunk_size = 1`, η κάθε επανάληψη θα εκτελείται και από ένα διαφορετικό νήμα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

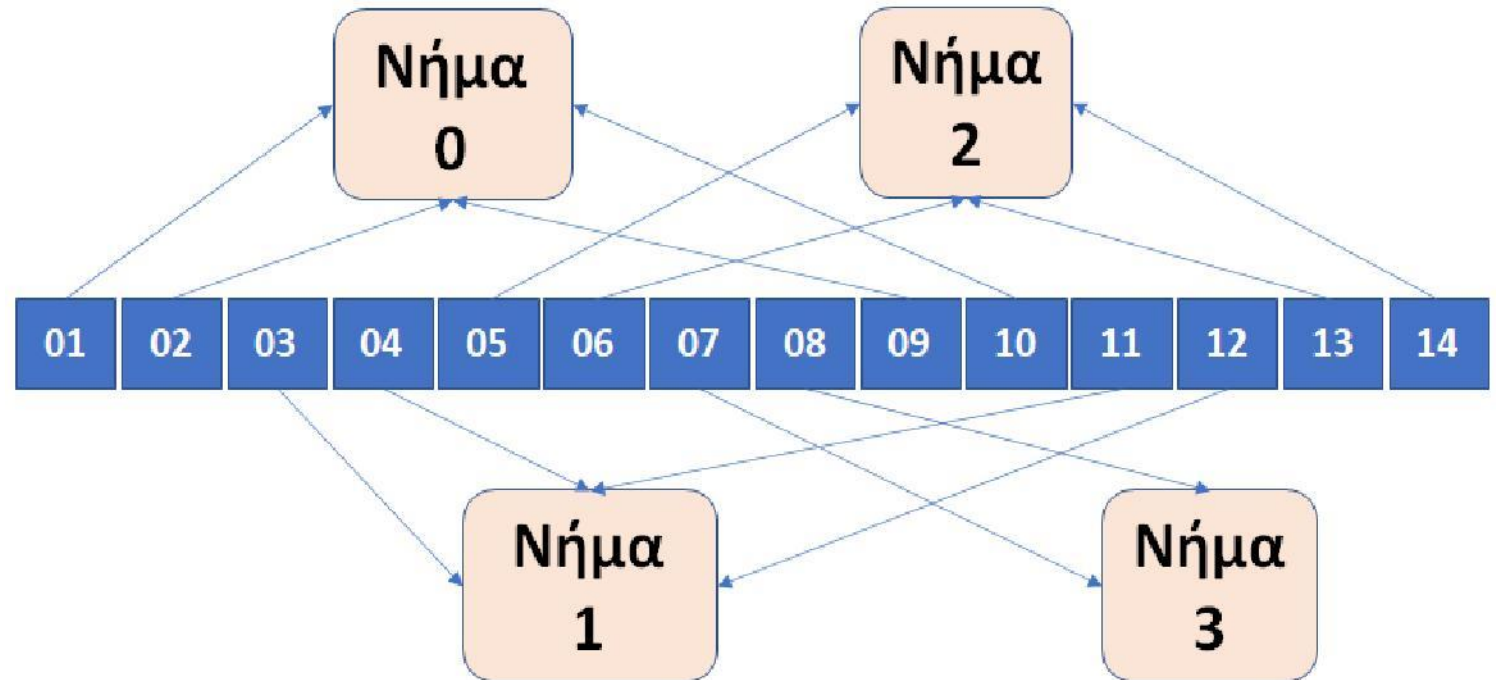
```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv) {
    int rank, size, i;
    omp_set_num_threads(4);

    #pragma omp parallel private(rank)
    {
        rank = omp_get_thread_num();
        size = omp_get_num_threads();
        #pragma omp for schedule (static,2)
        for (i=1; i<=14; i++)
            printf("Thread %02d of %d ==> Iteration No %02d\n",
                rank, size, i);
    }

    return 0; }
```

```
amarg@amarg-vbox:~$ gcc -o omp_for2 -fopenmp omp_for2.c
amarg@amarg-vbox:~$ ./omp_for2
Thread 03 of 4 ==> Iteration No 07
Thread 03 of 4 ==> Iteration No 08
Thread 01 of 4 ==> Iteration No 03
Thread 01 of 4 ==> Iteration No 04
Thread 01 of 4 ==> Iteration No 11
Thread 01 of 4 ==> Iteration No 12
Thread 02 of 4 ==> Iteration No 05
Thread 02 of 4 ==> Iteration No 06
Thread 02 of 4 ==> Iteration No 13
Thread 02 of 4 ==> Iteration No 14
Thread 00 of 4 ==> Iteration No 01
Thread 00 of 4 ==> Iteration No 02
Thread 00 of 4 ==> Iteration No 09
Thread 00 of 4 ==> Iteration No 10
amarg@amarg-vbox:~$
```



Ο διαμοιρασμός των επαναλήψεων του βρόχου στα νήματα για τον επισυναπτόμενο κώδικα

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Στην πολιτική **dynamic** λαμβάνει χώρα καταγραφή των **κατάλληλων πληροφοριών** σχετικά με την **τρέχουσα κατάσταση** του βρόχου και **διαμοιρασμός** τους σε όλα τα **εμπλεκόμενα νήματα**, προκειμένου αυτά, με βάση αυτές τις **πληροφορίες** να λάβουν τις **κατάλληλες αποφάσεις χρονοδρομολόγησης** με στόχο την **αποκατάσταση** μιας **ισοκατανομής** υπολογιστικού φόρτου.

Το **τίμημα** που **πληρώνουμε** για αυτή την **καλύτερη απόδοση**, είναι μία **πρόσθετη επιβάρυνση** λόγω της διαδικασίας **καταγραφής** και **διαμοιρασμού πληροφοριών** η οποία **δεν εμφανίζεται** στην πολιτική **static**, που ωστόσο, ενδέχεται να οδηγήσει σε συνθήκες **ανισοκατανομής υπολογιστικού φόρτου**.

Στην **πολιτική dynamic** δεν τηρείται **προκαθορισμένη σειρά** όσον αφορά στο **ποιες επαναλήψεις** θα εκχωρηθούν σε ποια νήματα, αλλά αντίθετα, η **εκχώρηση των ομάδων επαναλήψεων** στα νήματα της εφαρμογής, γίνεται με **δυναμικό τρόπο** και ανάλογα με την **διαθεσιμότητά** τους σύμφωνα με την πολιτική **FCFS (First Come First Served)**.

Εάν κατά την εκτέλεση, κάποια νήματα **ολοκληρώσουν** την εργασία τους, η **επόμενη προς ανάθεση** ομάδα επαναλήψεων ανατίθεται **στο πρώτο από αυτά τα νήματα**, με την διαδικασία να **συνεχίζεται** μέχρι να ολοκληρωθούν **όλες** οι επαναλήψεις.

Αυτός ο τρόπος διαμοιρασμού είναι **ιδιαίτερα χρήσιμος** όταν οι επαναλήψεις που διαμοιράζονται στα νήματα **δεν είναι** γνωστό εάν απαιτούν **τον ίδιο χρόνο εκτέλεσης ή όχι** ή εάν είναι γνωστό πως κάτι τέτοιο **δεν** συμβαίνει.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Η πολιτική **dynamic** επιτρέπει την **εκχώρηση επαναλήψεων** σύμφωνα με το χαρακτηριστικό της **μονοτονικότητας**, που απαιτεί την **κίνηση** του μετρητή του βρόχου **μόνο προς την μία κατεύθυνση**.

Δηλαδή, το **κάθε νήμα** εκτελεί **μόνο** επαναλήψεις, για τις οποίες ο **μετρητής του βρόχου**, έχει τιμή **μεγαλύτερη από όλες τις τιμές** που αντιστοιχούν σε επαναλήψεις οι οποίες **έχουν ήδη** πραγματοποιηθεί: η κίνηση γίνεται **πάντοτε προς την μία κατεύθυνση** (συνήθως **από τις μικρές προς τις μεγάλες** τιμές), ενώ η κίνηση προς την **αντίθετη** κατεύθυνση, **δεν είναι** επιτρεπτή.

Από την Έκδοση 5.0 του OpenMP και μετά, η **προεπιλεγμένη συμπεριφορά** είναι ο **μη μονοτονικός τρόπος**, ο οποίος επιτρέπει την κίνηση **και προς τα πίσω**, καθώς και την **εκ των υστέρων διεκδίκηση** για εκτέλεση, **επαναλήψεων**, που **έχουν προσπεραστεί** υπό την έννοια, πως ο μετρητής του βρόχου έχει τιμές **μικρότερες από αυτές** που χρησιμοποιούνται από το νήμα.

Στον **μη μονοτονικό** τρόπο λειτουργίας, όταν ένα νήμα **ολοκληρώσει** την εκτέλεση των επαναλήψεων που του έχουν ανατεθεί, αντί να περιέλθει σε **κατάσταση αδράνειας**, εκκινεί μία διαδικασία **αναζήτησης επαναλήψεων** οι οποίες έχουν εκχωρηθεί σε **κάποιο άλλο** νήμα, αλλά ωστόσο, **δεν έχουν εκτελεστεί** μέχρι αυτό το σημείο.

Εάν διαπιστωθεί **η ύπαρξη τέτοιων επαναλήψεων**, αυτές **αποσπώνται** από το νήμα στο οποίο αυτές είχαν εκχωρηθεί αρχικά, προκειμένου να **εκτελεστούν** από το νήμα που τις απέσπασε και το οποίο διαπράττει μία **κλοπή επαναλήψεων**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Στην πολιτική guided λαμβάνει χώρα ανάθεση σε ένα νήμα ενός **μεγάλου πλήθους επαναλήψεων** για εκτέλεση όταν υπάρχει διαθέσιμος **μεγάλος όγκος υπολογιστικού έργου** και ενός **μικρού πλήθους επαναλήψεων**, όταν το έργο αρχίσει να **εξαντλείται**.

Αυτή η πολιτική προσπαθεί να **αντισταθμίσει** την **επιβάρυνση** που σχετίζεται με την **πολιτική dynamic**, εκχωρώντας σε κάθε νήμα **διαφορετικό κάθε φορά πλήθος επαναλήψεων**, η τιμή του οποίου **μειώνεται** με την πάροδο του χρόνου.

Ειδικότερα, **κάθε φορά** που ένα νήμα ζητά **πρόσθετες επαναλήψεις**, εκχωρείται σε αυτό ένα **πλήθος επαναλήψεων** που είναι **ανάλογο** του πλήθους των επαναλήψεων που **δεν έχουν εκχωρηθεί** μέχρι αυτό το σημείο, διαιρεμένο με **το πλήθος των διαθέσιμων νημάτων**, με την τιμή αυτή να **μειώνεται προοδευτικά** μέχρι την **ελάχιστη τιμή** που είναι η τιμή **1**.

Η τιμή **group_size** του πλήθους των επαναλήψεων που εκχωρείται σε **κάθε νήμα** από την πολιτική **guided**, ορίζεται ως

$$\text{group_size} = \min \left[\text{remaining_iterations}, \max \left(\text{chunk_size}, \left\lceil \frac{\text{remaining_iterations}}{\text{number_of_threads}} \right\rceil \right) \right]$$

με την τιμή αυτή, όπως αναφέραμε, να **συρρικνώνεται προοδευτικά** με την πάροδο του χρόνου, μέχρι τελικά να προσεγγίσει την τιμή **chunk_size** που είναι η **ελάχιστη τιμή** που μπορεί να εκχωρηθεί σε αυτό το όρισμα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Στο OpenMP, η **πολιτική χρονοδρομολόγησης** ενός βρόχου, εκτός από την χρήση του όρου **schedule** ορίζεται και ως εξής:

1. Με την μεταβλητή περιβάλλοντος **OMP_SCHEDULE** η οποία σχετίζεται με την μεταβλητή ICV **run-sched-var** και αρχικοποιείται είτε μέσω της **export** σε φλοιούς της οικογένειας του **bash** είτε μέσω της **setenv** σε φλοιούς της οικογένειας του **csh**.

Για παράδειγμα, στον φλοιό **bash**, η δήλωση **export OMP_SCHEDULE "static, 1"** ορίζει ως πολιτική χρονοδρομολόγησης την πολιτική **static** με τιμή της παραμέτρου **chunk_size** ίση με **1**, ενώ στον φλοιό **csh** η δήλωση **setenv OMP_SCHEDULE "dynamic,5"**, ορίζει ως **πολιτική χρονοδρομολόγησης** την πολιτική **dynamic**, με τιμή της παραμέτρου **chunk_size** ίση με **5**.

2. Με τη συνάρτηση **omp_set_schedule** της **βιβλιοθήκης του OpenMP** το πρωτότυπο της οποίας έχει τη μορφή

```
void omp_set_schedule (omp_sched_t kind, int chunk_size);
```

με το όρισμα **kind** που είναι τύπου **omp_sched_t** να ορίζεται μέσω της δήλωσης **enum** ως μία λίστα της μορφής

```
typedef enum omp_sched_t {  
    omp_sched_static = 1,  
    omp_sched_dynamic = 2,  
    omp_sched_guided = 3,  
    omp_sched_auto = 4 } omp_sched_t;
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πολιτικές χρονοδρομολόγησης επαναληπτικών βρόχων

Η μεταβολή της **πολιτικής χρονοδρομολόγησης** που πραγματοποιείται συχνά στα πλαίσια **πειραματισμών**, απαιτεί την **πρόσβαση** στον πηγαίο κώδικα, ο οποίος μετά από την **κατάλληλη μεταβολή** θα πρέπει να **μεταγλωττιστεί** εκ νέου.

Εάν **δεν διαθέτουμε** τον **πηγαίο κώδικα** της εφαρμογής παρά μόνο το **εκτελέσιμο** αρχείο της, η **αλλαγή** της πολιτικής είναι δυνατό να πραγματοποιηθεί με τον ακόλουθο τρόπο:

Ορίζουμε την **νέα** πολιτική **εκτός της εφαρμογής** αρχικοποιώντας την μεταβλητή περιβάλλοντος **OMP_SCHEDULE** ενώ στον **πηγαίο κώδικα** χρησιμοποιούμε την οδηγία **schedule** με όρισμα τη συμβολοσειρά **runtime**, δηλαδή γράφοντας

```
#pragma omp parallel for schedule (runtime)
```

Στην περίπτωση αυτή, κατά την **εκτέλεση** της εφαρμογής, το OpenMP διαβάζει την **τρέχουσα τιμή** αυτής της **μεταβλητής περιβάλλοντος** και ορίζει την πολιτική της **χρονοδρομολόγησης** ανάλογα με την τιμή αυτής της μεταβλητής

Ο όρος **nowait** χρησιμοποιείται για την **απομάκρυνση** του **μηχανισμού της χρονικής φραγής** που τοποθετείται **αυτόματα** από το OpenMP στο τέλος μιας ενότητας **#pragma omp parallel** προκειμένου όλα τα νήματα να συνεχίσουν **συγχρονισμένα** την λειτουργία τους, γεγονός που επιτρέπει στο κάθε νήμα να **συνεχίσει την εκτέλεσή του** μετά την **ολοκλήρωση** του βρόχου, **ανεξάρτητα από την πρόοδο της εκτέλεσης** των άλλων νημάτων.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση ένθετων βρόχων – ο όρος `collapse`

Στην περίπτωση της χρήσης **πολλαπλών ένθετων βρόχων** εγείρεται εντελώς εύλογα το ερώτημα **σε ποιον** από τους βρόχους της **ένθετης** δομής θα εφαρμόσουμε την **διαδικασία της παραλληλοποίησης**.

Στην απλή περίπτωση ενός **διπλού βρόχου**, η παραλληλοποίηση του **εσωτερικού βρόχου** θα δημιουργήσει το **κατάλληλο πλήθος νημάτων** για την **κάθε επανάληψη του εξωτερικού βρόχου**, με αποτέλεσμα η **επιβάρυνση** που σχετίζεται με την **διαχείριση** όλων αυτών των νημάτων, να καταστεί **μεγαλύτερη** από τον χρόνο που εξοικονομούμε **παραλληλοποιώντας** την εφαρμογή.

Αντίθετα, εάν παραλληλοποιήσουμε τον **εξωτερικό βρόχο**, αυτή η επιβάρυνση εμφανίζεται για **μία και μοναδική** φορά.

Προκειμένου η εφαρμογή **να λειτουργήσει σωστά**, θα πρέπει οι μετρητές **αμφότερων** των βρόχων να δηλωθούν ως **ιδιωτικές**

Στους **πολλαπλούς ένθετους βρόχους** βάθους $d = n$, εάν επιθυμούμε ο διαμοιρασμός των επαναλήψεων να **μην περιοριστεί μόνο στον εξωτερικό βρόχο** αλλά να **επεκταθεί** και στους **εσωτερικούς βρόχους**, μπορούμε να χρησιμοποιήσουμε τον όρο **`collapse(k)`**, όπου $k \leq n$ είναι το **πλήθος των επιπέδων ένθεσης** στα οποία επιθυμούμε να εφαρμοσθεί αυτός ο **διαμοιρασμός**.

Η δήλωση **`collapse`** μετατρέπει έναν **βρόχο πολλαπλών επιπέδων** σε έναν **απλό βρόχο** τον οποίο στη συνέχεια **παραλληλοποιεί διαμοιράζοντας** τις επαναλήψεις του στα νήματα σύμφωνα με την οριζόμενη **πολιτική διαμοιρασμού**. Η διάταξη των επαναλήψεων υπαγορεύεται από την **φυσική διάταξη** που σχετίζεται με την **ακολουθιακή εκτέλεση** του ένθετου βρόχου.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση ένθετων βρόχων – ο όρος collapse

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv) {
    int rank, size, i;
    omp_set_num_threads(4);

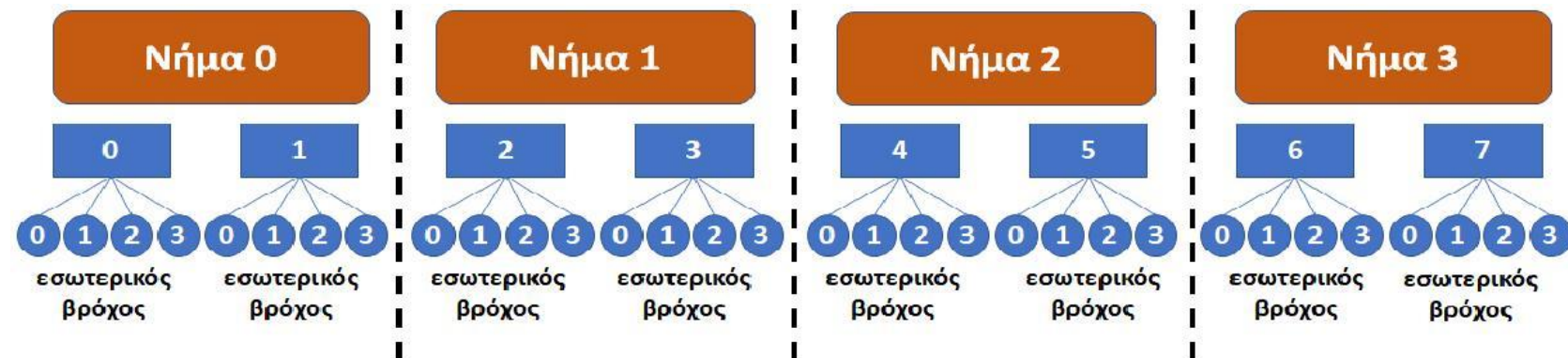
    #pragma omp parallel private(rank)
    {
        rank = omp_get_thread_num();
        size = omp_get_num_threads();
        #pragma omp for schedule (static,2)
        for (int i=0; i<8; i++) {
            for (int j=0; j<4; j++) {
                printf("Thread %d ==> i = %d, j = %d\n",
                    rank, i, j);
            }
        }
    }

    return 0; }
```

Thread 3 ==> i = 6, j = 0	Thread 2 ==> i = 4, j = 0
Thread 3 ==> i = 6, j = 1	Thread 2 ==> i = 4, j = 1
Thread 3 ==> i = 6, j = 2	Thread 2 ==> i = 4, j = 2
Thread 3 ==> i = 6, j = 3	Thread 2 ==> i = 4, j = 3
Thread 3 ==> i = 7, j = 0	Thread 2 ==> i = 5, j = 0
Thread 3 ==> i = 7, j = 1	Thread 2 ==> i = 5, j = 1
Thread 3 ==> i = 7, j = 2	Thread 2 ==> i = 5, j = 2
Thread 3 ==> i = 7, j = 3	Thread 2 ==> i = 5, j = 3
Thread 1 ==> i = 2, j = 0	Thread 0 ==> i = 0, j = 0
Thread 1 ==> i = 2, j = 1	Thread 0 ==> i = 0, j = 1
Thread 1 ==> i = 2, j = 2	Thread 0 ==> i = 0, j = 2
Thread 1 ==> i = 2, j = 3	Thread 0 ==> i = 0, j = 3
Thread 1 ==> i = 3, j = 0	Thread 0 ==> i = 1, j = 0
Thread 1 ==> i = 3, j = 1	Thread 0 ==> i = 1, j = 1
Thread 1 ==> i = 3, j = 2	Thread 0 ==> i = 1, j = 2
Thread 1 ==> i = 3, j = 3	Thread 0 ==> i = 1, j = 3

Διαμοιρασμός επαναλήψεων **διπλού** βρόχου για τον επισυναπτόμενο κώδικα.

Ο **διαμοιρασμός** αφορά **μόνο στις επαναλήψεις του εξωτερικού βρόχου**, ενώ ο εσωτερικός βρόχος **εκτελείται εξ ολοκλήρου** για την **κάθε επανάληψη** του εξωτερικού βρόχου.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση ένθετων βρόχων – ο όρος collapse

```
#include <stdio.h>
#include <omp.h>
```

```
int main(int argc, char **argv) {
    int rank, size, i;
    omp_set_num_threads(4);
```

```
#pragma omp parallel private(rank)
{
    rank = omp_get_thread_num();
    size = omp_get_num_threads();
```

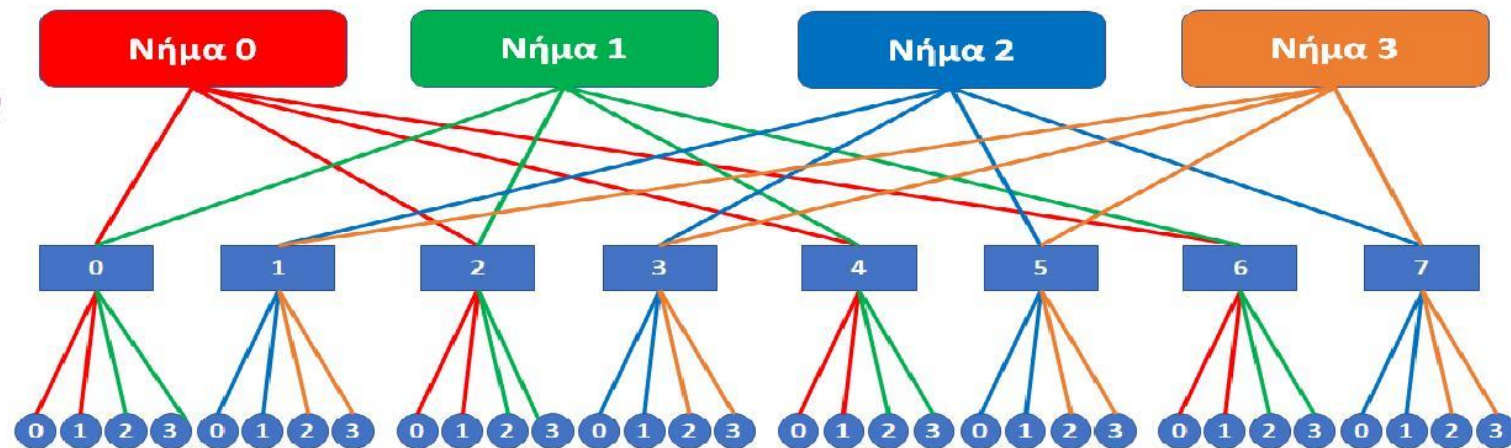
```
#pragma omp for schedule (static,2) collapse (2)
```

```
for (int i=0; i<8; i++) {
    for (int j=0; j<4; j++) {
        printf("Thread %d ==> i = %d, j = %d\n",
            rank, i, j);
    }
}
```

```
return 0; }
```

```
Thread 3 ==> i = 1, j = 2
Thread 3 ==> i = 1, j = 3
Thread 3 ==> i = 3, j = 2
Thread 3 ==> i = 3, j = 3
Thread 3 ==> i = 5, j = 2
Thread 3 ==> i = 5, j = 3
Thread 3 ==> i = 7, j = 2
Thread 3 ==> i = 7, j = 3
Thread 1 ==> i = 0, j = 2
Thread 1 ==> i = 0, j = 3
Thread 1 ==> i = 2, j = 2
Thread 1 ==> i = 2, j = 3
Thread 1 ==> i = 4, j = 2
Thread 1 ==> i = 4, j = 3
Thread 1 ==> i = 6, j = 2
Thread 1 ==> i = 6, j = 3
Thread 2 ==> i = 1, j = 0
Thread 2 ==> i = 1, j = 1
Thread 2 ==> i = 3, j = 0
Thread 2 ==> i = 3, j = 1
Thread 2 ==> i = 5, j = 0
Thread 2 ==> i = 5, j = 1
Thread 2 ==> i = 7, j = 0
Thread 2 ==> i = 7, j = 1
Thread 0 ==> i = 0, j = 0
Thread 0 ==> i = 0, j = 1
Thread 0 ==> i = 2, j = 0
Thread 0 ==> i = 2, j = 1
Thread 0 ==> i = 4, j = 0
Thread 0 ==> i = 4, j = 1
Thread 0 ==> i = 6, j = 0
Thread 0 ==> i = 6, j = 1
```

Εδώ ο **διαμοιρασμός**
αφορά στις επαναλήψεις
του **εξωτερικού** αλλά και
του **εσωτερικού** βρόχου



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση ένθετων βρόχων – ο όρος `collapse`

Ο όρος `schedule` δεν μπορεί να χρησιμοποιηθεί με **μη ορθογώνιους βρόχους** (π.χ. **τριγωνικούς** ή **τραπεζιοειδείς** βρόχους) και κατά συνέπεια, αν και ο όρος `collapse` χρησιμοποιείται όπως και πριν, ωστόσο, **η μεταγλώττιση θα αποτύχει**.

```
#pragma omp parallel for collapse(2) num_threads(2)
for (int i=0; i<M; i++)
    for (int j=i; j<M; j++) {
        float temp = 0.0f;
        for (int k=0; k<N; k++)
            temp += (a[i][k]-a[i][N])*(a[j][k]-a[j][N]);
        b[i][j] = temp/(a[i][N+1]*a[j][N+1]*(N-1));
        b[j][i] = b[i][j]; }

printf ("b[0][0] = %f, b[M-1][M-1] = %f\n",
        b[0][0], b[M-1][M-1]);
return 0; }
```

```
amarg@amarg-vbox:~$ gcc nonRect.c -fopenmp -o nonRect
nonRect.c: In function 'main':
nonRect.c:14:11: error: 'schedule' clause may not appear on non-rectangular 'for'
   14 |     #pragma omp parallel for collapse(2) num_threads(2) schedule (static,2)
      |     ^~~~
amarg@amarg-vbox:~$ gcc nonRect.c -fopenmp -o nonRect
amarg@amarg-vbox:~$ ./nonRect
b[0][0] = 0.342523, b[M-1][M-1] = 0.167836
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση ένθετων βρόχων – οι όροι `order` και `ordered`

Ο όρος `ordered`, επιτρέπει τον **ορισμό εντολών** οι οποίες επιθυμούμε να εκτελεστούν **σειριακά**, με την **ακολουθιακή** εκτέλεση να υπαγορεύεται από τις **τιμές του μετρητή επανάληψης** του βρόχου. Εάν μία ομάδα νημάτων που εκτελούνται **παράλληλα**, συναντήσουν την οδηγία `ordered`, η εκτέλεση των επαναλήψεων τους θα γίνει με **σειριακό τρόπο**.

Προκειμένου να υλοποιηθεί αυτή η λειτουργία, το νήμα που εκτελεί **την πρώτη επανάληψη** εκτελεί **πρώτο** τον κώδικα του όρου `ordered` **χωρίς καμία αναμονή**, ενώ τα υπόλοιπα νήματα **αναμένουν** την **ολοκλήρωση της εκτέλεσης** αυτού του κώδικα, δηλαδή των επαναλήψεων που προηγούνται των δικών τους, σύμφωνα με τις **τιμές** του μετρητή επανάληψης.

Ο όρος καλείται ως `ordered (n)` δηλαδή με τρόπο **παρόμοιο** με αυτόν που χαρακτηρίζει την χρήση του όρου `collapse`.

Εάν οι όροι `ordered` και `collapse` χρησιμοποιούνται **αμφότεροι** σε κάποια δήλωση, με ορίσματα `n` και `m` αντίστοιχα, θα πρέπει να ικανοποιείται η συνθήκη $n \geq m$. Εάν δεν ορισθεί κάποια τιμή για το όρισμα του όρου `ordered` τότε θα χρησιμοποιηθεί **η τιμή του ορίσματος** του όρου `collapse`, με την **προεπιλεγμένη** τιμή να είναι η τιμή 1.

Ο όρος `order` επιτρέπει τον προσδιορισμό μιας **αναμενόμενης σειράς εκτέλεσης** για τις επαναλήψεις των εμπλεκόμενων βρόχων. Εάν ως όρισμα χρησιμοποιείται η συμβολοσειρά `concurrent`, οι **λογικές επαναλήψεις** των βρόχων μπορούν να εκτελεστούν με **οποιαδήποτε σειρά**, ακόμη και **ταυτόχρονα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Το OpenMP διαθέτει **εργαλεία συγχρονισμού** και **ταυτοχρονισμού**, για την υλοποίηση τεχνικών **αμοιβαίου αποκλεισμού** και **συγχρονισμού χρονικής φραγής** που επιτρέπουν τον ορισμό της **σειράς ολοκλήρωσης** του κώδικα των νημάτων.

Ο παραπάνω συγχρονισμός αποτελεί **χρονικό συγχρονισμό**, υπό την έννοια πως τα νήματα συγχρονίζονται **στον χρόνο**, ενώ ένας **δεύτερος συγχρονισμός** αφορά **στις τιμές των κοινόχρηστων μεταβλητών** που είναι αποθηκευμένες στη μνήμη και για τον λόγο αυτό θα τον χαρακτηρίσουμε ως **συγχρονισμό περιεχομένου**.

Το OpenMP υποστηρίζει ένα **μοντέλο κοινόχρηστης μνήμης**, με όλα τα νήματα να έχουν πρόσβαση **στην ίδια περιοχή μνήμης** για την **αποθήκευση** και την **ανάκτηση** δεδομένων αλλά και με το **κάθε νήμα** να έχει πρόσβαση τόσο σε μία **δική του αποκλειστική μνήμη** τύπου **threadprivate**, όσο και σε μία **ενδιάμεση προσωρινή μνήμη** (π.χ. **καταχωρητές** και **κρυφή μνήμη**)

Σε αυτή την **προσωρινή περιοχή αποθήκευσης**, το κάθε νήμα μπορεί να αποθηκεύσει **προσωρινά μεταβλητές**, έτσι ώστε κάθε φορά που τις χρειάζεται, **να τις βρίσκει εκεί** και να **μην ανατρέχει** στην κύρια μνήμη του.

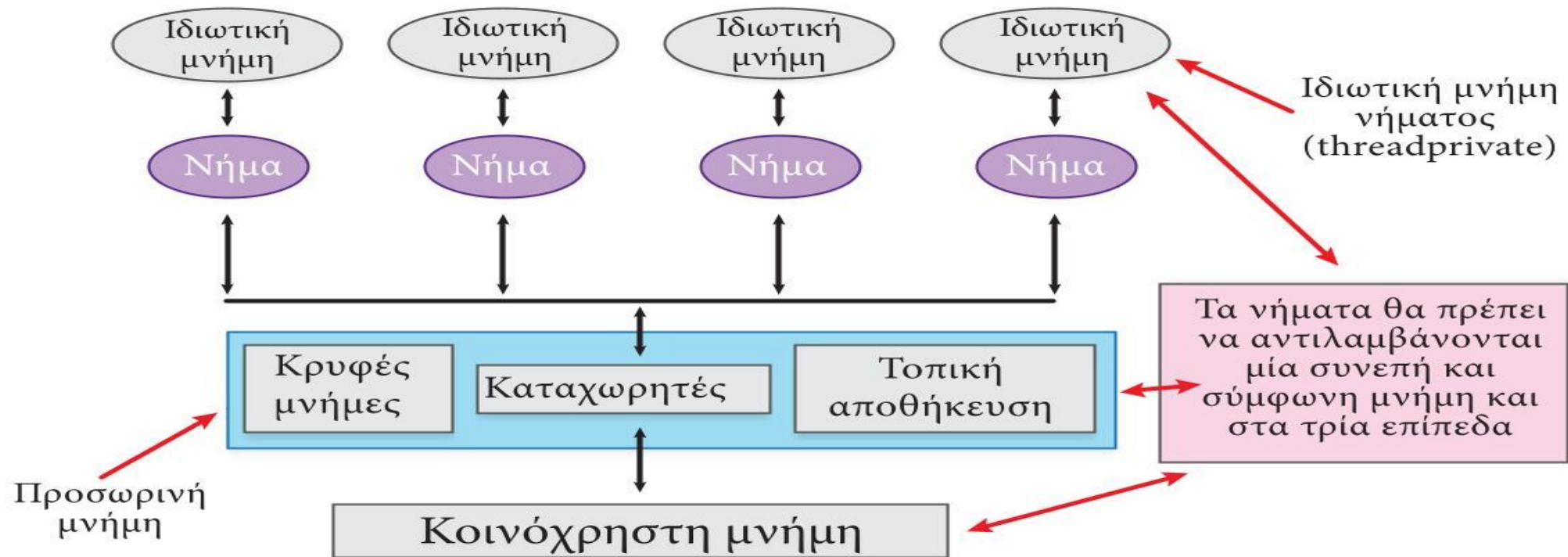


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Το **μοντέλο μνήμης** του **OpenMP** εμπλουτισμένο με το χαρακτηριστικό της **προσωρινής μνήμης**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Το **OpenMP** χρησιμοποιεί **δύο τύπους** μεταβλητών, ως προς την εμβέλεια τους, τις **ιδιωτικές** και τις **κοινόχρηστες** μεταβλητές.

Για **κάθε μεταβλητή** που χρησιμοποιείται σε ένα τμήμα **παράλληλου κώδικα**, υφίσταται μία **αντίστοιχη μεταβλητή** που βρίσκεται **εκτός** του κώδικα και περιγράφεται **από το ίδιο όνομα** με αυτήν. Υφίστανται **δύο** δυνατότητες:

- εάν η μεταβλητή είναι **κοινόχρηστη**, κάθε αναφορά σε αυτή τη μεταβλητή από εντολές **εντός του παράλληλου κώδικα**, αποτελεί ουσιαστικά αναφορά **προς την αυθεντική μεταβλητή**
- εάν η μεταβλητή είναι **ιδιωτική**, τότε λαμβάνει χώρα **δημιουργία** στη μνήμη του κάθε νήματος, ενός **αντιγράφου** της **αυθεντικής μεταβλητής** και η αναφορά γίνεται **σε αυτό το αντίγραφο**. Η τιμή αυτού του αντιγράφου, **δεν επηρεάζει** την τιμή που έχει καταχωρηθεί στην **αυθεντική μεταβλητή**.

Κάθε φορά που γίνεται **αναφορά** από ένα νήμα **εκτός του παράλληλου κώδικα** σε μία **ιδιωτική** μεταβλητή, αυτή η αναφορά αφορά στο **ιδιωτικό αντίγραφο** της εν λόγω μεταβλητής που βρίσκεται **στην μνήμη αυτού του νήματος**.

Η **διαδικασία πρόσβασης** της τιμής μιας μεταβλητής, μπορεί να σχετίζεται με **πολλαπλές διαδικασίες ανάγνωσης ή / και εγγραφής** και μιλώντας γενικά, **κανείς δεν μπορεί να μας βεβαιώσει**, πως όλες αυτές οι διαδικασίες θα εκτελεστούν **υπό καθεστώς ατομικής λειτουργίας** σε σχέση με άλλες διαδικασίες πρόσβασης της **ίδιας** μεταβλητής



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Συνοχή μνήμης: σχετίζεται με την συμπεριφορά του συστήματος της μνήμης, όταν κατά την εκτέλεση της παράλληλης εφαρμογής λαμβάνει χώρα προσπέλαση της ίδιας κοινόχρηστης διεύθυνσης από μία ομάδα νημάτων

Συνέπεια μνήμης: σχετίζεται με την χρονική σειρά με την οποία πραγματοποιούνται οι διαδικασίες πρόσβασης σε διαφορετικές θέσεις μνήμης, έτσι όπως αυτές γίνονται αντιληπτές από τα νήματα της εφαρμογής και με τέτοιο τρόπο, ώστε να διασφαλίζεται πως κάθε φορά τα νήματα θα έχουν στη διάθεσή τους τα σωστά δεδομένα.

Το OpenMP ορίζει κανόνες συνέπειας που υπαγορεύουν την σωστή σειρά πραγματοποίησης των διαδικασιών προσπέλασης στη μνήμη έτσι ώστε να μην ανακύψουν φαινόμενα ασυνέπειας δεδομένων.

Εάν δεν χρησιμοποιηθούν αυτοί οι κανόνες, οι διαδικασίες χαρακτηρίζονται ως μη διατεταγμένες και ανακύπτει ο κίνδυνος εμφάνισης συνθηκών ανταγωνισμού, εάν η προσπέλαση αποτελεί διαδικασία εγγραφής.

Στο OpenMP, η συνθήκη της συνέπειας δεν είναι υποχρεωτικό να ικανοποιείται ανάμεσα στην προσωρινή και στην κύρια μνήμη του νήματος και ως εκ τούτου, οι τιμές των αντιγράφων των μεταβλητών που βρίσκονται σε αυτές τις δύο μνήμες, δεν είναι υποχρεωτικό να ταυτίζονται σε κάθε χρονική στιγμή.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Η διαδικασία **συγχρονισμού** των **περιεχομένων μνήμης** στο OpenMP, εμφανίζεται σε **τρεις διαφορετικές** εκδοχές:

1. Ισχυρός συγχρονισμός: Αφορά στην λειτουργία **ενός και μοναδικού νήματος** και συνίσταται στον **συγχρονισμό** των τιμών μιας **ομάδας δεδομένων**, ανάμεσα στην **προσωρινή** και στην **κύρια μνήμη** του νήματος, διαδικασία που **αποκαθιστά** την **συνέπεια** ανάμεσα στα περιεχόμενα της **προσωρινής** και της **κύριας μνήμης** του νήματος.

Εάν κάποιο νήμα έχει **τροποποιήσει** την τιμή **κοινόχρηστης μεταβλητής** που διατηρείται στην **προσωρινή μνήμη** μετά από την **τελευταία αποθήκευσή της** στην κύρια μνήμη, εάν λάβει χώρα άλλη διαδικασία **ισχυρού συγχρονισμού**, στην οποία εμπλέκεται αυτή η μεταβλητή, αυτή **δεν ολοκληρώνεται**, παρά μόνο όταν η τιμή αυτής της μεταβλητής **έχει εγγραφεί** στην **κύρια μνήμη**.

2. Συγχρονισμός αποδέσμευσης που συνίσταται στον **συγχρονισμό** των τιμών των **κοινόχρηστων μεταβλητών** ανάμεσα στην **προσωρινή** και στην **κύρια μνήμη** του νήματος, προκειμένου αυτές να καταστούν **διαθέσιμες** για **ανάγνωση**.

3. Συγχρονισμός ανάκτησης που συνίσταται στην **ανάγνωση** από ένα νήμα, των **νέων ενημερωμένων τιμών** των **απαιτούμενων κοινόχρηστων μεταβλητών**, από την **κύρια μνήμη** ενός άλλου νήματος, το οποίο τις έχει ήδη **καταστήσει διαθέσιμες** έχοντας πραγματοποιήσει σε κάποια **προγενέστερη χρονική στιγμή**, μία διαδικασία **συγχρονισμού αποδέσμευσης**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP

Σημειώστε, πως ενώ στην διαδικασία του **ισχυρού συγχρονισμού** εμπλέκεται **ένα και μοναδικό νήμα**, στους άλλους δύο τύπους συγχρονισμού, εμπλέκονται **δύο νήματα** τα οποία προσπαθούν να **συγχρονιστούν** μεταξύ τους, έτσι ώστε να **αποκατασταθεί** η **συνέπεια** ανάμεσα στα **περιεχόμενα** της μνήμης που προσπελαίνουν.

Θεωρώντας δύο νήματα **A** και **B**, αυτό γίνεται σε **4 βήματα**, εκ των οποίων στα **2 πρώτα** το **νήμα A ενημερώνει** την μεταβλητή με την **νέα** τιμή και πραγματοποιεί **συγχρονισμό αποδέσμευσης** καθιστώντας τις τιμές των μεταβλητών ενδιαφέροντος, **διαθέσιμες προς χρήση**, ενώ **στα 2 τελευταία**, το νήμα **B** πραγματοποιεί **συγχρονισμό ανάκτησης** και στη συνέχεια **διαβάζει** τη **νέα τιμή**.

Μία διαδικασία **συγχρονισμού** τύπου **αποδέσμευσης**, πραγματοποιεί μία **χρονική κατάταξη** των **διαδικασιών προσπέλασης μνήμης**, με τέτοιο τρόπο, ώστε οι διαδικασίες που **προηγούνται** του συγχρονισμού, να πραγματοποιηθούν χρονικά **πριν** από τις διαδικασίες **προσπέλασης μνήμης** που πραγματοποιούνται από **κάποιο άλλο** νήμα, το οποίο **συγχρονίζεται** με το τρέχον νήμα.

Μία διαδικασία **συγχρονισμού** τύπου **ανάκτησης**, πραγματοποιεί μία **χρονική κατάταξη** των **διαδικασιών προσπέλασης μνήμης**, με τέτοιο τρόπο, ώστε οι διαδικασίες που **έπονται** του συγχρονισμού, να πραγματοποιηθούν χρονικά **μετά** από τις διαδικασίες **προσπέλασης μνήμης** που πραγματοποιούνται από **κάποιο άλλο** νήμα, το οποίο **συγχρονίζεται** με το τρέχον νήμα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP – η οδηγία flush

Η διασφάλιση της **συνέπειας** ανάμεσα στις τιμές των **κοινοχρήστων** μεταβλητών που χρησιμοποιεί το νήμα και σε αυτές που βρίσκονται **αποθηκευμένες στην κύρια μνήμη**, πραγματοποιείται με την οδηγία **flush** η οποία αποτελεί μία οδηγία **πολύ χαμηλού επιπέδου** και ως τέτοια, θα πρέπει να χρησιμοποιείται με **πολύ μεγάλη προσοχή**. Η εντολή καλείται ως

```
# pragma omp flush
```

διασφαλίζοντας τη **συνέπεια** για **όλα τα δεδομένα** που είναι **ορατά** και **προσπελάσιμα** από το **τρέχον νήμα** ή ως

```
# pragma omp flush(x,y)
```

διασφαλίζοντας τη **συνέπεια** μόνο για τις τιμές των μεταβλητών **x** και **y**

Το **OpenMP** πραγματοποιεί **και από μόνο του** αυτή την διαδικασία (**implicit flush**) σε περιπτώσεις όπως

- κατά την **είσοδο** και **έξοδο** σε τμήμα κώδικα που περιέχει κάποια από τις οδηγίες **parallel**, **critical** και **ordered**
- κατά την **εκτέλεση** μιας οδηγίας **barrier**
- κατά την **εκτέλεση κώδικα** που περιλαμβάνει **καταμερισμό υπολογιστικού φόρτου**
- κατά την **είσοδο** και την **έξοδο** από μία διαδικασία που έχει χαρακτηριστεί ως **atomic**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP – η οδηγία `critical`

Ένα νήμα βρίσκεται **στο κρίσιμο τμήμα του** όταν **τροποποιεί** την τιμή κάποιας **κοινόχρηστης** μεταβλητής.

Για να δηλώσουμε πως ένα **τμήμα κώδικα** αποτελεί **κρίσιμο τμήμα** χρησιμοποιούμε πριν από αυτό την οδηγία

```
#pragma omp critical [ ( identifier ) ]
```

με το όρισμα **identifier** να επιτρέπει την **ομαδοποίηση** των **κρίσιμων τμημάτων** μιας διεργασίας.

Εάν χρησιμοποιήσουμε **το ίδιο όρισμα** σε **πολλές** οδηγίες τύπου **critical**, αυτό προσδιορίζει **την ίδια μεταβλητή αμοιβαίου αποκλεισμού** με αποτέλεσμα, η συνθήκη του **αμοιβαίου αποκλεισμού** να εφαρμόζεται **μόνο** στην ομάδα των νημάτων που εκτελούν αυτές τις οδηγίες (αν και αυτό **δεν υποκαθιστά** τη χρήση των **μεταβλητών αμοιβαίου αποκλεισμού**).

Χρησιμοποιώντας επομένως **διαφορετικά ονόματα** στις οδηγίες **critical**, μπορούμε να ενεργοποιήσουμε **περισσότερες από μία κρυφές μεταβλητές αμοιβαίου αποκλεισμού** και με τον τρόπο αυτό, να **μειώσουμε τον βαθμό του ανταγωνισμού** ανάμεσα στα νήματα που εκτελούν τα διάφορα τμήματα του παράλληλου κώδικα.

Εάν **δεν χρησιμοποιηθεί** αυτό το όρισμα, τότε τα **κρίσιμα τμήματα** θεωρούνται πως περιγράφονται όλα από **το ίδιο** όνομα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

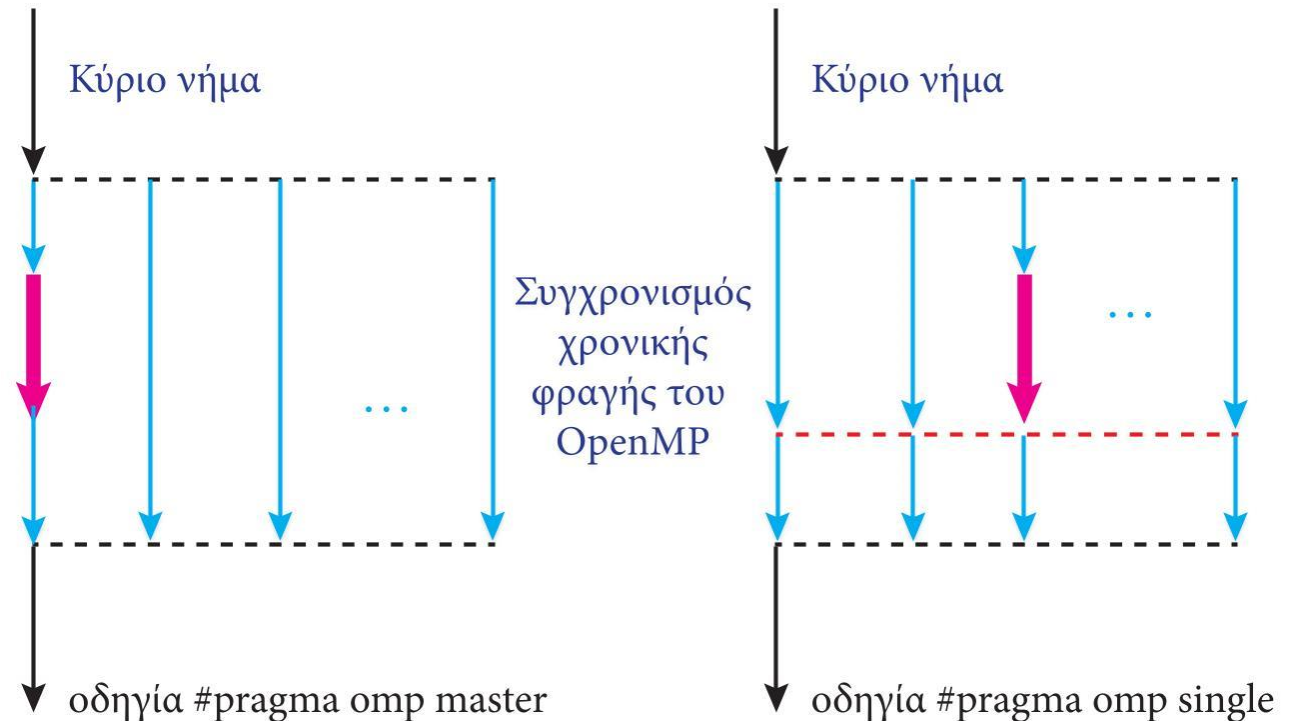
Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP – οι οδηγίες master και single

Η οδηγία **master** που καλείται με τη μορφή **#pragma omp master** επιτρέπει την **εκτέλεση** ενός **τμήματος παράλληλου κώδικα μόνο από το κύριο νήμα** μίας ομάδας νημάτων, για την πραγματοποίηση **πάσης φύσεως διαδικασιών**.

Η οδηγία **single** που καλείται με τη μορφή **#pragma omp single** (αν και γενικά, συνοδεύεται από **όρους**), υποδεικνύει την **ακολουθιακή εκτέλεση** του **σχετιζόμενου τμήματος κώδικα** από **ένα και μοναδικό νήμα** της ομάδας και ειδικότερα από το **πρώτο διαθέσιμο νήμα** κατά την χρονική στιγμή κλήσης της.

Ο κώδικας της **single** εκτελείται από **ένα και μοναδικό νήμα**, ενώ ο κώδικας της **critical** εκτελείται **από ένα και μοναδικό νήμα κάθε φορά**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Συγχρονισμός και ταυτοχρονισμός στο OpenMP – οι οδηγίες `barrier` και `atomic`

Η οδηγία `barrier` η οποία συντάσσεται ως `#pragma omp barrier` και **δεν** συνοδεύεται από κανέναν άλλο όρο, ορίζει μέσα στον κώδικα μία **ορισμένη** από τον χρήστη **χρονική φραγή**, γεγονός που προκαλεί τον **συγχρονισμό** των **νημάτων της τρέχουσας ομάδας**, όταν αυτά **προσεγγίσουν** την γραμμή αυτής της οδηγίας.

Σε αυτή την λειτουργία είτε συμμετέχουν **όλα τα νήματα της παράλληλης περιοχής**, είτε **δεν** συμμετέχει **κανένα** νήμα.

Χαρακτηριστικές περιπτώσεις σημείων στις οποίες μπορούμε να χρησιμοποιήσουμε την οδηγία `barrier` είναι

- **πριν από την είσοδο** σε μία οδηγία **καταμερισμού εργασιών**
- σε περιπτώσεις κατά τις οποίες επιθυμούμε να **αποκαταστήσουμε** καθεστώς **συγχρονισμού**
- σε περιπτώσεις **πραγματοποίησης επαναλήψεων** οι οποίες απαιτείται να είναι **συγχρονισμένες**
- σε περιπτώσεις υλοποίησης διαδικασιών παραλληλισμού **χαμηλού επιπέδου**

Η οδηγία `atomic` επιτρέπει την εκτέλεση ορισμένων απλών **διαδικασιών προσπέλασης μνήμης** υπό συνθήκες **ατομικής λειτουργίας**, οι οποίες είτε **θα εκτελεστούν στο σύνολό τους**, είτε **δεν θα εκτελεστούν καθόλου**.

Γνωστοί όροι που χρησιμοποιούνται με την οδηγία `atomic` είναι οι όροι **read**, **write** και **update**, που επιτρέπουν την πραγματοποίηση **ατομικών διαδικασιών ανάγνωσης, εγγραφής και ενημέρωσης** μιας θέσης μνήμης



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η τεχνική της μείωσης ή υποβιβασμού δεδομένων

Αποτελεί μορφή **συλλογικής επικοινωνίας** που συνίσταται στον **συνδυασμό** ή **συγχώνευση** μιας **ομάδας τιμών** σε μία **απλή τιμή**.

Με τον τρόπο αυτό, υφίσταται η δυνατότητα **παράλληλης εκτέλεσης επαναλαμβανόμενων διαδικασιών** ορίζοντας το τμήμα του κώδικα που θα εκτελεστεί **παράλληλα** καθώς και το **ποιες οντότητες εκτέλεσης** (νήματα ή διεργασίες) συμμετέχουν σε αυτή.

Στο OpenMP, οι τιμές που **συνδυάζονται μεταξύ τους** και **συγχωνεύονται σε μία τιμή**, είναι αποθηκευμένες στα **ιδιωτικά αντίγραφα** μιας **κοινόχρηστης** ως προς τη φύση της μεταβλητής, τα οποία **διατηρούνται στην μνήμη των νημάτων της παράλληλης εφαρμογής** και περιέχουν το αποτέλεσμα κάποιου υπολογισμού.

Η διαδικασία στηρίζεται στην επαναληπτική εφαρμογή μιας **δυαδικής πράξης** πάνω στις τιμές μιας ομάδας δεδομένων $\{ \alpha_1, \alpha_2, \dots, \alpha_n \}$, αντικαθιστώντας σε κάθε βήμα, ένα ζεύγος τιμών (α_i, α_j) ($1 \leq i, j \leq N$) με την τιμή $\beta_k = \alpha_i \otimes \alpha_j$.

Σε **κάθε βήμα**, το πλήθος των δεδομένων μειώνεται **προοδευτικά ένα**, ενώ η διαδικασία **τερματίζεται** όταν έχει παραμείνει **μία τιμή** ίση με $\alpha_1 \otimes \alpha_2 \otimes \dots \otimes \alpha_N$ η οποία είναι γνωστή ως **μεταβλητή μείωσης δεδομένων**.

- Βήμα 1: $\beta = \alpha_1$.
- Βήμα 2: $\beta = \alpha_1 \otimes \alpha_2$.
- Βήμα 3: $\beta = \alpha_1 \otimes \alpha_2 \otimes \alpha_3$.
- Βήμα 4: $\beta = \alpha_1 \otimes \alpha_2 \otimes \alpha_3 \otimes \alpha_4$.
- Βήμα 5: $\beta = \alpha_1 \otimes \alpha_2 \otimes \alpha_3 \otimes \alpha_4 \otimes \alpha_5$.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η τεχνική της μείωσης ή υποβιβασμού δεδομένων

Οι **μεταβλητές** του OpenMP που εμπλέκονται στις **διαδικασίες υποβιβασμού δεδομένων** και ο **τρόπος χρήσης** τους

Πράξη	Αρχική τιμή	Τρόπος υλοποίησης
+	omp_priv = 0	omp_out += omp_in
- (deprecated)	omp_priv = 0	omp_out += omp_in
*	omp_priv = 1	omp_out *= omp_in
&	omp_priv = ~ 0	omp_out &= omp_in
	omp_priv = 0	omp_out = omp_in
^	omp_priv = 0	omp_out ^= omp_in
&&	omp_priv = 1	omp_out = omp_in && omp_out
	omp_priv = 0	omp_out = omp_in omp_out
max	omp_priv = <i>Ελάχιστη τιμή που υποστηρίζεται από τον τύπο δεδομένων</i>	omp_out = omp_in > omp_out ? omp_in : omp_out
min	omp_priv = <i>Μέγιστη τιμή που υποστηρίζεται από τον τύπο δεδομένων</i>	omp_out = omp_in < omp_out ? omp_in : omp_out



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η τεχνική της μείωσης ή υποβιβασμού δεδομένων

Οι μεταβλητές μείωσης ή υποβιβασμού δεδομένων που χρησιμοποιούνται σε αυτές τις διαδικασίες, ενέχουν ιδιωτικό χαρακτήρα, αφού αρχικοποιούνται δια της συγχωνεύσεως ενός συνόλου επιμέρους αποτελεσμάτων τα οποία έχουν υπολογιστεί από μία ομάδα νημάτων που συμμετέχουν στην εκτέλεση ενός παράλληλου κώδικα και βρίσκονται αποθηκευμένα σε ιδιωτικά αντίγραφα μιας μεταβλητής τύπου `firstprivate`.

Η πράξη $x = x \otimes expr$ που χρησιμοποιείται σε αυτή την διαδικασία θα πρέπει να χαρακτηρίζεται από την προσεταιριστική ιδιότητα, ενώ η μεταβλητή x δεν θα πρέπει να εμφανίζεται στην έκφραση `expr` ή οπουδήποτε αλλού.

Εάν εκτός από την προσεταιριστική ικανοποιείται και η μεταθετική ιδιότητα, τότε η υλοποίηση της πράξης με έναν παράλληλο αλγόριθμο μπορεί να πραγματοποιηθεί υπό το καθεστώς της ισχύος λιγότερων περιορισμών.

Η χρήση αυτής της τεχνικής μείωσης δεδομένων στις εφαρμογές του OpenMP απαιτεί την προσθήκη του όρου

`reduction(<operator>:[variable list])`

στο τέλος των κατάλληλων οδηγιών παραλληλισμού, όπως είναι οι `for`, `loop`, `parallel`, `scope`, `sections`, `simd`, `taskloop` και `teams`, όπου `<operator>` είναι ένα από τα παραπάνω σύμβολα το οποίο προσδιορίζει την αντίστοιχη πράξη και `[variable list]` είναι η λίστα των μεταβλητών, οι τιμές των οποίων συγχωνεύονται με τον παραπάνω τρόπο



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η τεχνική της μείωσης ή υποβιβασμού δεδομένων

Τυπικά παραδείγματα χρήσης αυτού του όρου είναι η ενσωμάτωσή του στις οδηγίες

```
#pragma omp parallel(threads) reduction(+:result)
```

```
#pragma omp parallel for reduction(+:result)
```

Σε αμφότερες τις περιπτώσεις, το περιβάλλον του OpenMP

- δημιουργεί στο κάθε νήμα που συμμετέχει στον παράλληλο υπολογισμό, ένα ιδιωτικό αντίγραφο της μεταβλητής `result` το οποίο πριν από την είσοδο στην παράλληλη περιοχή, αρχικοποιείται στην κατάλληλη τιμή
- αποθηκεύει σε αυτή τη μεταβλητή το κατάλληλο σε κάθε περίπτωση αποτέλεσμα του μερικού υπολογισμού που πραγματοποιήθηκε από το αντίστοιχο νήμα
- στο τέλος της εμπλεκόμενης παράλληλης περιοχής και πριν την έξοδο από αυτή, συνδυάζει μεταξύ τους όλες αυτές τις τιμές για τον υπολογισμό του τελικού αποτελέσματος

Αυτή η διαδικασία πραγματοποιείται υπό συνθήκες ατομικής λειτουργίας, με τον χαρακτηρισμό της εν λόγω διαδικασίας ως κρίσιμο τμήμα και τις σχετιζόμενες μορφές διαχείρισης, να πραγματοποιούνται από το OpenMP.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση πρόσθεσης μονοδιάστατων πινάκων

Η πρόσθεση δύο μονοδιάστατων πινάκων **A** και **B** μήκους **N** στοιχείων ο καθένας, οδηγεί σε έναν τρίτο πίνακα **C** με **N** στοιχεία, το κάθε στοιχείο του οποίου περιέχει το άθροισμα των αντίστοιχων στοιχείων των πινάκων **A** και **B**. Η σειριακή εκδοχή αυτής της πράξης υλοποιείται από έναν μονοδιάστατο βρόχο της μορφής

```
for (i=0; i<N; i++) C[i]=A[i]+B[i];
```

Στον πηγαίο κώδικα της παράλληλης εκδοχής τα στοιχεία των δύο πινάκων λαμβάνουν τυχαίες τιμές στην περιοχή τιμών **[0,1]**.

Η εφαρμογή εκτελείται από την γραμμή εντολών, με δύο ορίσματα, εκ των οποίων το πρώτο όρισμα προσδιορίζει το πλήθος των νημάτων, ενώ το δεύτερο όρισμα προσδιορίζει το πλήθος των επαναλήψεων που ανατίθενται στο κάθε νήμα.

Εάν δεν δοθούν αυτές οι τιμές, ανατίθενται εξ ορισμού 10 επαναλήψεις στο κάθε νήμα, ενώ το πλήθος των νημάτων προσδιορίζεται από το διαθέσιμο πλήθος των επεξεργαστών.

Με εξαίρεση την εντολή ανάκτησης του αναγνωριστικού του κάθε νήματος και τις δύο γραμμές κώδικα στις οποίες καλείται η συνάρτηση **omp_get_wtime** για την καταμέτρηση της χρονικής διάρκειας εκτέλεσης της εφαρμογής, ο υπόλοιπος κώδικας είναι ο γνωστός σειριακός κώδικας ο οποίος παραλληλοποιείται με τις δύο οδηγίες τύπου **#pragma**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση πρόσθεσης μονοδιάστατων πινάκων

```
#include <omp.h>
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <stdlib.h>
#define N 10000

int main (int argc, char ** argv) {
    long int rank, i, threads, chunk;
    double A[N], B[N], C[N];
    double t1,t2;
    char mode [20];

    if (argc==1) {
        threads = omp_get_num_procs();
        chunk = 1; }
    else {
        threads = strtol (argv[1],NULL,10);
        chunk = strtol (argv[2],NULL,10); }

    rank=omp_get_thread_num();

    /* The main thread initializes vectors A and B */
    printf ("Using %ld threads with a chunk size of %ld\n",
            threads, chunk);
    for (i=0;i<N;i++) {
        A[i]=(double)rand()/RAND_MAX;
        B[i]=(double)rand()/RAND_MAX; }

    t1 = omp_get_wtime();
    #pragma parallel shared(A,B,C) private(i,rank)
    { /* start of parallel code */
        #pragma omp for schedule (static,chunk)
        for (i=0;i<N;i++) C[i]=A[i]+B[i];
    } /* end of parallel code */
    t2 = omp_get_wtime();

    printf ("Thread %ld: Process completed in %.12f sec\n", rank, t2-t1);
    printf ("Printing some elements ...\n");
    for (i=0;i<3;i++) printf ("A[i]=%f B[i]=%f ==> C[i]=%f\n",
        A[i],B[i],C[i]); }
```

```
amarg@amarg-vbox:~$ ./addvectors 5 100
Using 5 threads with a chunk size of 100
Thread 0: Process completed in 0.000053147000 sec
Printing some elements ...
A[i]=0.840188 B[i]=0.394383 ==> C[i]=1.234571
A[i]=0.783099 B[i]=0.798440 ==> C[i]=1.581539
A[i]=0.911647 B[i]=0.197551 ==> C[i]=1.109199
amarg@amarg-vbox:~$ ./addvectors 3 1000
Using 3 threads with a chunk size of 1000
Thread 0: Process completed in 0.000053895000 sec
Printing some elements ...
A[i]=0.840188 B[i]=0.394383 ==> C[i]=1.234571
A[i]=0.783099 B[i]=0.798440 ==> C[i]=1.581539
A[i]=0.911647 B[i]=0.197551 ==> C[i]=1.109199
amarg@amarg-vbox:~$
```

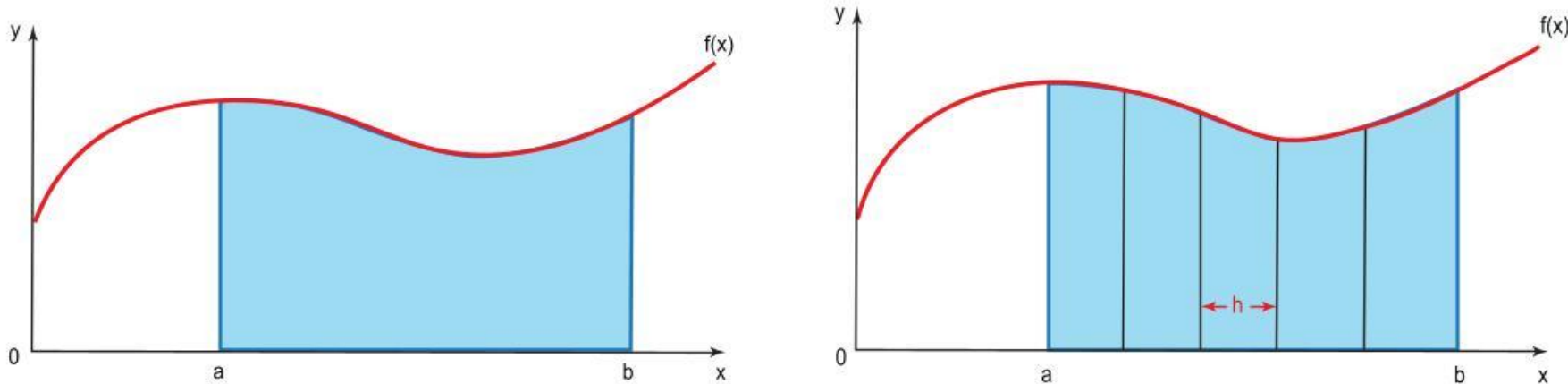


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραπεζίου

Από τα μαθηματικά γνωρίζουμε πως το ολοκλήρωμα μιας συνάρτησης $f(x)$ υπολογισμένο σε ένα διάστημα $[a,b]$, είναι ίσο με το εμβαδόν της επιφάνειας που ορίζεται από την καμπύλη της συνάρτησης $f(x)$, τις κατακόρυφες ευθείες $x = a$ και $x = b$ που τέμνουν τον οριζόντιο άξονα στα σημεία a και b , καθώς και τον ίδιο τον οριζόντιο άξονα.



Μία αριθμητική μέθοδος υπολογισμού αυτού του εμβαδού συνίσταται

- στην διαμέριση της παραπάνω επιφάνειας σε ένα πλήθος γεωμετρικών σχημάτων
- στον υπολογισμό του εμβαδού της επιφάνειας για το καθένα από αυτά τα γεωμετρικά σχήματα
- στην πρόσθεση μεταξύ τους όλων αυτών των τιμών που θα υπολογίσουμε.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

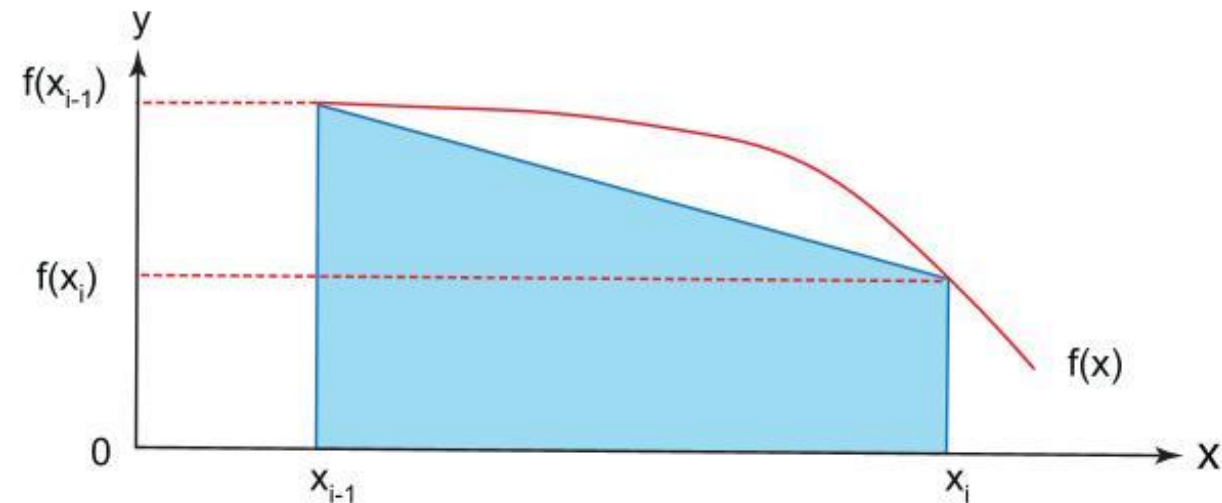
Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραpezίου

Υπάρχουν πολλές επιλογές σχετικά με το σχήμα των γεωμετρικών στοιχείων και εδώ θα θεωρήσουμε πως η επιφάνεια υποδιαιρείται σε ένα πλήθος N τραπεζίων, τα οποία έχουν όλα την ίδια βάση η οποία είναι ίση με $h=(\beta-\alpha)/N$ όπου α και β είναι τα άκρα του διαστήματος ολοκλήρωσης

Από τη μελέτη του προβλήματος διαπιστώνουμε πως η βάση του υπ' αριθμόν i τραpezίου, θα είναι το διάστημα $[\alpha+(i-1)h, \alpha h]$.

Ορίζοντας $x_i = \alpha + ih$, το ύψος της αριστερής πλευράς του τραpezίου θα είναι ίσο με $f(x_{i-1})$, ενώ το ύψος της δεξιάς πλευράς του τραpezίου, θα είναι ίσο με $f(x_i)$. Επομένως, το εμβαδόν του υπ' αριθμόν i τραpezίου θα δίδεται από τη σχέση

$$E_i = \frac{h}{2} (f(x_{i-1}) + f(x_i))$$



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραpezίου

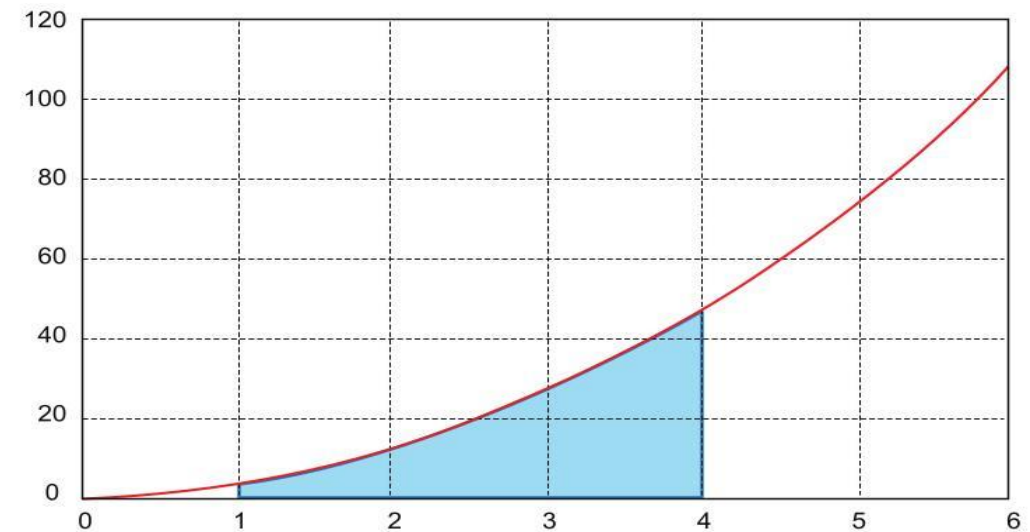
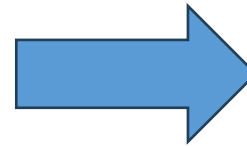
Προκειμένου να υπολογίσουμε λοιπόν το **συνολικό εμβαδόν της σκιασμένης επιφάνειας**, θα πρέπει να **αθροίσουμε** τα εμβαδά όλων αυτών των **στοιχειωδών τραπεζίων**. Επομένως, το **συνολικό εμβαδόν**, θα δίδεται από την σχέση

$$E = \frac{h}{2}[f(x_0) + f(x_1)] + \frac{h}{2}[f(x_1) + f(x_2)] + \cdots + \frac{h}{2}[f(x_{N-1}) + f(x_N)] = h \left[\frac{f(x_0)}{2} + \frac{f(x_N)}{2} + f(x_1) + f(x_2) + \cdots + f(x_{N-1}) \right]$$

Με τον τρόπο αυτό, μπορούμε να υπολογίσουμε **προσεγγιστικά** την τιμή του ολοκληρώματος **οποιασδήποτε** συνάρτησης **f(x)**.

Όσο **πιο μεγάλος** είναι ο αριθμός **N** των τραπεζίων στα οποία **υποδιαιρούμε** την **σκιασμένη επιφάνεια** το εμβαδόν της οποίας επιθυμούμε να υπολογίσουμε, **τόσο πιο πολύ** η υπολογιζόμενη τιμή θα **προσεγγίζει** την **πραγματική τιμή** του **ολοκληρώματος** της συνάρτησης **f(x)** στο **κλειστό διάστημα [a,b]**.

Υπολογισμός του ολοκληρώματος συνάρτησης **f(x) = 3x² + 1** στο κλειστό διάστημα **[1, 4]** με τη **μέθοδο των τραπεζίων**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραπεζίου

Ο πηγαίος κώδικας της σειριακής υλοποίησης του υπολογισμού του ολοκληρώματος

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>

double func (double x) {
    return (3*pow(x,2)+1); }

void main (int argc, char ** argv) {
    double x, h, a = 1.0, b = 4.0;
    double integral = 0.0;
    long int i, N;
    clock_t t1,t2;
    t1 = clock();
    if (argc==1) N=10;
    else N = strtol(argv[1],NULL,10);
    h = (b-a)/N;
    integral = (func(a)+func(b))/2;
    x=a;
    for (i=1;i<N-1;i++) {
        x=x+h;
        integral=integral+func(x); }
    integral = integral * h;
    t2 = clock()-t1;
    printf ("Integral: %.6f (N=%ld, T=%.6f secs)\n", integral, N, ((double)t2)/CLOCKS_PER_SEC); }
```

```
amarg@amarg-vbox:~$ ./serial_integration 1
Integral (N=1) = 79.500000 (T=0.000007 secs)
amarg@amarg-vbox:~$ ./serial_integration 10
Integral (N=10) = 53.514000 (T=0.000013 secs)
amarg@amarg-vbox:~$ ./serial_integration 100
Integral (N=100) = 64.552869 (T=0.000010 secs)
amarg@amarg-vbox:~$ ./serial_integration 1000
Integral (N=1000) = 65.853229 (T=0.000019 secs)
amarg@amarg-vbox:~$ ./serial_integration 10000
Integral (N=10000) = 65.985302 (T=0.000143 secs)
amarg@amarg-vbox:~$ ./serial_integration 100000
Integral (N=100000) = 65.998530 (T=0.001396 secs)
amarg@amarg-vbox:~$ ./serial_integration 1000000
Integral (N=1000000) = 65.999853 (T=0.015742 secs)
amarg@amarg-vbox:~$ ./serial_integration 10000000
Integral (N=10000000) = 65.999985 (T=0.142425 secs)
amarg@amarg-vbox:~$ ./serial_integration 100000000
Integral (N=100000000) = 65.999998 (T=1.408646 secs)
amarg@amarg-vbox:~$
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραπεζίου

Οι πληροφορίες που είναι αναγκαίες για την υλοποίηση της **παράλληλης αριθμητικής ολοκλήρωσης**, είναι τα **όρια** του διαστήματος **[α,β]** και το **πλήθος N των τραπεζίων** που θα χρησιμοποιήσουμε ενώ η ανάθεση έργου θα στηριχθεί στο αναγνωριστικό του νήματος **rank** και στο πλήθος **size** των νημάτων της εφαρμογής.

Επειδή σε αυτό το πρόβλημα **δεν υφίστανται εξαρτήσεις δεδομένων**, μπορούμε να δημιουργήσουμε μία ομάδα **N νημάτων** και να αναθέσουμε **στο καθένα** από αυτά, τη διαδικασία υπολογισμού του εμβαδού **ενός και μοναδικού τραπεζίου**. Μετά τον υπολογισμό **όλων των παραπάνω τιμών**, αυτές **θα πρέπει να προστεθούν μεταξύ τους** για να δώσουν το τελικό αποτέλεσμα

Επειδή στην πράξη **η τιμή του N μπορεί να είναι πολύ μεγάλη**, θα χρησιμοποιήσουμε **size = M νήματα** για τον υπολογισμό του εμβαδού **N τραπεζίων**, αναθέτοντας στο **καθένα από αυτά** τον υπολογισμό του εμβαδού **μίας ολόκληρης ομάδας τραπεζίων**. Εδώ, η τιμή του M, **διαιρεί επακριβώς** την τιμή του N, έτσι ώστε **το κάθε νήμα να υπολογίσει το εμβαδόν N/M τραπεζίων**, ενώ στην **αντίθετη** περίπτωση, η εφαρμογή θα εκτυπώσει μήνυμα σφάλματος:

```
if (N%size) {  
    printf("The number of trapezoids must be an integer multiple of the number of threads\n");  
    exit (0); }
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραpezίου

Ο παράλληλος αλγόριθμος υπολογισμού του ολοκληρώματος μιας συνάρτησης, περιλαμβάνει τα ακόλουθα στάδια:

1. Το κάθε νήμα θα υπολογίσει το δικό του διάστημα ολοκλήρωσης που ορίζεται από τις τιμές των μεταβλητών `localA` και `localB`.

Για τον υπολογισμό αυτών των τιμών, σύμφωνα με τα όσα αναφέραμε προηγουμένως, αρχικά υπολογίζουμε την βάση των τραπεζίων ως $base = (\beta - \alpha) / N$ και το πλήθος των τραπεζίων ανά διεργασία ως $localN = N / size$ και στη συνέχεια υπολογίζουμε τις παραπάνω τιμές ως $localA = \alpha + rank \cdot localN \times base$ και $localB = localA + localN \times base$.

2. Το κάθε νήμα θα προχωρήσει στον υπολογισμό του εμβαδού των τραπεζίων που ανήκουν στο διάστημα ολοκλήρωσης $[localA, localB]$ και το οποίο σύμφωνα με την ανάλυση που παραθέσαμε ορίζεται ως η περιοχή τιμών

$$\left[\alpha + rank \frac{\beta - \alpha}{size}, \alpha + (rank + 1) \frac{\beta - \alpha}{size} \right]$$

και στη συνέχεια θα προσθέσει την τιμή `localIntegral` που υπολόγισε στην κοινόχρηστη μεταβλητή `totalIntegral`, η οποία κατά τον τερματισμό της εφαρμογής θα περιέχει τη ζητούμενη τιμή του ολοκληρώματος της συνάρτησης. Για την αποφυγή συνθηκών ανταγωνισμού, χρησιμοποιούμε την οδηγία `#pragma omp critical` πάνω από την γραμμή `*totalIntegral += localIntegral;`



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραπεζίου

```
#include <omp.h>
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>

double func (double x) {
    return (3*pow(x,2)+1); }

void CalcIntegral (double a, double b, long int N, void main (int argc, char ** argv) {
    double *totalIntegral) {
    double x, localIntegral=0.0;
    double base = (b-a)/N;
    int rank = omp_get_thread_num();
    int size = omp_get_num_threads();
    long int i, localN = N/size;
    double localA = a+rank*localN*base;
    double localB = localA + localN*base;
    localIntegral = (func(localA)+func(localB))/2.0;
    x=localA;
    for (i=1;i<localN-1;i++) {
        x=x+base;
        localIntegral += func(x); }
    localIntegral = localIntegral*base;
    printf ("Thread %d: localA = %lf, localB = %lf,
            localN = %ld\n",rank, localA, localB, localN);
    printf ("Local Integral for Thread %d is %lf\n",
            rank, localIntegral);
    #pragma omp critical
    *totalIntegral += localIntegral; }
```

```
double a,b, totalIntegral = 0.0;
long int N, size;
clock_t t1,t2;
if (argc==1) size = 2;
else size = strtol(argv[1],NULL,10);
printf ("Number of threads is %ld\n", size);
printf ("Give the value of a: "); scanf ("%lf", &a);
printf ("Give the value of b: "); scanf ("%lf", &b);
printf ("Give the value of N: "); scanf ("%ld", &N);
t1 = clock();
#pragma omp parallel num_threads(size)
CalcIntegral (a,b,N,&totalIntegral);
t2 = clock()-t1;
printf ("Integral: %.6f (N=%ld, T=%.6f secs)\n",
        totalIntegral, N, ((double)t2)/CLOCKS_PER_SEC); }
```

Η παράλληλη υλοποίηση της
αριθμητικής ολοκλήρωσης με
χειρωνακτική ανάθεση έργου

```
amarg@amarg-vbox:~$ ./parallel_integration 4
Number of threads is 4
Give the value of a: 1.0
Give the value of b: 4.0
Give the value of N: 100000000
Thread 0: localA = 1.000000, localB = 1.750000, localN=25000000
Local Integral for Thread 0 is 5.109375
Thread 2: localA = 2.500000, localB = 3.250000, localN=25000000
Local Integral for Thread 2 is 19.453124
Thread 1: localA = 1.750000, localB = 2.500000, localN=25000000
Local Integral for Thread 1 is 11.015624
Thread 3: localA = 3.250000, localB = 4.000000, localN=25000000
Local Integral for Thread 3 is 30.421873
Integral: 65.999997 (N=100000000, T=6.894648 secs)
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – παραλληλοποίηση του κανόνα του τραπεζίου

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

double func (double x) { return (3*pow(x,2)+1); }

double integrate (double a, double b, int N) {
    double x = a, localIntegral = 0.0;
    double base = (b-a) / N;
    localIntegral = (func(a)+func(b))/2;
    localIntegral /= 2;
    #pragma omp parallel for private(x)
    for (int i=1;i<N;i++) {
        x = a + i*base;
        #pragma omp critical
        localIntegral += func(x);
    }
    localIntegral = localIntegral * base;
    return localIntegral; }
```

Η παράλληλη υλοποίηση της
αριθμητικής ολοκλήρωσης με
αυτόματη ανάθεση έργου

```
amarg@amarg-vbox:~$ gcc -o parfor_integration
amarg@amarg-vbox:~$ ./parfor_integration 10
Number of threads is 10
Give the value of a: 1.0
Give the value of b: 4.0
Give the value of N: 1000000000
Integral: 66.000000 (N=1000000000, T=11.893056 secs)
amarg@amarg-vbox:~$
```

```
void main (int argc, char ** argv) {
    double a,b, integral = 0.0;
    long int N, size;
    clock_t t1,t2;
    if (argc==1) size = 2;
    else size = strtol(argv[1],NULL,10);
    printf ("Number of threads is %ld\n", size);
    printf ("Give the value of a: ");
    scanf ("%lf", &a);
    printf ("Give the value of b: ");
    scanf ("%lf", &b);
    printf ("Give the value of N: ");
    scanf ("%ld", &N);
    t1 = clock();
    integral = integrate (a,b,N);
    t2 = clock()-t1;
    printf ("Integral: %.6f (N=%ld, T=%.6f secs)\n",
        integral, N, ((double)t2)/CLOCKS_PER_SEC); }
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση – Υπολογισμός της τιμής της σταθεράς π

Η σταθερά π που αποτελεί έναν άρρητο αριθμό με τιμή 3.14159265359..., ορίζεται ως ο λόγος της περιμέτρου ενός κύκλου προς την διάμετρό του και μπορεί να υπολογιστεί με πάρα πολλούς τρόπους.

Εδώ το π υπολογίζεται ως το αποτέλεσμα της ολοκλήρωσης της συνάρτησης $f(x) = 4/(1 + x^2)$ στο κλειστό διάστημα $[0,1]$. Σύμφωνα με αυτή τη μέθοδο, η τιμή του π δίδεται από την σχέση

$$\pi = \int_0^1 \frac{4}{1+x^2} dx = \sum_{i=0}^{N-1} \frac{4W}{1+[(i+0.5W)]^2}$$

όπου N είναι το πλήθος των διαστημάτων στα οποία χωρίζουμε το διάστημα $[0,1]$. Η ισχύς της παραπάνω σχέσης τεκμηριώνεται από το γεγονός, πως η παράγωγος ως προς x της συνάρτησης $f(x) = \tan^{-1}x$ είναι $f'(x) = 1/(1+x^2)$. Θα είναι λοιπόν

$$\int_0^1 \frac{dx}{1+x^2} = \tan^{-1}(1) - \tan^{-1}(0) = \frac{\pi}{4} \quad \text{από όπου προκύπτει ότι} \quad \pi = \int_0^1 \frac{4}{1+x^2} dx$$

Επομένως, ο υπολογισμός της τιμής του π αποτελεί ειδική περίπτωση της μεθόδου αριθμητικής ολοκλήρωσης χρησιμοποιώντας ως συνάρτηση $f(x)$ την συνάρτηση $f(x) = 4/(1+x^2)$ και ως διάστημα ολοκλήρωσης την περιοχή τιμών $[a,b] = [0,1]$.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Αριθμητική ολοκλήρωση και μείωση ή υποβιβασμός δεδομένων

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

double func (double x) {
    return (3*pow(x,2)+1); }

double integrate (double a, double b, int N) {
    double x = a, localIntegral = 0.0;
    double base = (b-a) / N;
    localIntegral = (func(a)+func(b))/2;
    localIntegral /= 2;
    #pragma omp parallel for private(x) \
        reduction(+:localIntegral)
    for (int i=1;i<N;i++) {
        x = a + i*base;
        localIntegral += func(x); }
    localIntegral = localIntegral * base;
    return localIntegral; }
```

Ο υπολογισμός του τελικού αθροίσματος
στηρίζεται στην πράξη της μείωσης δεδομένων

```
amarg@amarg-vbox:~$ ./reduction_integration 10
Number of threads is 10
Give the value of a: 1.0
Give the value of b: 4.0
Give the value of N: 1000000000
Integral: 66.000000 (N=1000000000, T=6.591523 secs)
amarg@amarg-vbox:~$
```

```
void main (int argc, char ** argv) {
    double a,b, integral = 0.0;
    long int N, size;
    clock_t t1,t2;
    if (argc==1) size = 2;
    else size = strtol(argv[1],NULL,10);
    printf ("Number of threads is %ld\n", size);
    printf ("Give the value of a: ");
    scanf ("%lf", &a);
    printf ("Give the value of b: ");
    scanf ("%lf", &b);
    printf ("Give the value of N: ");
    scanf ("%ld", &N);
    t1 = clock();
    integral = integrate (a,b,N);
    t2 = clock()-t1;
    printf ("Integral: %.6f (N=%ld, T=%.6f secs)\n",
        integral, N, ((double)t2)/CLOCKS_PER_SEC); }
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Υπολογισμός της τιμής του π με την μέθοδο του Monte Carlo

Στις μεθόδους τύπου **Monte Carlo** η υλοποίηση των **αλγορίθμων** στηρίζεται σε **τεχνικές στατιστικής δειγματοληψίας**

Η εφαρμογή της **μεθόδου του Monte Carlo** για τον υπολογισμό της **σταθεράς π** , πραγματοποιείται ως εξής:

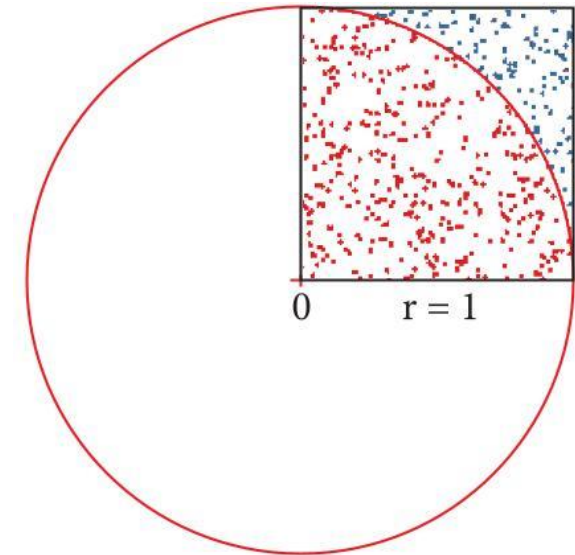
Θεωρώντας κύκλο **διαμέτρου D** και το **περιγεγραμμένο τετράγωνό** του **μήκους πλευράς D** , ο **λόγος των εμβαδών τους** είναι

$$\frac{E_c}{E_d} = \frac{\pi D^2/4}{D^2} = \frac{\pi}{4}$$

Για τον υπολογισμό αυτού του **λόγου** παράγουμε **τυχαία σημεία** με συντεταγμένες $0 \leq x, y \leq 1$ τα οποία ανήκουν σε **τετράγωνο** με **μοναδιαίο μήκος πλευράς**.

Εάν επιπλέον ισχύει $x^2 + y^2 \leq 1$ αυτά τα σημεία ανήκουν **επιπλέον** στο **εσωτερικό κύκλου μοναδιαίας ακτίνας**.

Εάν το πλήθος των σημείων είναι **πολύ μεγάλο**, ο **λόγος** των σημείων που βρίσκονται **στο εσωτερικό του μοναδιαίου κύκλου** προς το **συνολικό** πλήθος των σημείων (που βρίσκονται **στο εσωτερικό του μοναδιαίου τετραγώνου**) προσεγγίζει σε **πάρα πολύ καλό βαθμό** την τιμή του **λόγου** των εμβαδών E_c/E_r .



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Υπολογισμός της τιμής της σταθεράς π με την μέθοδο του Monte Carlo

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

void monteCarlo (int points, int threads) {
    double x, y, d, pi;
    int i = 0, pCircle = 0, pSquare = 0;

#pragma omp parallel firstprivate(x, y, d, i) \
    reduction(+ : pCircle, pSquare) \
    num_threads(threads)
{
    srand((int)time(NULL));
    for (i = 0; i < points; i++) {
        x = (double)rand()/RAND_MAX;
        y = (double)rand()/RAND_MAX;
        d = pow(x,2)+pow(y,2);
        if (d <= 1) pCircle++;
        pSquare++; }

    pi = 4.0 * ((double)pCircle / (double)(pSquare));
    printf("The estimated value of Pi is %f\n", pi); }
```

```
amarg@amarg-vbox:~$ ./parMonteCarlo 10 5
The estimated value of Pi is 3.200000
amarg@amarg-vbox:~$ ./parMonteCarlo 100 5
The estimated value of Pi is 3.000000
amarg@amarg-vbox:~$ ./parMonteCarlo 1000 5
The estimated value of Pi is 3.028000
amarg@amarg-vbox:~$ ./parMonteCarlo 10000 5
The estimated value of Pi is 3.158720
amarg@amarg-vbox:~$ ./parMonteCarlo 100000 5
The estimated value of Pi is 3.141024
amarg@amarg-vbox:~$ ./parMonteCarlo 1000000 5
The estimated value of Pi is 3.142659
amarg@amarg-vbox:~$ ./parMonteCarlo 10000000 5
The estimated value of Pi is 3.141643
amarg@amarg-vbox:~$ ./parMonteCarlo 100000000 5
The estimated value of Pi is 3.141654
```

```
int main(int argc, char ** argv) {
    long int randPoints, threads;
    if (argc==1) {
        randPoints = 100000;
        threads = 2; }
    else {
        randPoints = strtol(argv[1],NULL,10);
        threads = strtol(argv[2],NULL,10); }
    monteCarlo(randPoints, threads); }
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Υπολογισμός της τιμής της σταθεράς π με τον τύπο του Wallis

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

int main(int argc, char **argv) {
    long int iterations, threads;
    if (argc==1) {
        iterations = 100000;
        threads = 2; }
    else {
        iterations = strtol(argv[1],NULL,10);
        threads = strtol(argv[2],NULL,10); }
    double piValue=1.0, t1, t2;
    omp_set_num_threads(threads);
    t1 = omp_get_wtime();

    #pragma omp parallel for reduction(*:piValue)
    for (int i=1;i<=iterations;i++)
        piValue *= (double) (4*pow(i,2))/(double) (4*pow(i,2)-1);

    piValue *= 2;
    t2 = omp_get_wtime();
    printf("Estimated pi value is %.8f (Duration %.8f)\n", piValue, t2-t1);
    return 0; }
```

```
amarg@amarg-vbox:~$ gcc -o piWallis -fopenmp piWallis.c -lm
amarg@amarg-vbox:~$ ./piWallis 10000 4
Estimated pi value is 3.14151412 (Duration 0.00185478)
amarg@amarg-vbox:~$ ./piWallis 100000 4
Estimated pi value is 3.14158480 (Duration 0.01172005)
amarg@amarg-vbox:~$ ./piWallis 1000000 4
Estimated pi value is 3.14159187 (Duration 0.11921951)
amarg@amarg-vbox:~$ ./piWallis 10000000 4
Estimated pi value is 3.14159258 (Duration 1.10741088)
amarg@amarg-vbox:~$ ./piWallis 100000000 4
Estimated pi value is 3.14159264 (Duration 11.13281877)
amarg@amarg-vbox:~$
```

Ο τύπος του Wallis έχει τη μορφή

$$\frac{\pi}{2} = \left(\frac{2}{1} \cdot \frac{2}{3}\right) \cdot \left(\frac{4}{3} \cdot \frac{4}{5}\right) \cdot \left(\frac{6}{5} \cdot \frac{6}{7}\right) \cdot \left(\frac{8}{7} \cdot \frac{8}{9}\right) \cdots = \prod_{i=1}^{\infty} \frac{2i}{2i-1} \cdot \frac{2i}{2i+1} = \prod_{i=1}^{\infty} \frac{4i^2}{4i^2-1}$$

από όπου προκύπτει ότι

$$\pi = 2 \prod_{i=1}^{\infty} \frac{4i^2}{4i^2-1}$$



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Πράξεις υποβιβασμού δεδομένων ορισμένες από τον χρήστη

Η **declare reduction** επιτρέπει τον υπολογισμό του αθροίσματος των στοιχείων μονοδιάστατου πίνακα μιγαδικών αριθμών .

Το άθροισμα δύο μιγαδικών αριθμών $x = a + ib$ και $y = c + id$ είναι ο μιγαδικός αριθμός $z = x + y = (a + c) + i(b + d)$.

Επειδή η γλώσσα C δεν προσφέρει τύπους δεδομένων που να αναπαριστούν μιγαδικούς αριθμούς, ούτε υποστηρίζει τις σχετιζόμενες με αυτούς αριθμητικές πράξεις, αυτές θα πρέπει να οριστούν από τον χρήστη.

Κατά συνέπεια, προκειμένου να χρησιμοποιηθεί ο όρος **reduction**, θα πρέπει προηγουμένως να ορίσουμε την πρόσθεση ανάμεσα στους μιγαδικούς αριθμούς, χρησιμοποιώντας την οδηγία **declare reduction**, η οποία συνιστά μία δηλωτική οδηγία.

Στον επισυναπτόμενο κώδικα ο μιγαδικός αριθμός ορίζεται ως μία δομή **complex** με στοιχεία **real** και **imag** τύπου **double**, ενώ η πρόσθεση των μιγαδικών αριθμών πραγματοποιείται από τη συνάρτηση **addComplex**, που δέχεται ως ορίσματα δύο δείκτες προς δύο δομές **a** και **b** τύπου **complex** και επιστρέφει το άθροισμά τους μέσω του πρώτου ορίσματος **a**.

Η μορφή αυτή της συνάρτησης δικαιολογείται από το γεγονός πως η πράξη του υποβιβασμού δεδομένων συνίσταται στον προοδευτικό συνδυασμό των διαδοχικών τιμών των εμπλεκόμενων μεταβλητών, με αυτόν ακριβώς τον τρόπο.

Επίσης ορίζεται και το ουδέτερο στοιχείο της πράξης, που είναι ο μιγαδικός αριθμός $0 + i0$.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ο πηγαίος κώδικας της εφαρμογής αθροίσματος στοιχείων πίνακα μιγαδικών αριθμών

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>

#define N 100

struct complex {
    double real;
    double imag; };

void addComplex (struct complex * a, struct complex * b) {
    a->real = a->real+b->real;
    a->imag = a->imag+b->imag; }

#pragma omp declare reduction(sum : struct complex : \
    addComplex (&omp_out,&omp_in)) \
    initializer (omp_priv={0,0})
```

```
int main (void) {
    struct complex psum={0,0}, sum={0,0}, buffer[N];
    for (int i=0;i<N;i++) {
        buffer[i].real=(double)(i+1);
        buffer[i].imag=(double)(i+1); }
    #pragma omp parallel num_threads(5) firstprivate(buffer)
    {
        #pragma omp for reduction (sum:psum)
        for (int i=0;i<N;i++)
            addComplex(&psum,&buffer[i]);
        #pragma omp single
        printf ("Thread %d ==> Real = %f, Imaginary = %f\n",
            omp_get_thread_num(), psum.real, psum.imag); }
}
```

```
amarg@amarg-vbox:~$ gcc reduction.c -fopenmp -o udr
amarg@amarg-vbox:~$ ./udr
Thread 2 ==> Real = 5050.000000, Imaginary = 5050.000000
amarg@amarg-vbox:~$
```

Η εφαρμογή χρησιμοποιεί **πέντε νήματα**, με το κάθε τμήμα να καταχωρεί στη μεταβλητή **psum** το **μερικό άθροισμα** που υπολόγισε και τον όρο **reduction (sum:psum)** της οδηγίας **#pragma omp parallel for** να **προσθέτει μεταξύ τους** τις τιμές των **μερικών αθροισμάτων** και να υπολογίζει το **τελικό αποτέλεσμα**.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Παραλληλοποίηση της διαδικασίας αναζήτησης πρώτων αριθμών

Ένας αριθμός χαρακτηρίζεται ως **πρώτος**, όταν **δεν μπορεί να διατυπωθεί** ως ένα **γινόμενο** δύο **μικρότερων φυσικών αριθμών** (οπότε χαρακτηρίζεται ως **σύνθετος**) ή ισοδύναμα, όταν διαιρείται **ακριβώς** μόνο από **τον εαυτό του** και την **μονάδα**.

Μία μέθοδος **αναζήτησης** τέτοιων αριθμών, στηρίζεται στην παρατήρηση πως εάν ένας **φυσικός αριθμός x** μπορεί να γραφτεί ως το **γινόμενο** δύο **μικρότερων φυσικών αριθμών** της μορφής **$a \times b$** είναι **αδύνατον** αυτοί οι δύο αριθμοί να είναι **αμφότεροι** μεγαλύτεροι από **$\sqrt{x}$** , αφού στην περίπτωση αυτή το γινόμενο των δύο αριθμών θα είναι **μεγαλύτερο** από x .

Δηλαδή, σε κάθε απόπειρα **παραγοντοποίησης** ενός **φυσικού αριθμού x** ή ως ένα τέτοιο γινόμενο, **τουλάχιστον ο ένας** από τους δύο αριθμούς, θα είναι **μικρότερος** από την **τετραγωνική ρίζα** του x .

Προκειμένου λοιπόν να ελέγξουμε εάν κάποιος **φυσικός αριθμός $x \geq 3$** είναι **πρώτος** (το 2 **είναι** πρώτος **όχι όμως** και το 1)

- υπολογίζουμε την **τετραγωνική του ρίζα**
- εξετάζουμε εάν κάποιος από τους **φυσικούς αριθμούς** που έχουν τιμή **μικρότερη από το ακέραιο μέρος της τετραγωνικής ρίζας του αριθμού x** , διαιρεί **ακριβώς** τον αριθμό x .

Εάν διαπιστώσουμε πως **κανένας από αυτούς τους αριθμούς** δεν διαιρεί **ακριβώς** τον αριθμό x , τότε ο αριθμός x είναι **πρώτος αριθμός**, ενώ στην **αντίθετη** περίπτωση χαρακτηρίζεται ως **σύνθετος**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η σειριακή διαδικασία αναζήτησης πρώτων αριθμών

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

int is_prime (int number) {
    int factor, maxlimit = sqrt(number);
    for (factor=3; factor<=maxlimit; factor++) {
        if (number%factor!=0) continue;
        else return 0; }
    return 1; }

int main (int argc, char ** argv) {
    int * buffer, i, print;
    long int maxN, count = 0;
    if (argc!=2) { maxN = 1000; }
    else { maxN=strtol (argv[1],NULL,10);
           if (maxN<3) maxN=3; }
    printf ("Range of search is [1, %ld]\n", maxN);
    buffer = (int *) malloc(sizeof(int) * maxN);
    if (!buffer) { printf ("Not enough memory\n"); return (-1); }
    for (i=0;i<maxN;i++) buffer[i] = 0;
    for (i=3;i<maxN;i+=2) {
        if (is_prime (i)) {
            buffer[count]=i;
            count++; }}
    printf("Prime numbers found in range [1-%ld] is %ld\n", maxN,count);
    for (i=0;i<count;i++) printf("%d ",buffer[i]);
    printf("\n");
    free(buffer);
    return (0); }
```

```
amarg@amarg-vbox:~$ gcc -o omp_prime_serial omp_prime_serial.c -lm
amarg@amarg-vbox:~$ ./omp_prime_serial 1000
Range of search is [1, 1000]
Prime numbers found in range [1-1000] is 167
3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107
109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199 211 223
227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337
347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457
461 463 467 479 487 491 499 503 509 521 523 541 547 557 563 569 571 577 587 593
599 601 607 613 617 619 631 641 643 647 653 659 661 673 677 683 691 701 709 719
727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 829 839 853 857
859 863 877 881 883 887 907 911 919 929 937 941 947 953 967 971 977 983 991 997
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η σειριακή διαδικασία αναζήτησης πρώτων αριθμών με καταμέτρηση του πλήθους των επαναλήψεων

```
#include <stdio.h>
#include <omp.h>
#include <math.h>
#include <stdlib.h>

int is_prime (int number, int * iterations) {
    int temp = 0, factor, maxlimit = sqrt(number);
    for (factor=3; factor<=maxlimit; factor++) {
        if (number%factor!=0) { temp++; continue; }
        else return 0; }
    *iterations = temp;
    return 1; }
```

Σε αυτή την εκδοχή του κώδικα, η συνάρτηση `is_prime` επιστρέφει και το πλήθος των επαναλήψεων που πραγματοποίησε

```
int main (int argc, char ** argv) {
    int * buffer, i, * iters, iterations;
    long int maxN, count = 0;
    if (argc!=2) { maxN = 1000; }
    else { maxN=strtol (argv[1],NULL,10);
        if (maxN<3) maxN=3; }
    printf ("Range of search is [1, %ld]\n", maxN);
    buffer = (int *) malloc(sizeof(int) * maxN);
    iters = (int *) malloc(sizeof(int) * maxN);
    if ((!buffer)||(!iters)) { printf ("Not enough memory\n");
        return (-1); }

    for (i=0;i<maxN;i++) { buffer[i] = 0; iters[i] = 0; }
    for (i=3;i<maxN;i+=2) {
        iterations = 0;
        if (is_prime (i,&iterations)) {
            buffer[count]=i;
            iters[count]=iterations;
            count++; } }
    printf("Prime numbers found in [1- %ld] is %ld\n", maxN,count);
    for (i=0;i<count;i++) printf("%d(%d) ", buffer[i], iters[i]);
    printf("\n");
    free(buffer); free (iters);
    return (0); }
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η σειριακή διαδικασία αναζήτησης πρώτων αριθμών με καταμέτρηση του πλήθους των επαναλήψεων

Όσο μεγαλύτερος είναι ο αριθμός, τόσο μεγαλύτερο είναι και το πλήθος των επαναλήψεων που απαιτούνται

```
amarg@amarg-vbox:~$ ./omp_prime_serial_iter 1000
Range of search is [1, 1000]
Prime numbers found in range [1-1000] is 167
3(0) 5(0) 7(0) 11(1) 13(1) 17(2) 19(2) 23(2) 29(3) 31(3) 37(4) 41(4) 43(4) 47(4)
53(5) 59(5) 61(5) 67(6) 71(6) 73(6) 79(6) 83(7) 89(7) 97(7) 101(8) 103(8) 107(8)
109(8) 113(8) 127(9) 131(9) 137(9) 139(9) 149(10) 151(10) 157(10) 163(10) 167(10)
173(11) 179(11) 181(11) 191(11) 193(11) 197(12) 199(12) 211(12) 223(12) 227(12)
229(13) 233(13) 239(13) 241(13) 251(13) 257(14) 263(14) 269(14) 271(14) 277(14)
281(14) 283(14) 293(15) 307(15) 311(15) 313(15) 317(15) 331(16) 337(16) 347(16)
349(16) 353(16) 359(16) 367(17) 373(17) 379(17) 383(17) 389(17) 397(17) 401(17)
409(18) 419(18) 421(18) 431(18) 433(18) 439(18) 443(19) 449(19) 457(19) 461(19)
463(19) 467(19) 479(19) 487(20) 491(20) 499(20) 503(20) 509(20) 521(20) 523(20)
541(21) 547(21) 557(21) 563(21) 569(21) 571(21) 577(22) 587(22) 593(22) 599(22)
601(22) 607(22) 613(22) 617(22) 619(22) 631(23) 641(23) 643(23) 647(23) 653(23)
659(23) 661(23) 673(23) 677(24) 683(24) 691(24) 701(24) 709(24) 719(24) 727(24)
733(25) 739(25) 743(25) 751(25) 757(25) 761(25) 769(25) 773(25) 787(26) 797(26)
809(26) 811(26) 821(26) 823(26) 827(26) 829(26) 839(26) 853(27) 857(27) 859(27)
863(27) 877(27) 881(27) 883(27) 887(27) 907(28) 911(28) 919(28) 929(28) 937(28)
941(28) 947(28) 953(28) 967(29) 971(29) 977(29) 983(29) 991(29) 997(29)
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η σειριακή διαδικασία αναζήτησης πρώτων αριθμών με καταμέτρηση του πλήθους των επαναλήψεων

Η παραλληλοποίηση της διαδικασίας αναζήτησης πρώτων αριθμών, συνίσταται στον διαμοιρασμό των επαναλήψεων του βρόχου αναζήτησης στα νήματα της εφαρμογής και μπορεί να πραγματοποιηθεί είτε αναθέτοντας ρητά στο κάθε νήμα, την αναζήτηση πρώτων αριθμών σε μία συγκεκριμένη περιοχή τιμών, είτε αναθέτοντας στο OpenMP να διαμοιράσει με τον κατάλληλο τρόπο τις επαναλήψεις του βρόχου αναζήτησης, χρησιμοποιώντας την οδηγία `#pragma omp parallel for`.

Δεν χρησιμοποιείται ο όρος `schedule` και κατά συνέπεια η παραλληλοποίηση πραγματοποιείται με τον προεπιλεγμένο τρόπο, δηλαδή ακολουθείται η πολιτική `static` ενώ το πλήθος των επαναλήψεων που ανατίθεται στο κάθε νήμα είναι περίπου ίσο με τον λόγο της τιμής του N δια την τιμή του πλήθους των νημάτων.

Προκειμένου να αντιληφθούμε τις μεγάλες διαφορές στους χρόνους εκτέλεσης, χρησιμοποιήθηκε ο όρος `nowait` στον βρόχο παραλληλοποίησης, έτσι ώστε να απενεργοποιήσουμε τον μηχανισμό της χρονικής φραγής που επιβάλλεται από το περιβάλλον του OpenMP στο τέλος του παράλληλου κώδικα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η σειριακή διαδικασία αναζήτησης πρώτων αριθμών με καταμέτρηση του πλήθους των επαναλήψεων

Πλήθος εξεταζόμενων αριθμών	Πλήθος επαναλήψεων ανά νήμα					Συνολικές επαναλήψεις για όλα τα νήματα	Πλήθος πρώτων αριθμών
	RANK=0	RANK=1	RANK=2	RANK=3	RANK==4		
1000	295 0,000012	491 0,000014	617 0,000016	692 0,000018	796 0,000019	2891	168
5000	2891 0,000077	4847 0,000125	6065 0,00012	6799 0,000131	7617 0,000143	28219	669
10000	7738 0,000182	12864 0,000268	15783 0,000316	18136 0,000344	20494 0,000359	75015	1229
50000	75015 0,001434	123098 0,000659	152480 0,002294	176580 0,002939	194843 0,003266	722016	5133
100000	198113 0,00204	329060 0,003791	409185 0,015537	466177 0,020286	521772 0,009931	1924307	9592
500000	1924307 0,07857	3209167 0,150693	3976404 0,175683	4625917 0,175996	5128889 0,153723	18864684	41538
1000000	5133474 0,241701	8602321 0,349764	10714428 0,401664	12377194 0,418298	13753026 0,489796	50580443	78498
5000000	50580443 2.052.074	85560715 3.219.329	106934673 3,724226	123786966 3,939661	138415927 4,184647	505278724	348513
10000000	136141158 5,431349	230721639 8,136747	289072918 9,850464	335139680 10,304328	373838328 10,864564	1364913723	664579
50000000	1364913723 52,661844	2331746283 81,493298	2928753768 94,848466	3401482348 101,615093	3802982319 106,800216	13829878441	3001134
100000000	3696660006 145,329726	6330236116 223,60463	7960180688 258,416192	9251521971 278,125834	10351015320 292,294638	37589614101	5761455



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Γενική παραλληλοποίηση εργασιών – η οδηγία sections

Η οδηγία **sections**, επιτρέπει τον ορισμό ενός συνόλου εργασιών, εντελώς διαφορετικών και ασυσχέτιστων μεταξύ τους, η καθεμία των οποίων (η οποία χαρακτηρίζεται ως ενότητα κώδικα), εκτελείται από ένα και μοναδικό (οποιοδήποτε) νήμα της ομάδας για μία και μοναδική φορά (δηλαδή όχι επαναληπτικά, όπως στην περίπτωση ενός βρόχου).

Είναι πιο γενική από την οδηγία **for**, στην οποία η παραλληλοποίηση είναι εντελώς συμμετρική και επιτρέπει την ανάθεση σε κάθε νήμα ενός διαφορετικού προβλήματος προς εκτέλεση, υλοποιώντας τον παραλληλισμό δεδομένων.

Επιπλέον υποστηρίζεται η ταυτόχρονη εκτέλεση διαφορετικών μεταξύ τους διαδικασιών πάνω στο ίδιο ή σε διαφορετικά σύνολα δεδομένων υλοποιώντας τον παραλληλισμό διαδικασιών.

Οι διαδικασίες μέσα σε κάθε ενότητα που ορίζεται μέσω της οδηγίας **sections**, θα εκτελεστούν ακολουθιακά από το κάθε νήμα, αλλά όλα αυτά τα νήματα θα εκτελεστούν παράλληλα μεταξύ τους.

Δεν υφίστανται περιορισμοί όσον αφορά στην συσχέτιση ανάμεσα στο πλήθος των εργασιών και στο πλήθος των νημάτων

Εάν υπάρχουν περισσότερα νήματα, κάποια από αυτά θα παραμένουν ανενεργά, ενώ εάν υπάρχουν περισσότερες ενότητες, αυτές ανατίθενται στα νήματα ανάλογα με την διαθεσιμότητα των τελευταίων. Ο μηχανισμός χρονικής φραγής μπορεί να παρακαμφθεί με τον όρο **nowait**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Γενική παραλληλοποίηση εργασιών – η οδηγία sections

```
#include <omp.h>
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <time.h>

#define LENGTH 1000

double length (double * vector, int N) {
    int i; double temp = 0.0;
    for (i=0;i<N;i++) temp += pow(vector[i],2)
    return sqrt(temp); }

double min (double * vector, int N) {
    int i; double temp = vector[0];
    for (i=0;i<N;i++)
        if (vector[i]<temp) temp=vector[i];
    return temp; }

double max (double * vector, int N) {
    int i; double temp = vector[0];
    for (i=0;i<N;i++)
        if (vector[i]>temp) temp=vector[i];
    return temp; }

double average (double * vector, int N) {
    int i; double temp = 0.0;
    for (i=0;i<N;i++) temp += vector[i];
    return temp/N; }
```

```
void main (int argc, char ** argv) {
    long int i, threads;
    double len, minVal, maxVal, avg;
    double Vec[LENGTH];
    double t1,t2;
    if (argc==1) threads = omp_get_num_procs();
    else threads = strtol (argv[1],NULL,10);
    omp_set_num_threads (threads);
    printf ("Using %ld threads\n", threads);
    for (i=0;i<LENGTH;i++) {
        Vec[i]=(double)rand()/RAND_MAX;
        Vec[i]=(double)rand()/RAND_MAX; }

#pragma omp parallel sections shared(Vec)
{
    #pragma omp section /* this directive is optional */
    {
        len = length (Vec, LENGTH);
        printf ("Thread %d ==> Length is %f\n", omp_get_thread_num(), len); }
    #pragma omp section
    {
        minVal = min (Vec, LENGTH);
        printf ("Thread %d ==> Min is %f\n", omp_get_thread_num(), minVal); }
    #pragma omp section
    {
        maxVal = max (Vec, LENGTH);
        printf ("Thread %d ==> Max is %f\n", omp_get_thread_num(), maxVal); }
    #pragma omp section
    {
        avg = average (Vec, LENGTH);
        printf ("Thread %d ==> Average is %f\n", omp_get_thread_num(), avg); }}}}
```

```
amarg@amarg-vbox:~$ ./sect_example 4
Using 4 threads
Thread 3 ==> Length is 18.273937
Thread 0 ==> Min is 0.002209
Thread 1 ==> Max is 0.998142
Thread 2 ==> Average is 0.498577
amarg@amarg-vbox:~$ ./sect_example 3
Using 3 threads
Thread 1 ==> Length is 18.273937
Thread 0 ==> Min is 0.002209
Thread 2 ==> Average is 0.498577
Thread 1 ==> Max is 0.998142
amarg@amarg-vbox:~$ ./sect_example 2
Using 2 threads
Thread 1 ==> Length is 18.273937
Thread 1 ==> Max is 0.998142
Thread 1 ==> Average is 0.498577
Thread 0 ==> Min is 0.002209
amarg@amarg-vbox:~$
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Στην οδηγία **sections** ο ορισμός των **ενοτήτων εκτέλεσης** μιας **παράλληλης** εφαρμογής αποτελεί μία **στατική διαδικασία**, υπό την έννοια πως εάν το **πλήθος των νημάτων** είναι **μεγαλύτερο** από το **πλήθος των ενοτήτων εκτέλεσης**, τότε τα νήματα που **δεν συμμετέχουν** στην πραγματοποίηση υπολογισμών, **παραμένουν** αδρανή, αναμένοντας την **ολοκλήρωση** των υπολοίπων.

Επιπλέον αυτή η οδηγία **δεν** υποστηρίζει **μεταβλητό πλήθος ενοτήτων κώδικα** οι οποίες εκτελούνται **ταυτόχρονα**.

Η οδηγία **task** επιλύει τα παραπάνω προβλήματα, επιτρέποντας την **δυναμική δημιουργία και δρομολόγηση εργασιών** που χαρακτηρίζονται από **απουσία της παραμικρής δομής** και μάλιστα, εκτελούνται με **ασύγχρονο** τρόπο.

Μία **εργασία** ορίζεται ως ένα **στιγμιότυπο εκτελέσιμου κώδικα** που προορίζεται να εκτελεστεί **παράλληλα**, εφοδιασμένο με το **δικό του περιβάλλον δεδομένων** και το οποίο έχει συσχετιστεί με κάποιο **νήμα**, που θα αναλάβει την **εκτέλεση** του.

Κατά την **αυτόματη δημιουργία** μιας εργασίας από το περιβάλλον του OpenMP στα πλαίσια της εκτέλεσης ενός κώδικα που περιέχει μία οδηγία **parallel**, λαμβάνει χώρα μία διαδικασία **δύο** φάσεων:

- την **δημιουργία** της **δομής δεδομένων** που περιγράφει την **εργασία** ως **οντότητα** και την κατάλληλη **αρχικοποίηση** της
- την **εκχώρηση** μιας **εργασίας** σε ένα **νήμα** για **εκτέλεση**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

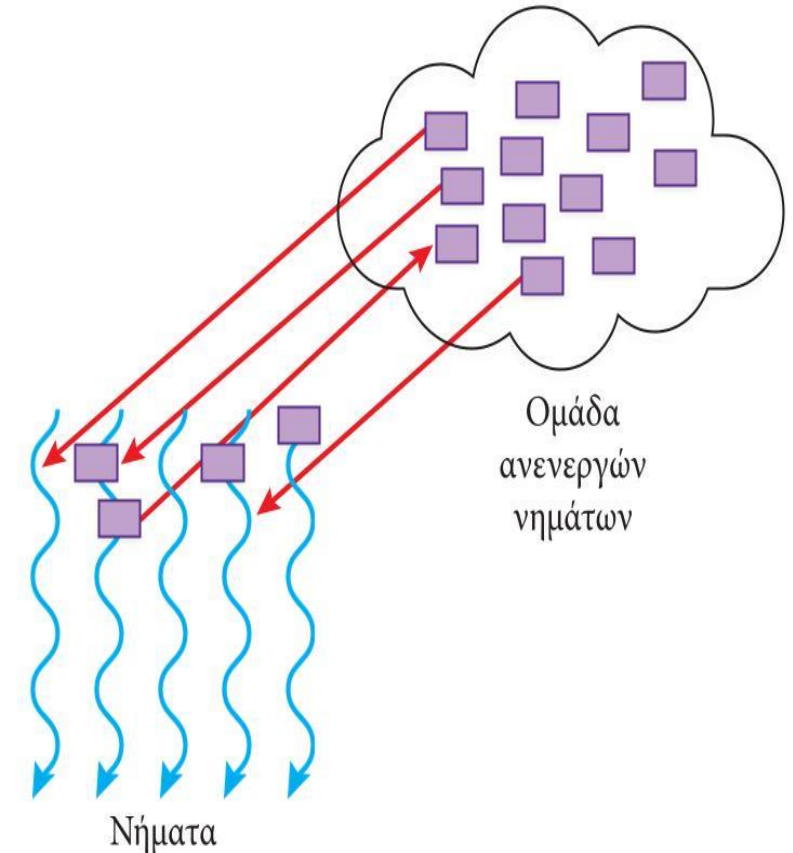
Αντίθετα, χρησιμοποιώντας την οδηγία **task** η οποία συντάσσεται με τη μορφή

```
#pragma omp task [optional clause(s)]
```

αυτές οι δύο φάσεις **αποσυνδέονται** και **διαχωρίζονται** μεταξύ τους υπό την έννοια πως η δεύτερη φάση **δεν είναι υποχρεωτικό** να εκτελεστεί **αμέσως μετά** την **ολοκλήρωση** της πρώτης φάσης και από το **ίδιο** νήμα.

Αντίθετα, αυτό είναι κάτι που μπορεί να γίνει **αργότερα** και μάλιστα, από **διαφορετικό** νήμα, το οποίο **αναστέλλει** την εργασία που εκτελεί σε **αυτή την χρονική στιγμή** για να **εξυπηρετήσει** την νέα εργασία.

Σε **πλήρη αναλογία** με την περίπτωση των **νημάτων**, οι εργασίες που **εμπλέκονται** στη διαδικασία, προσπελούνται μέσω μιας **ομάδας εργασιών (task pool)**, η οποία είναι παρόμοια με την **ομάδα νημάτων (thread pool)** που είχαμε παρουσιάσει στις **Διαφάνειες 39 και 40**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Σε αυτή τη **νέα προσέγγιση**, η οδηγία **parallel** εξακολουθεί να έχει ως αποτέλεσμα την **δημιουργία μιας ομάδας νημάτων** στα οποία **κατανέμεται** προς εκτέλεση **υπολογιστικό έργο**, με την **διαφορά** πως στην **προκειμένη περίπτωση**, ανάμεσα στο **νήμα** και στο **υπολογιστικό έργο**, παρεμβάλλεται η έννοια της **εργασίας (task)**, δηλαδή λέμε πως **το νήμα εκτελεί μία εργασία**.

Αυτές οι **εργασίες** επιτρέπουν την υλοποίηση **μη δομημένου παραλληλισμού**, καθώς και τον ορισμό **μονάδων υπολογιστικού έργου**, που **δημιουργούνται** και **εκτελούνται** με **δυναμικό** τρόπο.

Ενώ στις **αρχικές υλοποιήσεις** του OpenMP, το πλήθος των εργασιών **δημιουργούνταν** **ex αρχής** και ήταν **ίσο** με το **πλήθος των νημάτων**, πλέον υφίσταται η **δυνατότητα δημιουργίας** και ενός **μεταβλητού πλήθους εργασιών** από τον **προγραμματιστή**, οι οποίες γενικά μπορούν να **συσχετιστούν** με **διαφορετικά** νήματα.

Επιπλέον είναι **δυνατή** η εκτέλεση **δύο διαφορετικών τμημάτων** της **ίδιας εργασίας** από **διαφορετικά** νήματα

Η εκτέλεση της οδηγίας task **δεν μεταβάλλει το πλήθος των νημάτων** το οποίο παραμένει **σταθερό**, αλλά ωστόσο, το **πλήθος των εργασιών** που **δημιουργούνται** και **ανατίθενται** σε αυτά τα νήματα, μπορεί πλέον να **αυξομειώνεται δυναμικά**, με την εκτέλεσή τους είτε να πραγματοποιείται **αμέσως μετά** από την **δημιουργία** τους είτε να **αναβάλλεται** για αργότερα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Η δημιουργία πρόσθετου υπολογιστικού φόρτου με την οδηγία `task` πραγματοποιείται σε σημεία κώδικα στα οποία δεν υπάρχουν μηχανισμοί συγχρονισμού τύπου χρονικής φραγής.

Επιπλέον, μέσα στον κώδικα μιας οδηγίας `task`, μπορεί να χρησιμοποιηθεί ξανά αυτή η οδηγία με τις εργασίες που εμπλέκονται σε αυτή την διαδικασία, να χαρακτηρίζονται από μία ιεραρχική σχέση γονέα / παιδιού.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Αυτή η **ένθεση** των εργασιών, επιτρέπει τον **διαμοιρασμό** στις **θυγατρικές εργασίες**, μίας **μεταβλητής**, η οποία στην **γονική εργασία**, καθώς και στους **προγόνους** αυτής της εργασίας, ήταν **ιδιωτική**, αν και ασφαλώς θα πρέπει να **ληφθεί μέριμνα** προκειμένου να αποφευχθούν **συνθήκες ανταγωνισμού** ανάμεσα στις **εργασίες**.

Ο **χρόνος ζωής** των **εργασιών** που κατασκευάζονται μέσω της **task**, υπαγορεύεται από την **επομένη εντολή χρονικής φραγής** που θα συναντηθεί, αφού ο **ομώνυμος** μηχανισμός **συγχρονίζει νήματα** και **όχι εργασίες**.

Αντίθετα, ο μηχανισμός που **διασφαλίζει** πως τα νήματα θα προσεγγίσουν το σημείο της χρονικής φραγής **μόνο όταν ολοκληρώσουν όλο τον σχετιζόμενο με αυτά υπολογιστικό φόρτο**, είναι ο **χρονοπρογραμματιστής εργασιών (task scheduler)**, η λειτουργία του οποίου καθοδηγείται με την βοήθεια **ιεραρχικών συσχετίσεων** που ορίζονται ανάμεσα στις **αδελφικές** εργασίες.

Το **σημείο χρονοδρομολόγησης εργασιών** ορίζεται ως **το χρονικό σημείο στην εκτέλεση του κώδικα** στο οποίο λαμβάνει χώρα **εναλλαγή εργασίας**, είτε επειδή κάποια εργασία πρόκειται να **διακοπεί** είτε επειδή **ολοκληρώθηκε**. Αυτή η **εναλλαγή** συμβαίνει

- κάθε φορά που λαμβάνει χώρα η **δημιουργία** μίας **νέας εργασίας** με την οδηγία **task**, καθώς και **αμέσως μετά** από αυτή
- κάθε φορά που συναντάται μία **εντολή χρονικής φραγής**
- κατά την εκτέλεση μιας **οδηγίας target** αλλά και **αμέσως μετά** από αυτή
- κατά την χρήση των όρων **taskwait** και **taskgroup**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Η **οδηγία task** μπορεί να χρησιμοποιηθεί μαζί με τους επόμενους **όρους**:

affinity: επιτρέπει στον **προγραμματιστή** να ορίσει **πληροφορίες** σχετικά με την **θέση** και τον **τρόπο εκτέλεσης** των **εργασιών**.

depend (dependence_type:list): επιτρέπει την δήλωση **ιεραρχικών εξαρτήσεων** που υφίστανται ανάμεσα στις **εργασίες** της εφαρμογής και στις πάσης φύσεως **διαδικασίες**, προσφέροντας, ένα είδος **συγχρονισμού** μεταξύ των **εργασιών**.

detach: η **νέα εργασία** είναι **αποσπώμενη εργασία**, η οποία **ολοκληρώνεται** όταν έχει ολοκληρωθεί το **σχετιζόμενο** τμήμα **εκτελέσιμου κώδικα** και **ταυτόχρονα** έχει **εμφανιστεί** το κατάλληλο **συμβάν**

final (scalar-expression): εάν η τιμή της **scalar-expression** είναι **true**, τόσο η **θεωρούμενη** εργασία, όσο και οι **θυγατρικές** της εργασίες καθίστανται **τελικές διεργασίες** κάτι που σημαίνει πως **δεν είναι σε θέση** να δημιουργήσουν **νέες** εργασίες.

mergeable: υποδηλώνει πως η εργασία **διαμοιράζεται** το ίδιο **περιβάλλον δεδομένων** συμπεριλαμβανομένων και των **εσωτερικών μεταβλητών ελέγχου**, με την **εργασία** που την έχει **δημιουργήσει**

priority(i): Ορίζει για την **εργασία** μία **τιμή προτεραιότητας** ίση με **i**.

untied: η νέα εργασία **δεν σχετίζεται** με το νήμα που την δημιούργησε **από την αρχή μέχρι το τέλος της**, αλλά εάν η εκτέλεσή της **διακοπεί**, αυτή μπορεί να συνεχιστεί, όταν η εργασία επανέλθει σε **καθεστώς εκτέλεσης**, από ένα **άλλο νήμα**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία `task`

Η οδηγία `taskloop`, μπορεί να χρησιμοποιηθεί στην θέση της `parallel for` και επιτρέπει τον **διαμοιρασμό** των **επαναλήψεων** ενός **βρόχου** εκπεφρασμένου σε **κανονική** μορφή σε **εργασίες**, που δημιουργούνται από την `taskloop` ειδικά για αυτόν τον σκοπό. Ωστόσο, **δεν είναι δυνατή** η χρήση του όρου `schedule` για την **χρονοδρομολόγηση** του βρόχου.

Το **πλήθος των επαναλήψεων** που εκχωρούνται **στην κάθε εργασία**, υπολογίζεται **εκ των προτέρων**.

Ως **δυνατότητα** είναι **ιδιαίτερα χρήσιμη** σε περιπτώσεις στις οποίες το **κόστος εκτέλεσης** του **σώματος** του **βρόχου** μεταβάλλεται σε **σημαντικό βαθμό** ανάμεσα στις **επαναλήψεις**, καθώς και όταν οι εργασίες που δημιουργούνται, πρόκειται να **συνεργαστούν** με εργασίες που έχουν δημιουργηθεί **έξω από τον βρόχο**.

Ο όρος `grainsize` επιτρέπει τον ορισμό του **πλήθους** των **λογικών επαναλήψεων** που θα εκχωρηθούν στην **κάθε νέα εργασία**.

Η οδηγία χρησιμοποιείται **πάντοτε** μαζί με την `single` ή την `master` έτσι ώστε να εκτελεστεί **μόνο από ένα νήμα**.

Το **πλήθος των εργασιών** που θα δημιουργηθούν από την `taskloop`, προσδιορίζεται μέσω του όρου `num_tasks` ενώ η **κάθε εργασία** θα πρέπει να συσχετιστεί με **τουλάχιστον μία** επανάληψη



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Παράδειγμα χρήσης της οδηγίας `taskloop` του OpenMP για τον **πολλαπλασιασμό δισδιάστατων πινάκων**

```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <sys/time.h>

#define N 1000

double A[N][N], B[N][N], C[N][N];

int main (int argc, char ** argv) {
    int i,j,k,threads,chunk;
    if (argc==1) {
        threads = omp_get_num_procs();
        chunk = 1; }
    else {
        threads = strtol (argv[1],NULL,10);
        chunk = strtol (argv[2],NULL,10); }
    omp_set_num_threads(threads);
    printf ("Using %d threads, chunk size: %d\n",
            threads, chunk);
    struct timeval tv1, tv2;
    struct timezone tz;
    double elapsed;
```

```
    omp_set_num_threads (omp_get_num_procs());
    for (i=0;i<N;i++)
        for (j=0;j<N;j++) {
            A[i][j] = (double)rand()/RAND_MAX;
            B[i][j] = (double)rand()/RAND_MAX; }
    gettimeofday(&tv1, &tz);

    #pragma omp parallel private(i,j,k) shared(A,B,C)
    #pragma omp master
    #pragma omp taskloop grainsize(chunk)
    for (i=0;i<N;++i) {
        for (j=0;j<N;++j) {
            for (k=0;k<N;++k) {
                C[i][j]+=A[i][k]*B[k][j]; }}}
    gettimeofday(&tv2, &tz);
    elapsed = (double)(tv2.tv_sec-tv1.tv_sec)+
              (double)(tv2.tv_usec-tv1.tv_usec)*1.e-6;
    printf ("Elapsed time = %f seconds\n", elapsed); }
```

```
amarg@amarg-vbox:~$ ./tloop 10 10
Using 10 threads, chunk size: 10
Elapsed time = 4.830404 seconds
amarg@amarg-vbox:~$
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Συγχρονισμός εργασιών Πραγματεύεται το ζήτημα της **ενημέρωσης** της **γονικής εργασίας** πως οι **θυγατρικές εργασίες** βαίνουν προς τον **τερματισμό** τους και στηρίζεται στην χρήση **ειδικών οδηγιών**, όπως είναι οι οδηγίες **taskwait**, **taskgroup** και **taskyield**

Η **οδηγία taskwait** αποτελεί το **ισοδύναμο** της οδηγίας **barrier** για την περίπτωση των **εργασιών** αλλά εφαρμόζεται μόνο στην ομάδα των **θυγατρικών εργασιών** μιας εργασίας: εάν αυτές οι **θυγατρικές εργασίες** έχουν δημιουργήσει **τις δικές τους θυγατρικές εργασίες**, αυτές **δεν εμπλέκονται** σε αυτή την διαδικασία.

Εάν επιθυμούμε να εφαρμόσουμε αυτή την **διαδικασία συγχρονισμού** στο **σύνολο των εργασιών** της **δεντρικής δομής**, που ως **ρίζα**, έχει την **θεωρούμενη εργασία**, θα πρέπει να χρησιμοποιήσουμε την **οδηγία taskgroup**

Η οδηγία **taskgroup** επιτρέπει την **υλοποίηση υποβιβασμού δεδομένων** μέσω των όρων **task_reduction** και **in_reduction**.

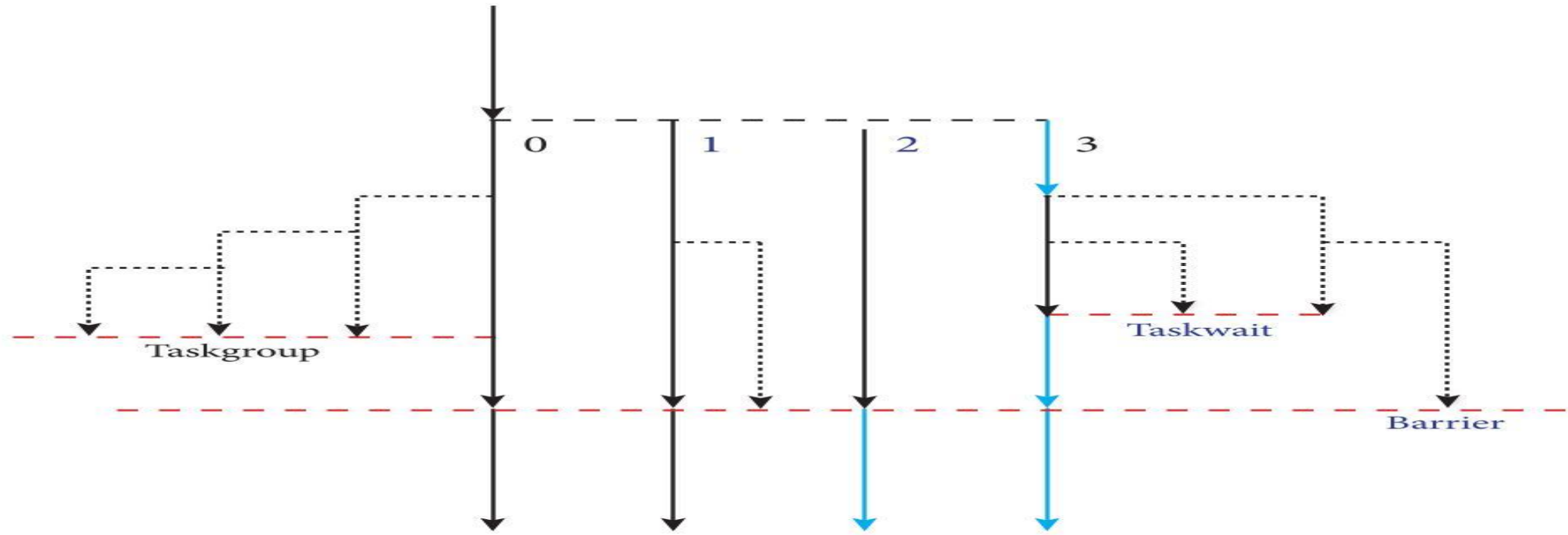
Η **οδηγία taskyield** επιτρέπει τον ορισμό ενός **σημείου συγχρονισμού διεργασιών** ορισμένο από τον χρήστη στο οποίο, η **τρέχουσα εργασία** μπορεί να **αναστείλει** την λειτουργία της προκειμένου το νήμα που την εκτελεί να αναλάβει την **εκτέλεση** κάποιας **άλλης** εργασίας. Αυτή η δυνατότητα επιτρέπει την **βελτιστοποίηση** της **διαδικασίας διαχείρισης** εργασιών.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task



Οι τρεις μηχανισμοί συγχρονισμού εργασιών στο OpenMP (taskwait, taskgroup και barrier).

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

Παράδειγμα χρήσης των όρων **task_reduction** και **in_reduction**, για τον **παράλληλο υπολογισμό** του **αθροίσματος** των τιμών που είναι αποθηκευμένοι στους **κόμβους** μιας **απλής συνδεδεμένης λίστας** με τη βοήθεια μιας **ομάδας νημάτων**.

```
#include<stdlib.h>
#include<stdio.h>

#define N 10

typedef struct node {
    int val;
    struct node *next; } node_t;

/* This function gets as argument the head of the list */
/* and returns the sum of the stored values */

int listSum (node_t *p) {
    int res = 0;
    #pragma omp taskgroup task_reduction(+: res)
    {
        node_t* tmp = p;
        while (tmp!=0) {
            #pragma omp task in_reduction(+: res)
            res += tmp->val;
            tmp = tmp->next; }
    return res; }
```

```
int main(int argc, char *argv[]) {
    int i;
    /* A linked list is dynamically created in this point */
    node_t * root = (node_t*) malloc (sizeof(node_t));
    root->val = 1; node_t* tmp = root;
    for(i=2;i<=N;++i){
        tmp->next = (node_t*) malloc(sizeof(node_t));
        tmp = tmp->next;
        tmp->val = i; }
    tmp->next = 0;
    /* The sum of the list data is created in parallel */
    #pragma omp parallel
    #pragma omp single
    {
        int result = listSum (root);
        printf("The value of sum is %d\n", result); }
    /* The list must be freed before return */
    return 0; }
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Από τα νήματα στις εργασίες – η οδηγία task

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp task
        {
            #pragma omp critical
            printf ("Task 1\n");
            #pragma omp task
            {
                sleep(1);
                #pragma omp critical
                printf ("Task 2\n");
            }
        }
        #pragma omp taskwait
        #pragma omp task
        {
            #pragma omp critical
            printf ("Task 3\n");
        }
    }
    return 0;
}
```

```
amarg@amarg-vbox:~$ ./taskwait
Task 1
Task 3
Task 2
amarg@amarg-vbox:~$
```

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp taskgroup
        {
            #pragma omp task
            {
                #pragma omp critical
                printf ("Task 1\n");
                #pragma omp task
                {
                    sleep(1);
                    #pragma omp critical
                    printf ("Task 2\n");
                }
            }
            #pragma omp task
            {
                #pragma omp critical
                printf ("Task 3\n");
            }
        }
    }
    return 0;
}
```

```
amarg@amarg-vbox:~$ ./taskgroup
Task 1
Task 2
Task 3
amarg@amarg-vbox:~$
```

Παραδείγματα χρήσης των οδηγιών **taskwait** και **taskgroup** που επιτρέπουν τον **συγχρονισμό εργασιών** στο **OpenMP**.

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Απότομος τερματισμός της εκτέλεσης νήματος – η οδηγία `cancel`

Ο **απότομος τερματισμός** της λειτουργίας των **νημάτων** στο **OpenMP** όπως είναι για παράδειγμα ο **τερματισμός** ενός βρόχου λόγω της εντολής **`break`**, σχετίζεται με ερωτήματα όπως:

Με ποιον τρόπο θα συμπεριφερθούν τα **υπόλοιπα νήματα**, όταν κάποιο από τα νήματα της εφαρμογής **διακόψει** την εκτέλεση του τμήματος του βρόχου; Θα εγκαταλείψουν **και αυτά** την εκτέλεση του **δικού τους τμήματος** του βρόχου, ή θα **συνεχίσουν** και θα την **ολοκληρώσουν** κανονικά;

Αμφότερες οι δυνατότητες **δεν θεωρούνται ικανοποιητικές**, αφού οδηγούν σε **ασάφεια** και σε **απρόβλεπτη** συμπεριφορά.

Αυτή η **δυνατότητα ακύρωσης** είναι **χρήσιμη** και σε διαδικασίες **παράλληλης αναζήτησης** σε **αρχείο** ή **βάση δεδομένων**, με το κάθε νήμα να πραγματοποιεί την αναζήτηση σε ένα **διαφορετικό τμήμα** του αρχείου.

Από την στιγμή που κάποιο νήμα εντοπίσει το **ζητούμενο**, τα **υπόλοιπα νήματα** θα πρέπει να έχουν τη δυνατότητα να **εγκαταλείψουν άμεσα την αναζήτηση** και να **τερματίσουν** την λειτουργία τους, αφού η πληροφορία **έχει βρεθεί**.

Η **ακύρωση της εκτέλεσης** ενός **παράλληλου** κώδικα, πραγματοποιείται με την **οδηγία `cancel`**, που καλείται ως

```
#pragma omp cancel construct [ if ( [cancel: ] expr )]
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Απότομος τερματισμός της εκτέλεσης νήματος – η οδηγία `cancel`

όπου `construct` είναι κάποια από τις οδηγίες `parallel`, `for`, `sections` ή `taskgroup`, ενώ το προαιρετικό όρισμα `if ( [ cancel: ] expr )` επιτρέπει την εκτέλεση της εντολής ακύρωσης υπό συνθήκη.

Για να διαπιστωθεί εάν έχει λάβει χώρα η ενεργοποίηση της διαδικασίας ακύρωσης, ακριβώς κάτω από την παραπάνω οδηγία χρησιμοποιούμε την οδηγία

```
#pragma omp cancellation point construct
```

Η χρήση αυτής της οδηγίας, δεν είναι υποχρεωτική, αφού η ενεργοποίηση μιας διαδικασίας ακύρωσης μπορεί να ανιχνευθεί αυτόματα από ένα νήμα όταν συναντάται μία εντολή συγχρονισμού χρονικής φραγής.

Στην περίπτωση κατά την οποία ένα νήμα ανιχνεύσει ένα σημείο ακύρωσης της διαδικασίας που εκτελείται, μεταφέρει αυτόματα τον έλεγχο αμέσως μετά από το πέρας της σχετιζόμενης περιοχής παράλληλου κώδικα, η εκτέλεση του οποίου ακυρώνεται, έτσι ώστε η εκτέλεση της εφαρμογής να συνεχίσει από το σημείο αυτό και μετά.

Στην πραγματικότητα, η λειτουργία που ακυρώνεται, δεν αφορά στο νήμα αλλά στην εργασία που εκτελείται από αυτό.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Απότομος τερματισμός της εκτέλεσης νήματος – η οδηγία `cancel`

Παράδειγμα χρήσης της οδηγίας `cancel`

```
#include <stddef.h>
#include <omp.h>

typedef struct btree_s {
    int val;
    struct btree_s *left;
    struct btree_s *right; } btree_t;

btree_t *search_parallel (btree_t *tree, int val) {
    btree_t *found = NULL;
    #pragma omp parallel shared(found, tree, val)
    {
        #pragma omp masked
        {
            #pragma omp taskgroup
            { found = search(tree, val, 0); }}}
    return found; }
```

Παράλληλη αναζήτηση μιας αριθμητικής τιμής από μία ομάδα εργασιών σε μία αναδρομική δομή δυαδικού δέντρου

```
btree_t *search (btree_t *tree, int val, int level) {
    btree_t *found = NULL;
    if (tree) {
        if (tree->val == val) found = tree;
        else {
            #pragma omp task shared(found) if(level < 10)
            {
                btree_t *found_left = NULL;
                found_left = search (tree->left, val, level+1);
                if (found_left) {
                    #pragma omp atomic write
                    found = found_left;
                    #pragma omp cancel taskgroup
                }
            }
            #pragma omp task shared(found) if(level < 10)
            {
                btree_t *found_right = NULL;
                found_right = search (tree->right, val, level+1);
                if (found_right) {
                    #pragma omp atomic write
                    found = found_right;
                    #pragma omp cancel taskgroup
                }
            }
            #pragma omp taskwait
        }
        return found; }
```

Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάθεση υπολογισμών σε απομακρυσμένες διατάξεις

Το OpenMP υποστηρίζει την **ανάθεση υπολογιστικού έργου** σε **απομακρυσμένες διατάξεις**, όπως είναι οι **μονάδες GPU**.

Κατά συνέπεια, αποτελεί ένα **ετερογενές περιβάλλον**, αφού τα νήματά του εκτελούνται σε **υπολογιστικούς κόμβους** που χαρακτηρίζονται από **διαφορετικές αρχιτεκτονικές**.

Η συσκευή στην οποία δημιουργείται το **κεντρικό νήμα**, χαρακτηρίζεται ως **μητρική συσκευή**, ενώ οι διατάξεις στις οποίες το OpenMP εκχωρεί **υπολογιστικό έργο για εκτέλεση**, χαρακτηρίζονται ως **συσκευές / στόχοι**.

Η **κάθε** διάταξη διαθέτει **το δικό της περιβάλλον δεδομένων**, ενώ μία **μεταβλητή** μπορεί να διαθέτει **πολλαπλά αντίγραφα** στις μνήμες όλων αυτών των διατάξεων, με την **συνέπεια δεδομένων** να διασφαλίζεται από τον **προγραμματιστή**.

```
int omp_get_num_devices ();           /* Returns the number of the available devices */
int omp_get_default_device ();        /* Return the ID of the default device */
void omp_set_default_device (int ID) /* Sets the default device */
int omp_get_device_num ();           /* Returns the ID of the device executing the current thread */
int omp_get_initial_device ();       /* Return the ID for the host device */
```

Οι συναρτήσεις **διαχείρισης των συσκευών** στο περιβάλλον του **OpenMP**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάθεση υπολογισμών σε απομακρυσμένες διατάξεις

Η εκτέλεση κώδικα σε απομακρυσμένη συσκευή γίνεται με την οδηγία **target**, η οποία συντάσσεται ως

```
#pragma omp target
```

και συνοδεύεται από τους προαιρετικούς όρους **device**, **map** και **if**. Αυτή η οδηγία επιτρέπει την εκτέλεση του κώδικα στην απομακρυσμένη συσκευή και την ανταλλαγή δεδομένων ανάμεσα στην μνήμη του επεξεργαστή και στην μνήμη της συσκευής.

Προκειμένου η εφαρμογή να εκτελεστεί παράλληλα από μία ομάδα νημάτων, θα πρέπει να καταφύγουμε στην χρήση της οδηγίας **teams** η οποία είναι η αντίστοιχη της **parallel**, υλοποιεί τον παραλληλισμό στην συσκευή – στόχο και συντάσσεται ως

```
#pragma omp teams
```

Κάθε φορά που ένα νήμα συναντά την οδηγία **teams**, δημιουργεί ένα σύνολο από ομάδες νημάτων. Η καθεμία από αυτές τις ομάδες αποτελεί μία αρχική ομάδα, με το αρχικό νήμα στην καθεμία από αυτές τις ομάδες, να εκτελεί την οδηγία **teams**.

Το πλήθος των νέων ομάδων υπαγορεύεται από το αποτέλεσμα της αποτίμησης των όρων **if** και **num_teams** ή μέσω της

```
void omp_set_num_teams (int num_teams);
```

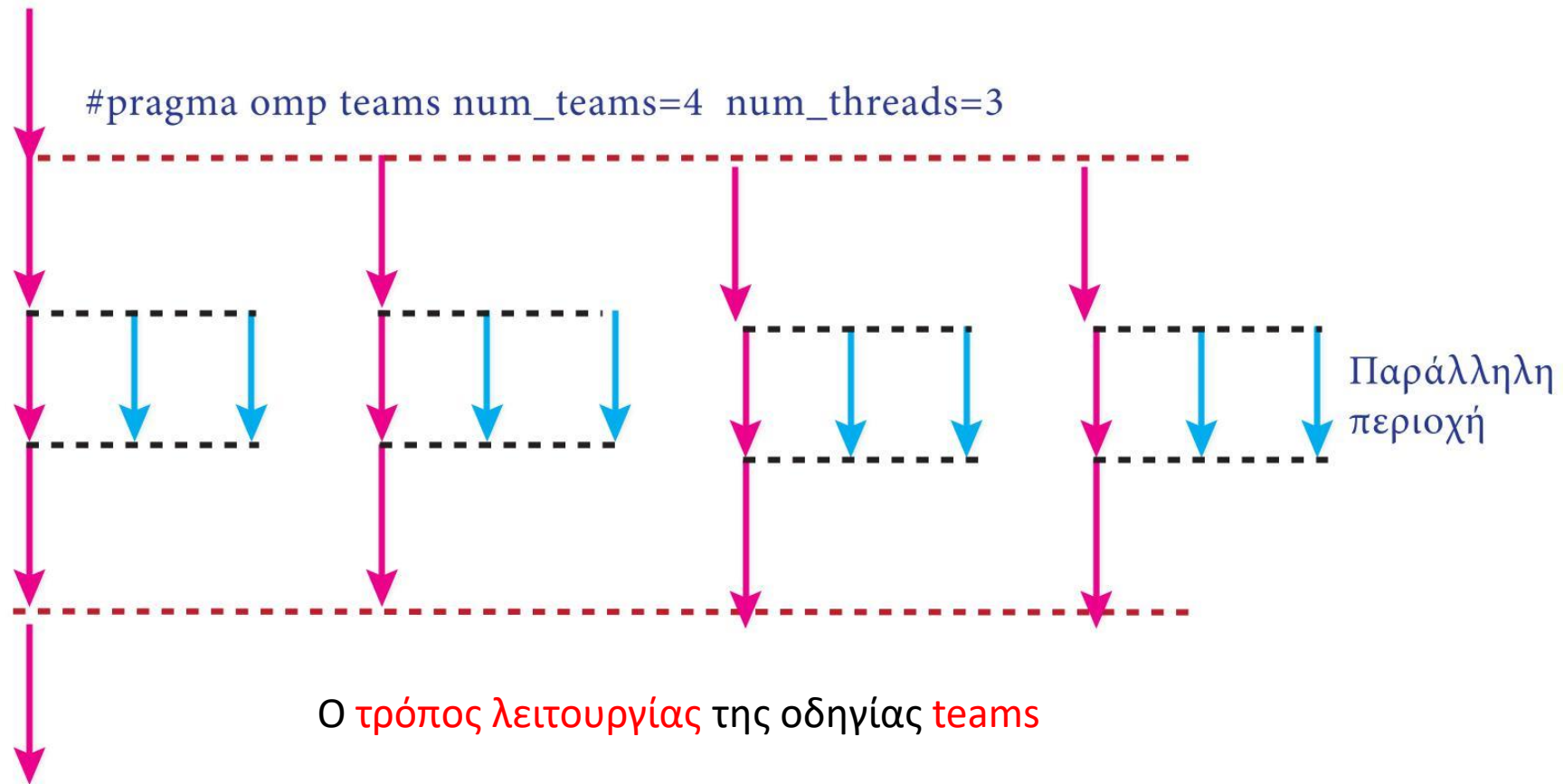
η οποία λειτουργεί όπως η συνάρτηση **omp_set_num_threads** που σχετίζεται με την οδηγία **parallel**



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάθεση υπολογισμών σε απομακρυσμένες διατάξεις



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάθεση υπολογισμών σε απομακρυσμένες διατάξεις

Προκειμένου η εφαρμογή να λειτουργήσει **αποδοτικά**, θα πρέπει να **αναθέσουμε** στην **κάθε ομάδα** νημάτων **διαφορετικό υπολογιστικό έργο** και να **διασφαλίσουμε**, πως **όλα τα νήματα** της κάθε ομάδας θα συμμετέχουν στους υπολογισμούς.

Αν και μπορούμε να **ταυτοποιήσουμε** το **κάθε νήμα** της κάθε ομάδας και να αναθέσουμε σε αυτό το **κατάλληλο έργο** ωστόσο, μπορούμε να **ζητήσουμε από το OpenMP** να πραγματοποιήσει αυτή τη **διαδικασία** με την **οδηγία distribute** που καλείται ως

```
#pragma omp target teams distribute parallel for [parallel code]
```

Στην περίπτωση αυτή, ο κώδικας που βρίσκεται **ανάμεσα** στις οδηγίες **teams** και **distribute** θα εκτελεστεί **από όλα τα νήματα της αρχικής ομάδας**. Ειδικότερα, οι επαναλήψεις των **εμπλεκόμενων** βρόχων θα διανεμηθούν στα **αρχικά νήματα** όλων των **αρχικών ομάδων** που εκτελούν τον κώδικα της **οδηγίας teams** με την οποία σχετίζεται η **οδηγία distribute**.

Οι οδηγίες **target** και **teams**, μπορούν **αμφότερες** να χρησιμοποιηθούν με τον όρο **reduce**, αν και σε αυτή την **διαδικασία μείωσης δεδομένων**, θα πρέπει να χρησιμοποιηθούν **μόνο** νήματα που ανήκουν **στην ίδια ομάδα** αφού στην **αντίθετη περίπτωση**, η συμπεριφορά της εφαρμογής **ενδέχεται να είναι προβληματική**, οδηγώντας στην εμφάνιση **σφαλμάτων**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Ανάθεση υπολογισμών σε απομακρυσμένες διατάξεις

Σε περιπτώσεις στις οποίες επιθυμούμε να αναθέσουμε **υπολογιστικό έργο** σε μία **απομακρυσμένη συσκευή**, μόνο **εφόσον ικανοποιείται** κάποια **συνθήκη**, θα πρέπει να χρησιμοποιήσουμε στην οδηγία **target** τον όρο **if**, μέσω μίας δήλωσης της μορφής

```
#pragma omp target if (boolean-expr)
```

Εάν **δεν εμφανίζεται** αυτός ο όρος ή διαπιστωθεί πως **δεν υπάρχουν απομακρυσμένες συσκευές**, ο κώδικας της εφαρμογής θα εκτελεστεί στην **μητρική συσκευή**, ενώ το ίδιο θα συμβεί και εάν η **συνθήκη** του **όρου if** αποτιμηθεί ως **ψευδής**

Η **εγκατάσταση** ενός **περιβάλλοντος δεδομένων** στη μνήμη της **απομακρυσμένης διάταξης**, πραγματοποιείται με την οδηγία

```
#pragma omp target data
```

η οποία θα πρέπει να **περικλείει** οποιαδήποτε άλλη δήλωση τύπου **target**. Εάν μέσα σε αυτή την οδηγία δημιουργήσουμε **ομάδες νημάτων** καλώντας την οδηγία **teams**, αυτές οι **ομάδες θα διαμοιράζονται το ίδιο περιβάλλον δεδομένων** και ως εκ τούτου, **δεν απαιτείται** ανάγκη **συγχρονισμού**.

Εάν **απαιτείται συγχρονισμός** ανάμεσα στα **περιβάλλοντα δεδομένων** της **μητρικής** και της **απομακρυσμένης** συσκευής, ο **τρόπος πραγματοποίησης του συγχρονισμού** και η **χρονική στιγμή** κατά την οποία αυτός θα **πραγματοποιηθεί**, ορίζονται χρησιμοποιώντας μαζί με την παραπάνω οδηγία, **έναν ή περισσότερους όρους map** (για λεπτομέρειες ανατρέξτε στη **σελ. 562**)



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Απεικόνιση νημάτων σε στοιχεία επεξεργασίας

Ορίζουμε ως **τοποθεσία** ή **θέση** ένα **μη διατεταγμένο σύνολο** από **στοιχεία επεξεργασίας**, τα οποία περιλαμβάνονται σε μία **συσκευή**, όπως είναι το σύνολο των **πυρήνων** και των **νημάτων σε επίπεδο υλικού** ενός **πολυπύρηνου επεξεργαστή**.

Ωστόσο, ως **θέση** μπορούμε να ορίσουμε έναν **ολόκληρο επεξεργαστή** ή ακόμη και μία **ομάδα** τέτοιων επεξεργαστών.

Προκειμένου να **ανακτήσουμε** το **πλήθος των θέσεων** που είναι **διαθέσιμες** για την **εκτέλεση νημάτων**, καθώς και το **πλήθος των στοιχείων επεξεργασίας** για την **κάθε** θέση, χρησιμοποιούμε **αντίστοιχα** τις συναρτήσεις

```
int omp_get_num_places (void);    και    int omp_get_place_num_procs (int place_num);
```

ενώ προκειμένου να **ανακτήσουμε** τα **αναγνωριστικά** αυτών **των στοιχείων επεξεργασίας**, χρησιμοποιούμε την συνάρτηση

```
void omp_get_place_proc_ids (int place_num, int *ids);
```

Κατά την **εκκίνηση** του OpenMP, λαμβάνει χώρα η **αυτόματη ανάκτηση** αυτών των πληροφοριών, έτσι ώστε το περιβάλλον να γνωρίζει τις **δυνατότητες** του **υποκειμένου υπολογιστικού συστήματος**, όσον αφορά στην **εκτέλεση** των **νημάτων**.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Απεικόνιση νημάτων σε στοιχεία επεξεργασίας

Η **συσχέτιση** ενός **νήματος προς εκτέλεση** με μία **θέση** και πιο συγκεκριμένα, η **απεικόνιση** του νήματος σε κάποιο **στοιχείο επεξεργασίας** αυτής της θέσης, αποτελεί παράγοντα με **κρίσιμη σημασία** για την **αποδοτικότητα** της εφαρμογής, όσον αφορά στο χαρακτηριστικό της **προσπέλασης μνήμης**.

Ο ορισμός μιας **σταθερής** και **αμετάβλητης απεικόνισης** νημάτων σε επεξεργαστές, οδηγεί σε **καλύτερο βαθμό χρήσης της κρυφής μνήμης**: εάν κάποιο νήμα **αποσπαστεί** από τον επεξεργαστή, προκειμένου αυτός να εκτελέσει **κάποιο άλλο νήμα**, τα δεδομένα του είναι πιθανό **να διατηρηθούν στην κρυφή μνήμη**, έτσι ώστε να **χρησιμοποιηθούν ξανά** στο μέλλον.

Η απεικόνιση των νημάτων στα στοιχεία επεξεργασίας, είναι **ιδιαίτερα σημαντική** στις **αρχιτεκτονικές NUMA** (Ενότητα 1-3.8, σελίδα 47), στις οποίες το **κόστος προσπέλασης** της μνήμης **δεν είναι το ίδιο** για όλους τους επεξεργαστές του συστήματος.

Μία σημαντική διαδικασία των **συστημάτων εικονικής μνήμης**, είναι η **αντιστοίχιση λογικών διευθύνσεων σε φυσικές διευθύνσεις**, με τις τιμές των **παραμέτρων** της διαδικασίας να **επιλέγονται** κατά τον **χρόνο εκτέλεσης** της εφαρμογής.

Προκειμένου να **αυξηθεί η απόδοση** που σχετίζεται με την **προσπέλαση** αυτών των σελίδων μνήμης, είναι εύλογο για το λειτουργικό σύστημα, να αντιστοιχήσει μία **λογική σελίδα μνήμης** σε μία **φυσική σελίδα**, η οποία βρίσκεται **κοντά στον πυρήνα** που προσπέλασε για **πρώτη φορά** το περιεχόμενο αυτής της σελίδας



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι πολιτικές ανάθεσης νημάτων σε επεξεργαστές

Οι θέσεις εκτέλεσης των νημάτων του OpenMP, καθορίζονται μέσω της μεταβλητής περιβάλλοντος **OMP_PLACES**.

Στην περίπτωση αρχιτεκτονικής NUMA με M πολυπύρηνους επεξεργαστές, ο καθένας των οποίων περιέχει την δική του μνήμη και N πυρήνες με L νήματα σε επίπεδο υλικού ο καθένας, η μεταβλητή **OMP_PLACES**, αρχικοποιείται ως εξής:

- Ορίζουμε μία λίστα πυρήνων ή νημάτων σε επίπεδο υλικού.
- Χρησιμοποιούμε ένα συμβολικό όνομα που μπορεί να λάβει κάποια από τις τιμές
 1. **threads** (η κάθε θέση αντιστοιχεί σε ένα νήμα σε επίπεδο υλικού)
 2. **cores** (η κάθε θέση αντιστοιχεί σε έναν πυρήνα)
 3. **sockets** (η κάθε θέση αντιστοιχεί σε έναν πολυπύρρηνο επεξεργαστή)
 4. **ll_caches** (η κάθε θέση αντιστοιχεί σε ένα σύνολο πυρήνων που διαμοιράζονται την κρυφή μνήμη του τελευταίου επιπέδου - εάν αυτή η μνήμη διαμοιράζεται σε όλους τους πυρήνες, η λίστα των θέσεων είναι ίδια με πριν)
 5. **numa_domains** (η κάθε θέση ορίζεται από όλους τους πυρήνες που διαμοιράζονται την πιο κοντινή ως προς αυτούς μνήμη συστήματος, από την οποία όλοι οι πυρήνες θα πρέπει να ισαπέχουν).



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι πολιτικές ανάθεσης νημάτων σε επεξεργαστές

Η **απεικόνιση** των νημάτων στα **στοιχεία επεξεργασίας**, πραγματοποιείται με κάποιον από τους **επόμενους** τρόπους:

- Χρησιμοποιούμε στην οδηγία **parallel** τον όρο **proc_bind**. Αυτός ο όρος δέχεται ως όρισμα, μία εκ των επόμενων **τριών συμβολοσειρών**, οι οποίες ορίζουν **ισάριθμες πολιτικές απεικόνισης** και πιο συγκεκριμένα:
 - primary** Όλα τα νήματα σε μία **ομάδα νημάτων**, ανατίθενται για εκτέλεση **στην ίδια θέση** στην οποία έχει ανατεθεί και το **κύριο νήμα της ομάδας**, κάτι που **συνιστάται**, εάν απαιτείται **γρήγορη** πρόσβαση σε **τοπικά** δεδομένα.
 - close** Τα νήματα της ομάδας ανατίθενται στις θέσεις με τέτοιο τρόπο, ώστε να βρίσκονται **όσο το δυνατόν πιο κοντά** στη θέση στην οποία εκτελείται το **κύριο** νήμα της ομάδας.
 - spread** Τα T νήματα της ομάδας **διασκορπίζονται** στις P **διαθέσιμες θέσεις** της **υφιστάμενης διαμέρισης**, όσο πιο **μακριά** γίνεται το ένα από το άλλο.
- Αρχικοποιούμε με τον **κατάλληλο τρόπο** τη μεταβλητή περιβάλλοντος **OMP_PROC_BIND** η οποία αντιστοιχεί στην **εσωτερική μεταβλητή ελέγχου** **bind-var**, σε κάποια από τις παραπάνω τιμές.

Από τη στιγμή που ένα **νήμα** της **ομάδας νημάτων**, εκχωρηθεί σε μία **θέση**, η **υποκείμενη υλοποίηση** του OpenMP **δεν θα πρέπει να το μετακινήσει** σε μία **άλλη** θέση.

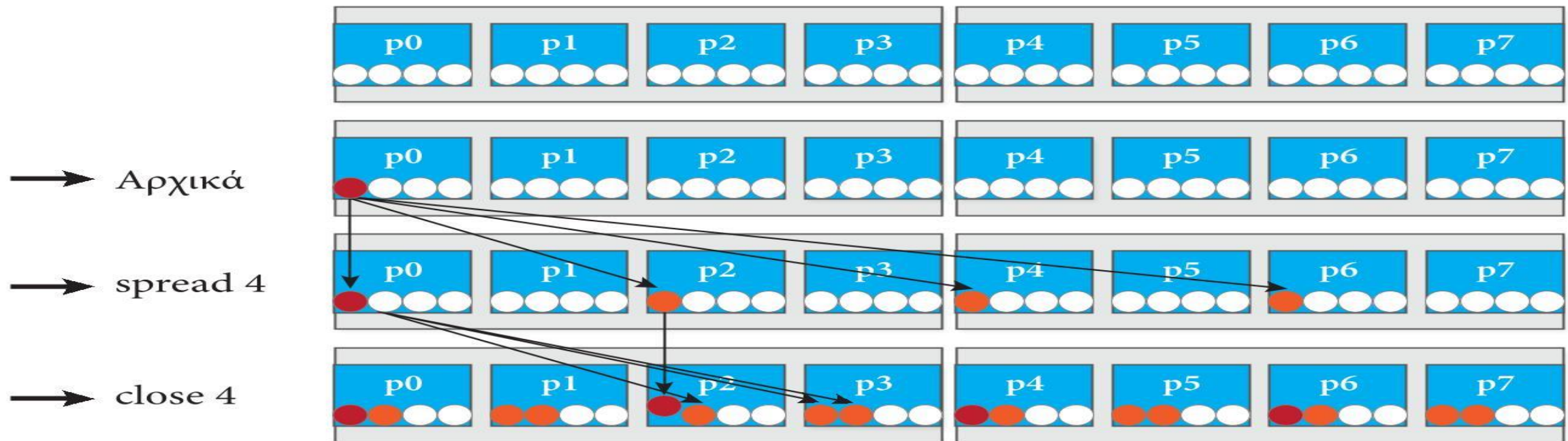


Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι πολιτικές ανάθεσης νημάτων σε επεξεργαστές

Παράδειγμα εφαρμογής των πολιτικών **primary**, **close** και **thread** που ορίζονται δια της χρήσεως του όρου **proc_bind** για την περίπτωση ενός **παράλληλου υπολογιστή** που περιέχει **δύο πολυπύρηνους επεξεργαστές** με **τέσσερις πυρήνες** ανά επεξεργαστή και με **τέσσερα νήματα** σε επίπεδο υλικού ανά πυρήνα.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η διαχείριση της μνήμης στο OpenMP

Στο **OpenMP** χρησιμοποιούμε τον όρο **χώρο μνήμης**, προκειμένου να προσδιορίσουμε ένα **σύνολο αποθηκευτικών πόρων** με **διαφορετικές γενικά ιδιότητες** και **χαρακτηριστικά**, που χρησιμοποιούνται για την **αποθήκευση** πάσης φύσεως **δεδομένων**.

Σε κάθε **συσκευή** που χρησιμοποιείται για την εκτέλεση **υπολογιστικού έργου**, θα πρέπει να υφίσταται **τουλάχιστον ένας** τέτοιος **χώρος αποθήκευσης**, στον οποίο **καταφεύγουμε** για να **δεσμεύσουμε** και να **αποδεσμεύσουμε** χώρο μνήμης.

Στο περιβάλλον του **OpenMP** ορίζονται οι επόμενοι **χώροι μνήμης**:

- **omp_default_mem_space**. Αναπαριστά τον **προεπιλεγμένο** χώρο αποθήκευσης του συστήματος.
- **omp_large_cap_mem_space**. Αναπαριστά χώρο μνήμης με **μεγάλη χωρητικότητα**.
- **omp_const_mem_space**. Αναπαριστά χώρο μνήμης που έχει **βελτιστοποιηθεί** για την αποθήκευση **σταθερών τιμών**.
- **omp_high_bw_mem_space**. Αναπαριστά χώρο μνήμης με **μεγάλο εύρος ζώνης**.
- **omp_low_lat_mem_space**. Αναπαριστά χώρο μνήμης με **μικρό χρόνο προσπέλασης**.

Η **κατανομή της μνήμης** πραγματοποιείται από **ειδικούς μηχανισμούς** οι οποίοι όταν δεχθούν ένα **αίτημα ανάθεσης μνήμης** προχωρούν στην **παραχώρηση** μιας **λογικά συνεχούς περιοχής μνήμης** στο νήμα που την έχει ζητήσει, η οποία ανήκει στον **σχετιζόμενο** χώρο μνήμης και το μέγεθος της οποίας είναι **τουλάχιστον ίσο** με το μέγεθος που έχει ζητηθεί.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Η διαχείριση της μνήμης στο OpenMP

Ο προσδιορισμός της συμπεριφοράς της εφαρμογής όσον αφορά στα αιτήματα ανάθεσης μνήμης από τα νήματα που την απαρτίζουν, απαιτεί την αρχικοποίηση ενός ή περισσότερων από τα επόμενα χαρακτηριστικά:

- **sync_hint** Περιγράφει τον τρόπο υποβολής αιτημάτων ανάθεσης μνήμης από την ομάδα νημάτων της εφαρμογής
- **alignment** Επιτρέπει τον καθορισμό της τιμής που χρησιμοποιείται στην ευθυγράμμισης μνήμης
- **access** Προσδιορίζει την ομάδα των νημάτων που δικαιούνται να προσπελάσουν την περιοχή μνήμης που έχει διατεθεί
- **pool_size** Προσδιορίζει τον μέγιστο χώρο μνήμης σε bytes που μπορεί να ανατεθεί στα νήματα της εφαρμογής.
- **fallback** Προσδιορίζει τον τρόπο συμπεριφοράς του μηχανισμού ανάθεσης όταν μπορεί να ικανοποιήσει ένα αίτημα
- **pinned** Προσδιορίζει εάν οι εκχωρήσεις μνήμης που έχουν πραγματοποιηθεί, σχετίζονται για όλο το χρονικό διάστημα της ζωής τους, με τον ίδιο αποθηκευτικό χώρο και με την ίδια θέση μνήμης ή όχι
- **partition** Περιγράφει τον τρόπο με τον οποίο η μνήμη που έχει ανατεθεί στα νήματα της εφαρμογής, διαμερίζεται σε όλους τους αποθηκευτικούς χώρους που συνιστούν τον χώρο μνήμης που χρησιμοποιείται από τον μηχανισμό ανάθεσης.



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Υλοποίηση διανυσματικών υπολογισμών – η οδηγία `simd`

Η οδηγία `simd` εκμεταλλεύεται τα προηγμένα χαρακτηριστικά των ομώνυμων αρχιτεκτονικών όπως είναι το σύνολο εντολών **AVX (Advanced vector eXtensions)** της αρχιτεκτονικής **x86-64** ή οι διανυσματικές εντολές των **128 bit** της αρχιτεκτονικής **ARM**.

Μία πολύ γνωστή εφαρμογή αυτής της διαδικασίας, είναι η χρήση της στην εκτέλεση επαναληπτικών βρόχων, ενώ ως διαδικασία παραλληλοποίησης χρησιμοποιείται από κοινού με τη χρήση πολλών νημάτων. Η οδηγία συντάσσεται ως

```
#pragma omp simd [ optional clause(s) ]  
loop source code
```

Παράδειγμα κώδικα πριν και μετά την εφαρμογή της οδηγίας `simd` ακολουθεί στη συνέχεια

```
#define N 100  
double A[N], B[N], C[N];  
initialization of A and B arrays  
#pragma omp parallel for  
#pragma omp simd  
for (int i=0;i<N;i++) C[i]=A[i]+B[i];
```

```
#define N 100  
double A[N], B[N], C[N];  
initialization of A and B arrays  
#pragma omp parallel for simd  
for (int i=0;i<N;i++) C[i]=A[i]+B[i];
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι συναρτήσεις της βιβλιοθήκης του OpenMP

```
/* *****  
void omp_set_num_threads (int num_threads); /* Affects the number of threads for subsequent parallel regions */  
int omp_get_num_threads (void); /* Returns the number of threads in the current team */  
int omp_get_max_threads (void); /* Returns an upper bound on the number of threads in a new team */  
int omp_get_thread_num (void); /* Returns the thread number of the calling thread in the current team */  
int omp_in_parallel (void); /* Returns true if the active-levels-var ICV is greater than zero */  
void omp_set_dynamic (int dynamic_threads); /* Enables or disables dynamic adjustment of thread number (dyn-var ICV) */  
int omp_get_dynamic (void); /* Returns the value of dyn-var ICV (dynamic adjustment of thread number) */  
int omp_get_cancellation (void); /* Returns the value of cancel-var ICV (cancellation is ON or OFF? ) */  
int omp_get_thread_limit (void); /* Returns the maximum number of OpenMP threads in the current group */  
int omp_get_supported_active_levels (void); /* Returns the number of active levels of parallelism */  
void omp_set_max_active_levels (int max_levels); /* Limits the number of nested active parallel regions */  
int omp_get_max_active_levels (void); /* Returns the value of the max-active-levels-var ICV */  
int omp_get_level (void); /* Returns the value of the levels-var ICV (maximum nested parallel regions) */  
int omp_get_ancestor_thread_num (int level); /* Returns the thread number of the ancestor of the current thread */  
int omp_get_team_size (int level); /* Returns the size of the thread team for the ancestor of the current thread */  
int omp_get_active_level (void); /* Returns the value of the active-levels-var ICV */  
  
void omp_set_schedule (omp_sched_t kind, int chunk_size); /* Sets the applied schedule for 'runtime' policy (run-sched-var) */  
void omp_get_schedule (omp_sched_t *kind, int *chunk_size); /* Returns the applied schedule for 'runtime' policy */  
  
/* *****  
omp_proc_bind_t omp_get_proc_bind (void); /* Returns thread affinity policy for nested parallel regions (no proc_bind) */  
  
typedef enum omp_proc_bind_t {  
    omp_proc_bind_false = 0, /* OpenMP threads may move between OpenMP places and thread affinity is disabled */  
    omp_proc_bind_true = 1, /* OpenMP threads should not moved between OpenMP places and thread affinity is enabled */  
    omp_proc_bind_primary = 2, /* Thread affinity policy is set to primary */  
    omp_proc_bind_close = 3, /* Thread affinity policy is set to close */  
    omp_proc_bind_spread = 4, /* Thread affinity policy is set to spread */  
} omp_proc_bind_t; /* END of enum omp_proc_bind_t */  
  
int omp_get_num_places (void); /* Returns the number of places available to the exec environment */  
int omp_get_place_num_procs (int place_num); /* Returns the number of available CPUs in the specified place */  
void omp_get_place_proc_ids (int place_num, int *ids); /* Returns the IDs of the CPUs available in the specified place */  
int omp_get_place_num (void); /* Returns the place number of the place for the encountering thread */  
int omp_get_partition_num_places (void); /* Returns the place No in the partition of the innermost implicit task */  
void omp_get_partition_place_nums (int *place_nums); /* Returns place number list of the innermost implicit task */  
void omp_set_affinity_format (const char *format); /* Sets the affinity format (the value of affinity-format-var ICV) */  
void omp_display_affinity (const char *format); /* Prints the OpenMP thread affinity information using format spec. */  
  
size_t omp_get_affinity_format (char *buffer, size_t size); /* Returns the affinity-format-var ICV on device */  
size_t omp_capture_affinity (char *buffer, size_t size, const char *format); /* Prints the thread affinity info */
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι συναρτήσεις της βιβλιοθήκης του OpenMP

```

/***** TEAMS REGION ROUTINES *****/
int omp_get_num_teams (void); /* Returns the number of initial teams in the current teams region */
int omp_get_team_num (void); /* Returns the initial team number of the calling thread */
void omp_set_num_teams (int num_teams); /* Sets No of threads for subsequent teams regions (no num_teams clause) */
int omp_get_max_teams (void); /* Returns upper bound on the No of teams created by teams (no num_teams) */
void omp_set_teams_thread_limit (int thread_limit); /* Defines the maximum number of threads in each contention group */
int omp_get_teams_thread_limit (void); /* Returns the maximum number of threads avail. in each contention group */

/***** TASKING ROUTINES *****/
int omp_get_max_task_priority (void); /* Returns the maximum value that can be specified in the priority clause */
int omp_in_explicit_task (void); /* Returns the value of the explicit-task-var ICV */
int omp_in_final (void); /* Returns true if the routine is executed in a final task region ise */

/***** RESOURCE RELINQUISHING ROUTINES *****/
int omp_pause_resource (omp_pause_resource_t kind, int device_num); /* Allows runtime to relinquish resources used on device */
typedef enum omp_pause_resource_t {
    omp_pause_soft = 1, /* Results in a soft pause of resource on the specified device */
    omp_pause_hard = 2, /* Results in h hard pause of resource on the specified device */
} omp_pause_resource_t;

int omp_pause_resource_all (omp_pause_resource_t kind); /* Allows runtime to relinquish resources on all devices */

/***** DEVICE INFORMATION ROUTINES *****/
int omp_get_num_procs (void); /* Returns the number of processors available to the device */
void omp_set_default_device (int device_num); /* Controls the default target device via default-device-var ICV */
int omp_get_default_device (void); /* Returns the default target device */
int omp_get_num_devices (void); /* Returns the number of non-host devices available for offloading */
int omp_get_device_num (void); /* Returns the device number of the device executes the calling thread */
int omp_is_initial_device (void); /* Returns true if the current task is executing on the host device */
int omp_get_initial_device (void); /* Returns a device number that represents the host device */

```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι συναρτήσεις της βιβλιοθήκης του OpenMP

```
/******                                DEVICE MEMORY ROUTINES                                *****/

void* omp_target_alloc (size_t size, int device_num);          /* Allocates memory in a device data environment */
void omp_target_free (void *device_ptr, int device_num);      /* Frees the device memory allocated by omp_target_alloc routine */
int omp_target_is_present (const void *ptr, int device_num);  /* Checks if a host pointer refers to storage mapped to device */
int omp_target_is_accessible (const void *ptr, size_t size,   /* Tests whether host memory is accessible from a given device */
                             int device_num);

/* Copies memory between any combination of host/device pointers */
int omp_target_memcpy (void *dst, const void *src, size_t length, size_t dst_offset, size_t src_offset,
                      int dst_device_num, int src_device_num);

/* Copies a rectangular subvolume from a multidimensional array to another multidimensional array */
int omp_target_memcpy_rect (void *dst, const void *src, size_t element_size, int num_dims, const size_t *volume,
                           const size_t *dst_offsets, const size_t *src_offsets, const size_t *dst_dims, const size_t *src_dims,
                           int dst_device_num, int src_device_num);

/* Asynchronous copy between any combination of host and device pointers */
int omp_target_memcpy_async (void *dst, const void *src, size_t len, size_t dst_offset, size_t src_offset, int dst_device_num,
                             int src_device_num, int depobj_count, omp_depend_t *depobj_list);

/* Asynchronous copy between any combination of host and device pointers */
int omp_target_memcpy_rect_async (void *dst, const void *src, size_t element_size, int num_dims, const size_t *volume,
                                  const size_t *dst_offsets, const size_t *src_offsets, const size_t *dst_dimensions, const size_t *src_dims,
                                  int dst_device_num, int src_device_num, int depobj_count, omp_depend_t *depobj_list);

/* Maps a device pointer (returned from omp_target_alloc or implementation-defined runtime routines) to a host pointer */
int omp_target_associate_ptr (const void *host_ptr, const void *device_ptr, size_t size, size_t device_offset, int device_num);

int omp_target_disassociate_ptr (const void *ptr, int device_num); /* Removes the pointer for a given device from host pointer */
void * omp_get_mapped_ptr (const void *ptr, int device_num); /* Returns the device pointer associated with a host pointer */
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι συναρτήσεις της βιβλιοθήκης του OpenMP

```
/* ***** LOCK ROUTINES ***** */
void omp_init_lock (omp_lock_t *lock); /* Initialize an OpenMP lock without a hint */
void omp_init_nest_lock (omp_nest_lock_t *lock); /* Initialize an OpenMP nestable lock without a hint */
void omp_init_lock_with_hint(omp_lock_t *lock, /* Initialize an OpenMP lock with a hint */
omp_sync_hint_t hint);
void omp_init_nest_lock_with_hint (omp_nest_lock_t *lock, /* Initialize an OpenMP nestable lock with a hint. */
omp_sync_hint_t hint);
void omp_destroy_lock (omp_lock_t *lock); /* Ensures that the OpenMP lock is uninitialized */
void omp_destroy_nest_lock (omp_nest_lock_t *lock); /* Ensures that the OpenMP nestable lock is uninitialized */
void omp_set_lock (omp_lock_t *lock); /* Provides a means of setting an OpenMP lock */
void omp_set_nest_lock (omp_nest_lock_t *lock); /* Provides a means of setting an OpenMP nestable lock */
void omp_unset_lock (omp_lock_t *lock); /* Provides the means of unsetting an OpenMP lock */
void omp_unset_nest_lock (omp_nest_lock_t *lock); /* Provides the means of unsetting an OpenMP nestable lock */
int omp_test_lock (omp_lock_t *lock); /* Attempts to set an OpenMP lock but do not suspend execution of the task */
int omp_test_nest_lock (omp_nest_lock_t *lock); /* Attempts to set an OpenMP nestable lock but do not suspend execution */

/* ***** TIMING ROUTINES ***** */
double omp_get_wtime (void); /* Returns elapsed wall clock time in seconds */
double omp_get_wtick (void); /* Returns the precision of the timer used by omp_get_wtime */

/* ***** EVENT ROUTINE ***** */
void omp_fulfill_event(omp_event_handle_t event); /* Fulfills and destroys an OpenMP event */

/* ***** INTEROPERABILITY ROUTINES ***** */
int omp_get_num_interop_properties (const omp_interop_t interop); /* Retrieves number of properties for omp_interop_t objects */
omp_intptr_t omp_get_interop_int (const omp_interop_t interop, /* An integer property from an omp_interop_t object */
omp_interop_property_t property_id, int *ret_code);
void* omp_get_interop_ptr (const omp_interop_t interop, /* Retrieves a pointer property from an omp_interop_t object */
omp_interop_property_t property_id, int *ret_code);
const char* omp_get_interop_str (const omp_interop_t interop, /* Retrieves a string property from an omp_interop_t object */
omp_interop_property_t property_id, int *ret_code);
const char* omp_get_interop_name (const omp_interop_t interop, /* Retrieves a property name from an omp_interop_t object */
omp_interop_property_t property_id);
const char* omp_get_interop_type_desc (const omp_interop_t interop, /* A description of property type for omp_interop_t objects */
omp_interop_property_t property_id);
const char* omp_get_interop_rc_desc (const omp_interop_t interop, /* A description of return code for omp_interop_t objects */
omp_interop_rc_t ret_code);
```



Κεφάλαιο 4: Προγραμματισμός σε συστήματα κοινόχρηστης μνήμης

Το περιβάλλον προγραμματισμού OpenMP

Οι συναρτήσεις της βιβλιοθήκης του OpenMP

```
/****** MEMORY MANAGEMENT ROUTINES *****/

typedef enum omp_alloctrail_key_t {
    omp_atk_sync_hint = 1, omp_atk_alignment = 2, omp_atk_access = 3, omp_atk_pool_size = 4, omp_atk_fallback = 5,
    omp_atk_fb_data = 6, omp_atk_pinned = 7, omp_atk_partition = 8 } omp_alloctrail_key_t;

typedef enum omp_alloctrail_value_t {
    omp_atv_false = 0, omp_atv_true = 1, omp_atv_contended = 3, omp_atv_uncontended = 4, omp_atv_serialized = 5,
    omp_atv_private = 6, omp_atv_all = 7, omp_atv_thread = 8, omp_atv_pteam = 9, omp_atv_cgroup = 10,
    omp_atv_default_mem_fb = 11, omp_atv_null_fb = 12, omp_atv_abort_fb = 13, omp_atv_allocator_fb = 14,
    omp_atv_environment = 15, omp_atv_nearest = 16, omp_atv_blocked = 17, omp_atv_interleaved = 18 } omp_alloctrail_value_t;

typedef struct omp_alloctrail_t {
    omp_alloctrail_key_t key; omp_uintptr_t value; } omp_alloctrail_t;

omp_allocator_handle_t omp_init_allocator (omp_memspace_handle_t memspace, int ntraits,
    const omp_alloctrail_t traits[]); /* Initializes allocator & associates it with memory space */
void omp_destroy_allocator (omp_allocator_handle_t allocator); /* Releases all resources used by allocator handle */
void omp_set_default_allocator (omp_allocator_handle_t allocator); /* Sets the default memory allocator and allocate clauses */
omp_allocator_handle_t omp_get_default_allocator (void); /* Returns handle the memory allocator and allocate clauses */
void *omp_alloc (size_t size, omp_allocator_handle_t allocator); /* Requests a memory allocation from a memory allocator */
void *omp_aligned_alloc (size_t alignment, size_t size,
    omp_allocator_handle_t allocator); /* Requests a memory allocation from a memory allocator */
void omp_free (void *ptr, omp_allocator_handle_t allocator); /* Deallocates previously allocated memory */
void *omp_calloc (size_t nmemb, size_t size,
    omp_allocator_handle_t allocator); /* Requests a zero initialized memory allocation */
void *omp_aligned_calloc (size_t alignment, size_t nmemb,
    size_t size, omp_allocator_handle_t allocator); /* Requests a zero initialized memory allocation */
void *omp_realloc (void *ptr, size_t size,
    omp_allocator_handle_t allocator,
    omp_allocator_handle_t free_allocator); /* Deallocates allocated memory, requests memory allocation */

/****** TOOL CONTROL ROUTINES *****/

int omp_control_tool(int command, int modifier, void *arg); /* Enables a program to pass commands to an active tool */

/****** ENVIRONMENTAL DISPLAY ROUTINE *****/

void omp_display_env (int verbose); /* Displays OpenMP version and initial values of some ICVs */
```





Το παρόν συνοδευτικό έργο **παρέχεται** αποκλειστικά και μόνο στους διδάσκοντες που έχουν επιλέξει το αντίστοιχο βιβλίο μέσω του συστήματος του **Ευδόξου** ως διδακτικό σύγγραμμα για τη διδασκαλία των μαθημάτων τους και την αξιολόγηση της εκμάθησης των φοιτητών και για όσο χρονικό διάστημα διατηρείται η επιλογή του συγκεκριμένου συγγράμματος. Η **διάδοση, δημοσίευση, αναπαραγωγή ή πώληση** οπουδήποτε μέρους αυτού του έργου με οποιοδήποτε μέσο καταστρέφει την ακεραιότητα του έργου **και για σκοπούς διαφορετικούς από αυτούς για τους οποίους έχει χορηγηθεί η σχετική άδεια δεν επιτρέπεται**. Το περιεχόμενο του έργου **δεν θα πρέπει να διατίθεται** στους φοιτητές παρά μόνο όταν αυτό αναφέρεται ρητά ότι μπορεί να γίνει. Η **χρήση** του παρόντος έργου γίνεται αποκλειστικά από τον διδάσκοντα στα πλαίσια των εκπαιδευτικών καθηκόντων του και με τη χρήση των μέσων που αυτός διαχειρίζεται και έχει στη διάθεσή του από τις **επίσημες υπηρεσίες του εκπαιδευτικού ιδρύματος** όπου ανήκει, διασφαλίζοντας τη μη περαιτέρω διάδοση, δημοσίευση και αναπαραγωγή του έργου προς τρίτους. Ο διδάσκων δύναται να **προσαρμόζει** μέρος του υλικού των διαφανειών σύμφωνα με τις διδακτικές του ανάγκες, αναφερόμενος όμως πάντοτε στην αρχική πηγή αναφοράς. Το παρόν έργο προστατεύεται από την ελληνική και ευρωπαϊκή νομοθεσία περί πνευματικής ιδιοκτησίας.



Επιστημονικές Εκδόσεις

T Z I O Λ Α